

**UNIVERSIDADE DO VALE DO ITAJAÍ
ESCOLA POLITÉCNICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO**

**ANÁLISE DE TINYML COM ACELERAÇÃO EM HARDWARE EM
SISTEMAS EMBARCADOS**

por

Luiz Renato Henning Carneiro

Itajaí (SC), junho de 2026

**UNIVERSIDADE DO VALE DO ITAJAÍ
ESCOLA POLITÉCNICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO**

**ANÁLISE DE TINYML COM ACELERAÇÃO EM HARDWARE EM
SISTEMAS EMBARCADOS**

Área de Sistemas Embarcados

por

Luiz Renato Henning Carneiro

Relatório apresentado à Banca Examinadora do
Trabalho Técnico-científico de Conclusão de
Curso de Engenharia de Computação para análise
e aprovação.

Orientador(a): Felipe Viel, Dr

Itajaí (SC), junho de 2026

RESUMO

A execução local de modelos de inteligência artificial em dispositivos embarcados tem se tornado uma alternativa relevante para aplicações de sensoriamento remoto, especialmente em cenários com restrições de comunicação, latência e consumo energético. Nesse contexto, o Tiny Machine Learning (TinyML) permite adaptar modelos de aprendizado de máquina para plataformas com recursos limitados, viabilizando a inferência próxima à fonte de aquisição dos dados. Este trabalho apresenta a implementação e a avaliação comparativa de soluções TinyML para classificação de imagens multiespectrais do dataset SAT-6 em três plataformas embarcadas com características distintas: NVIDIA Jetson Orin Nano, MAX78000 e PYNQ-Z2 com FINN. A metodologia contemplou a preparação e normalização dos dados, treinamento e adaptação dos modelos, quantização conforme as restrições de cada plataforma, implementação dos fluxos de inferência e avaliação experimental por meio de acurácia, matriz de confusão, latência, vazão, consumo energético e complexidade de implementação. Na Jetson Orin Nano, foi utilizado um fluxo em Python com TensorFlow/Keras, explorando a flexibilidade da GPU embarcada. Na MAX78000, o modelo foi convertido para firmware em C e validado em hardware real por meio de uma abordagem hardware-in-the-loop. Na PYNQ-Z2, foi empregado o fluxo Brevitas/QONNX/FINN para geração de acelerador em FPGA a partir de modelo quantizado. Os resultados indicaram acurácia de 99,34% na Jetson Orin Nano, 94,70% na MAX78000 e 93,50% na validação física complementar da PYNQ-Z2. A Jetson apresentou o melhor desempenho geral em acurácia e vazão, a MAX78000 destacou-se pelo baixo consumo de potência, e a PYNQ-Z2 evidenciou o potencial de customização de hardware por meio do FINN. Conclui-se que a escolha da plataforma TinyML depende diretamente do compromisso entre acurácia, consumo energético, latência, flexibilidade de software e complexidade de implementação.

Palavras-chave: TinyML; Sensoriamento Remoto; Sistemas Embarcados; Quantização; FINN; MAX78000; PYNQ-Z2; Jetson Orin Nano.

ABSTRACT

The local execution of artificial intelligence models on embedded devices has become a relevant alternative for remote sensing applications, especially in scenarios with communication, latency and energy consumption constraints. In this context, Tiny Machine Learning (TinyML) enables the adaptation of machine learning models to resource-constrained platforms, allowing inference to be performed close to the data acquisition source. This work presents the implementation and comparative evaluation of TinyML solutions for the classification of multispectral images from the SAT-6 dataset on three embedded platforms with distinct characteristics: NVIDIA Jetson Orin Nano, MAX78000 and PYNQ-Z2 with FINN. The methodology included data preparation and normalization, model training and adaptation, quantization according to the restrictions of each platform, implementation of the inference flows and experimental evaluation based on accuracy, confusion matrix, latency, throughput, energy consumption and implementation complexity. On the Jetson Orin Nano, a Python-based flow with TensorFlow/-Keras was used, exploring the flexibility of the embedded GPU. On the MAX78000, the model was converted into C firmware and validated on real hardware using a hardware-in-the-loop approach. On the PYNQ-Z2, the Brevitas/QONNX/FINN flow was adopted to generate an FPGA accelerator from a quantized model. The results showed an accuracy of 99.34% on the Jetson Orin Nano, 94.70% on the MAX78000 and 93.50% in the complementary physical validation of the PYNQ-Z2. The Jetson achieved the best overall performance in terms of accuracy and throughput, the MAX78000 stood out for its low power consumption, and the PYNQ-Z2 demonstrated the potential of hardware customization through FINN. It is concluded that the choice of a TinyML platform depends directly on the trade-off among accuracy, energy consumption, latency, software flexibility and implementation complexity.

Keywords: TinyML; Remote Sensing; Embedded Systems; Quantization; FINN; MAX78000; PYNQ-Z2; Jetson Orin Nano.

LISTA DE ILUSTRAÇÕES

Figura 1 – Relação entre Inteligência Artificial, Aprendizado de Máquina e Aprendizado Profundo.	20
Figura 2 – Aprendizado de Máquina x Aprendizado profundo.	21
Figura 3 – Aprendizado Supervisionado.	22
Figura 4 – Não Supervisionado.	23
Figura 5 – Aprendizado por Reforço.	23
Figura 6 – TinyML = Machine Learning + Sistemas Embarcados.	25
Figura 7 – Diagrama de um sistema SoC-FPGA baseado na plataforma Zynq, ilustrando a integração entre CPU e lógica reconfigurável para aceleração de redes neurais.	31
Figura 8 – Kit de desenvolvimento Jetson Orin Nano.	33
Figura 9 – Microcontrolador MAX78000 com acelerador neural integrado	34
Figura 10 – Arquiteturas CNN para detecção de nuvens em CubeSats.	36
Figura 11 – Arquitetura de sistema para detecção de objetos em dispositivo de borda. . .	37
Figura 12 – Super-rede KWS baseada em Once-For-All para o MAX78000.	37
Figura 13 – Visão geral do projeto proposto.	42
Figura 14 – Topologias CNN de referência para classificação de imagens com o framework FINN.	47
Figura 15 – Exemplos de imagens RGB e NIR do SAT-6.	52
Figura 16 – Fluxo de implementação na PYNQ-Z2 com FINN.	53
Figura 17 – Fluxo de implementação na NVIDIA Jetson Orin Nano.	54
Figura 18 – Fluxo de implementação na MAX78000.	55
Figura 19 – Plataformas embarcadas utilizadas nos experimentos.	62
Figura 20 – Visão geral do fluxo de implementação nas plataformas avaliadas.	63
Figura 21 – Plataforma MAX78000 utilizada na validação embarcada.	67
Figura 22 – Plataforma utilizada na implementação com NVIDIA Jetson Orin Nano. . .	70
Figura 23 – Matriz de confusão normalizada obtida na NVIDIA Jetson Orin Nano. . . .	84
Figura 24 – Benchmark da NVIDIA Jetson Orin Nano.	86
Figura 25 – Matriz de confusão obtida na validação HIL da MAX78000.	88
Figura 26 – Acurácia por classe obtida na validação da MAX78000.	89
Figura 27 – Montagem experimental para medição de consumo da MAX78000.	92
Figura 28 – Matriz de confusão obtida na execução física da PYNQ-Z2 com t_{2w8_500fps}	95

LISTA DE TABELAS

Tabela 1 – Comparativo entre trabalhos relacionados e o presente trabalho.	40
Tabela 2 – Matriz de confusão obtida na NVIDIA Jetson Orin Nano.	83
Tabela 3 – Métricas por classe obtidas na NVIDIA Jetson Orin Nano.	84
Tabela 4 – Benchmark de inferência na NVIDIA Jetson Orin Nano.	85
Tabela 5 – Resumo dos resultados obtidos na NVIDIA Jetson Orin Nano.	86
Tabela 6 – Métricas por classe obtidas na validação da MAX78000.	89
Tabela 7 – Medições de latência, vazão e eficiência energética obtidas na MAX78000. .	90
Tabela 8 – Resultados de desempenho obtidos no fluxo FINN.	93
Tabela 9 – Resultados de área Vivado/OOC para as configurações geradas pelo FINN. .	93
Tabela 10 – Resultado da execução física complementar na PYNQ-Z2	94
Tabela 11 – Acurácia por classe obtida na execução física da PYNQ-Z2	95
Tabela 12 – Comparação geral dos resultados obtidos nas plataformas avaliadas.	96

LISTA DE QUADROS

Quadro 1 – Requisitos funcionais	44
Quadro 2 – Requisitos não funcionais	45
Quadro 3 – Arquitetura CNN de referência para o dataset SAT-6	48
Quadro 4 – Descrição do conjunto de dados SAT-6	51
Quadro 5 – Verificação dos requisitos funcionais	57
Quadro 6 – Verificação dos requisitos não funcionais	58
Quadro 7 – Hardware utilizado no desenvolvimento e validação experimental	59
Quadro 8 – Software e ferramentas utilizados	61
Quadro 9 – Resumo da implementação na MAX78000	69
Quadro 10 – Resumo da implementação na NVIDIA Jetson Orin Nano	74
Quadro 11 – Resumo do fluxo FINN voltado à PYNQ-Z2	80
Quadro 12 – Resumo da validação HIL na MAX78000	87

LISTA DE ABREVIATURAS E SIGLAS

AM	Aprendizado de Máquina
FINN	framework open-source
CSV	valores separados por vírgula
CNN	Rede Neural Convolucional (Convolutional Neural Network)
CPU	Unidade Central de Processamento (Central Processing Unit)
DL	Aprendizado Profundo (Deep Learning)
DPU	Unidade de Processamento de Aprendizado Profundo
Edge AI	Inteligência Artificial na Borda
FPGA	Field-Programmable Gate Array
FP32	Ponto flutuante de 32 bits
FPS	Quadros por segundo
GPU	Unidade de Processamento Gráfico (Graphics Processing Unit)
HIL	Hardware in the Loop
JetPack	Pacote de software da NVIDIA para plataformas Jetson
I/O	Entrada e Saída
IA	Inteligência Artificial
INT8	Inteiros de 8 bits
IoT	Internet das Coisas
KWS	Reconhecimento de Palavras-Chave (Keyword Spotting)
MCU	Microcontrolador
MFCCS	Coeficientes Cepstrais de Frequência Mel
ML	Aprendizado de Máquina (Machine Learning)
MPU	Microprocessador
NAS	Busca de Arquitetura Neural
NIR	Infravermelho próximo

NPU	Unidade de Processamento Neural (Neural Processing Unit)
OFA	Once-For-All
PLN	Processamento de Linguagem Natural
PTQ	Quantização Pós-Treinamento
QAT	Treinamento Consciente de Quantização
RF	Requisito Funcional
RNF	Requisito Não Funcional
QONNX	Open Neural Network Exchange
SoC	System-on-a-Chip
SDK	Kit de Desenvolvimento de Software
SWaP	Tamanho, Peso e Potência (Size, Weight and Power)
TCC	Trabalho de Conclusão de Curso
TinyML	Tiny Machine Learning
ViT	Vision Transformers
VDD_IN	Tensão/potência de entrada medida no sistema de alimentação
LUTs	Tabelas de consulta usadas em FPGAs
FFs	elementos de armazenamento lógico em FPGAs
BRAMs	blocos de memória interna do FPGA
DSPs	blocos de processamento de sinais digitais
WNS	Worst Negative Slack (margem de temporização negativa)

SUMÁRIO

1	INTRODUÇÃO	13
1.1	Problematização	15
1.1.1	Formulação do Problema	15
1.1.2	Solução Proposta	16
1.2	Objetivos	16
1.2.1	Objetivo Geral	17
1.2.2	Objetivos Específicos	17
1.2.3	Justificativa	17
1.3	Metodologia	18
1.4	Estrutura do Trabalho	18
2	FUNDAMENTAÇÃO TEÓRICA	20
2.1	A Relação entre Inteligência Artificial, Aprendizado de Máquina e Apre-	
	ndizado Profundo	20
2.1.1	Tipos de Aprendizado de Máquina:	21
2.1.2	Aplicações de Aprendizado de Máquina	24
2.2	Inteligência Artificial na Borda	24
2.2.1	Edge AI e TinyML: Conceitos e Vantagens	24
2.2.2	Treinamento ciente de quantização	26
2.2.3	Sistemas Embarcados e Computação de Borda	28
2.2.4	Tecnologias de Processadores e Circuitos	28
2.2.5	Framework FINN para aceleração de redes quantizadas em FPGA	29
2.2.6	Aceleração em Hardware para IA Embarcada	30
2.2.6.1	GPU embarcada	32
2.2.6.2	Aceleradores neurais dedicados em SoCs e microcontroladores	33
2.3	Trabalhos Relacionados	35
2.3.1	Aceleração de CNNs em FPGA para detecção de nuvens em CubeSats	35
2.3.2	BED: sistema de detecção de objetos para dispositivos de borda	36
2.3.3	Co-otimização de hardware e software para reconhecimento de palavras-chave	37
2.3.4	HAViTHI: acelerador baseado em Transformer para classificação de imagens hiperespectrais	38
2.3.5	Análise Comparativa	39
3	PROJETO	41
3.1	Visão Geral e Premissas	41
3.1.1	Premissas	42
3.2	Análise de requisitos	43
3.3	Delimitação de escopo	45

3.4	Modelos e arquiteturas adotadas	46
3.4.1	Arquiteturas CNN de referência para o SAT-6	46
3.4.2	Modelos utilizados nas plataformas avaliadas	48
3.4.3	Treinamento e preparação dos modelos	49
3.4.3.1	Preparação para NVIDIA Jetson Orin Nano	49
3.4.3.2	Preparação para PYNQ-Z2 com FINN/Brevitas	50
3.4.3.3	Preparação para MAX78000	50
3.4.4	Conjunto de dados	51
3.5	Fluxos de implementação nas plataformas embarcadas	52
3.5.1	Fluxo de implementação na PYNQ-Z2 com FINN	52
3.5.2	Fluxo de implementação na NVIDIA Jetson Orin Nano	54
3.5.3	Fluxo de implementação na MAX78000	54
3.6	Plano de verificação	55
3.7	Metodologia experimental	58
3.7.1	Materiais	59
3.7.2	Hardware	59
3.7.3	Software e ferramentas	60
4	IMPLEMENTAÇÃO	62
4.1	Visão geral da implementação	63
4.2	Conjunto de dados e preparação das amostras	64
4.3	Modelos implementados	65
4.4	Implementação na plataforma MAX78000	65
4.4.1	Adequação do modelo ao acelerador CNN	66
4.4.2	Geração e gravação do firmware	66
4.4.3	Validação por teste <i>hardware-in-the-loop</i>	67
4.4.4	Registro das predições e preparação das métricas	68
4.4.5	Resumo da implementação na MAX78000	68
4.5	Implementação na plataforma NVIDIA Jetson Orin Nano	69
4.5.1	Preparação do ambiente	70
4.5.2	Preparação das amostras para inferência	70
4.5.3	Execução da inferência na Jetson	71
4.5.4	Medição de latência e consumo de energia	73
4.5.5	Resumo da implementação na Jetson	73
4.6	Fluxo FINN na plataforma PYNQ-Z2	74
4.6.1	Organização do fluxo <code>sat6_cnn</code>	75
4.6.2	Preparação do dataset para o fluxo FINN	75
4.6.3	Definição das CNNs quantizadas	75
4.6.4	Treinamento e validação inicial em software	77
4.6.5	Exportação dos modelos para QONNX	77

4.6.6	Transformações e geração dos aceleradores com FINN	78
4.6.7	Relatórios de desempenho e área	78
4.6.8	Artefatos gerados pelo fluxo	79
4.6.9	Resumo do fluxo FINN	79
5	TESTES E RESULTADOS	81
5.1	CrITÉrios de avaliaÇ��o	81
5.2	Resultados na NVIDIA Jetson Orin Nano	82
5.2.1	Desempenho de classifica��o	83
5.2.2	Lat��ncia, vaz��o e consumo de energia	85
5.2.3	S��ntese dos resultados da Jetson	86
5.3	Resultados na MAX78000	87
5.3.1	Lat��ncia, vaz��o e consumo de energia	90
5.4	Resultados na PYNQ-Z2 com FINN	92
5.4.1	Resultados de desempenho do fluxo FINN	93
5.4.2	Resultados de ��rea Vivado/OOC	93
5.4.3	Valida���o f��sica complementar com t2w8_500fps	94
5.5	Compara���o entre as plataformas	96
5.6	Discuss��o geral dos resultados	97
5.7	Limita���es dos resultados	98
5.8	S��ntese dos resultados	99
6	CONSIDERA���ES FINAIS	101
6.1	Trabalhos futuros	101
	REFER��NCIAS	103

1 INTRODUÇÃO

A crescente demanda por soluções de sensoriamento remoto em tempo real em áreas como agricultura de precisão, monitoramento ambiental e cidades inteligentes estimula o desenvolvimento de sistemas embarcados capazes de processar dados localmente. Essa abordagem reduz latência, consumo de energia e dependência da nuvem (SINGH et al., 2023). O Tiny Machine Learning (TinyML) se destaca como alternativa promissora ao possibilitar a execução de modelos de aprendizado de máquina em dispositivos com recursos restritos. Revisões recentes, incluindo uma análise conforme diretrizes prisma de 66 experimentos com ML em FPGAs para sensoriamento remoto, ressaltam a eficiência de arquiteturas embarcadas nessa área (LÉONARD; STÖBER; SCHULZ, 2025). Estudos e levantamentos também indicam que a inferência na borda pode superar abordagens baseadas em nuvem em cenários com restrições de largura de banda e necessidade de resposta imediata (SINGH et al., 2023; ALSAADE et al., 2024).

Dentre plataformas de prateleira, destacam-se: MAX78000: microcontrolador com acelerador CNN dedicado. (Analog Devices, 2020; GONG et al., 2023; Analog Devices, 2021). NVIDIA Jetson: suporta frameworks populares (TensorFlow, PyTorch, TensorRT) com APIs Python e C++, sendo eficaz em aplicações que exigem aceleração e flexibilidade energética, embora com consumo superior ao dos MCUs otimizados; possui modos de potência de 5 W e 10 W (NVIDIA, 2019). Xilinx Pynq + FINN: permite implantação ágil de redes neurais quantizadas em FPGA, alcançando alta eficiência em tarefas críticas (UMUROGLU et al., 2017). Modelos compactos como *TinyissimoYOLO* demonstraram detecção de múltiplas classes com alta taxa de quadros (até 180 fps) e consumo de apenas 196 μ J por inferência no MAX78000 (MOOSMANN et al., 2023), enquanto versões em GAP9 mostraram desempenho ainda melhor ($\approx 150 \mu$ J e 2,12 ms) (MOOSMANN et al., 2024).

No contexto agrícola, propostas recentes empregam *knowledge distillation* para transferir conhecimento de modelos maiores para modelos leves (por exemplo, MobileNetV3), mantendo alta acurácia com redução de complexidade/latência (HUANG et al., 2023; ZHANG et al., 2024; HIMEUR et al., 2024), demonstrando o potencial de TinyML para diagnóstico de doenças e monitoramento em campo.

Apesar desses avanços, persistem lacunas na literatura: faltam comparações diretas entre plataformas (MCUs, FPGAs, SoCs) quanto a consumo energético, latência, acurácia e viabilidade em cenários reais de sensoriamento. Esses estudos são necessários para validar a adequação de certas arquiteturas a aplicações específicas. O trabalho, portanto, tem como objetivo investigar a aplicação de Tiny Machine Learning (TinyML) em dispositivos embarcados de baixo consumo energético, com foco na tarefa de classificação de imagens multiespectrais no contexto de sensoriamento remoto. A análise foi realizada com base no dataset SAT-6,

que contém imagens multiespectrais amplamente utilizadas para estudos de uso e cobertura do solo (BASU et al., 2015). foram empregados modelos de aprendizado profundo previamente treinados e otimizados com técnicas de quantização, adaptando-os para execução local em dispositivos com restrições de memória, processamento e energia. Três plataformas embarcadas foram comparadas: MAX78000, NVIDIA Jetson e Xilinx Pynq com FINN. Cada uma representa diferentes abordagens arquiteturais de microcontrolador, sistema com GPU e FPGA, respectivamente.

O escopo do trabalho está delimitado às seguintes etapas: Comparação técnica das plataformas selecionadas quanto à compatibilidade com modelos otimizados para TinyML. Implantação de um modelo pré-treinado adaptado a cada dispositivo, utilizando o dataset SAT-6 como base de teste (BASU et al., 2015). Avaliação e comparação das plataformas com base em três métricas principais: acurácia de classificação, tempo de inferência (latência) e consumo energético.

Com o avanço da Internet das Coisas (IoT) e a crescente demanda por soluções inteligentes em tempo real, especialmente em setores como agricultura de precisão, monitoramento ambiental e cidades inteligentes, surge a necessidade de sistemas embarcados capazes de processar dados localmente com eficiência energética e baixa latência cenário típico em que TinyML se encaixa (SINGH et al., 2023). Este trabalho concentra-se na aplicação de TinyML para a classificação de imagens multiespectrais em cenários de sensoriamento remoto, utilizando como base o dataset SAT-6 (BASU et al., 2015). A escolha por imagens multiespectrais visa aumentar a acurácia dos modelos em aplicações ambientais, em comparação a imagens RGB convencionais.

Apesar dos avanços recentes, são escassos os estudos comparativos que avaliem o desempenho de diferentes plataformas embarcadas frente à execução de modelos otimizados em contextos computacionalmente restritos, sobretudo com dados multiespectrais. Assim, existe uma lacuna relevante no que diz respeito à avaliação de métricas como acurácia, tempo de processamento (inferência) e consumo energético em aplicações práticas de TinyML.

O trabalho propõe uma análise comparativa entre três plataformas embarcadas de prateleira: o microcontrolador MAX78000 dotado de acelerador neural, a plataforma NVIDIA Jetson voltada à inferência de alto desempenho e o Xilinx Pynq com FINN baseado em FPGA e compatível com redes quantizadas (UMUROGLU et al., 2017). Essas arquiteturas foram selecionadas por sua representatividade e diversidade de abordagem frente às restrições comuns em aplicações remotas.

Além disso, a proposta utiliza modelos previamente treinados e otimizados por meio de

técnicas de quantização (LAI; SUDA; CHANDRA, 2018; HIMEUR et al., 2024), garantindo viabilidade de execução local sem a necessidade de reprocessamento completo em cada dispositivo. Essa abordagem torna o escopo adequado aos limites de um trabalho de conclusão de curso, ao mesmo tempo em que contribui tecnicamente com uma análise prática do uso de TinyML para sensoriamento remoto com dados multiespectrais.

1.1 PROBLEMATIZAÇÃO

1.1.1 Formulação do Problema

O problema central deste trabalho está relacionado ao custo computacional e energético envolvido na execução de modelos de inteligência artificial aplicados ao sensoriamento remoto. Plataformas como satélites, CubeSats, veículos aéreos não tripulados e sistemas embarcados de observação da Terra podem gerar grande volume de dados, incluindo imagens multiespectrais e hiperespectrais. Em muitos cenários, esses dados precisam ser analisados rapidamente para apoiar tomadas de decisão, como detecção de alterações no solo, monitoramento ambiental, agricultura de precisão, identificação de áreas urbanas e análise de corpos d'água. A abordagem tradicional consiste em transmitir os dados coletados para uma estação terrestre, servidor local ou ambiente em nuvem, onde ocorre o processamento. Embora essa estratégia permita utilizar infraestrutura computacional mais robusta, ela apresenta limitações importantes quando aplicada a sistemas embarcados e dispositivos de borda.

A primeira limitação está relacionada à latência. O envio dos dados para processamento externo aumenta o tempo entre a aquisição da imagem e a obtenção do resultado. Esse atraso pode ser inadequado para aplicações que exigem resposta rápida ou operação autônoma. A segunda limitação está associada ao consumo de banda de comunicação. Em sistemas de sensoriamento remoto, especialmente em plataformas espaciais ou móveis, o canal de comunicação pode ser limitado, intermitente ou de alto custo operacional. Transmitir grandes volumes de dados brutos pode saturar esse canal e reduzir a eficiência do sistema. A terceira limitação envolve o consumo energético, uma vez que a transmissão contínua de dados pode representar uma parcela significativa do consumo total em dispositivos alimentados por bateria ou submetidos a restrições severas de energia.

Nesse contexto, a execução local da inferência surge como uma alternativa para reduzir a dependência de comunicação com servidores externos. Ao realizar a classificação diretamente no dispositivo de borda, torna-se possível transmitir apenas informações processadas ou resultados relevantes, reduzindo a latência, o volume de dados transmitidos e o consumo associado à comunicação. Entretanto, a implantação de modelos de inteligência artificial em hardware embarcado apresenta desafios próprios, como restrições de memória, capacidade de processamento, formato numérico suportado, consumo energético e compatibilidade com ferramentas

de desenvolvimento. Além disso, diferentes plataformas embarcadas oferecem diferentes compromissos entre desempenho, consumo, flexibilidade e complexidade de implementação. Uma plataforma com GPU embarcada pode oferecer alto desempenho e maior facilidade de desenvolvimento, mas tende a apresentar maior consumo. Um microcontrolador com acelerador neural dedicado pode operar com consumo extremamente reduzido, mas impõe restrições mais rígidas à arquitetura do modelo. Já uma plataforma baseada em FPGA permite maior customização do hardware, mas exige fluxos de desenvolvimento mais complexos, envolvendo quantização, síntese e integração com a lógica programável. Dessa forma, o problema investigado neste trabalho pode ser formulado da seguinte maneira: como diferentes plataformas de hardware embarcado se comportam na execução de modelos de inteligência artificial para classificação de imagens multiespectrais, considerando simultaneamente acurácia, latência, consumo energético ?

1.1.2 Solução Proposta

A solução proposta é a migração da inferência de Inteligência Artificial da nuvem para a borda (Edge AI), especificamente para o dispositivo de aquisição de dados. Este conceito é conhecido como Processamento de Imagem Embarcado (On-Board Image Processing - OBIP). Ao utilizar os princípios do Tiny Machine Learning (TinyML), é possível executar modelos de Deep Learning (como CNNs) otimizados para operar dentro das restrições severas de hardware de baixo consumo. Contudo, a escolha da plataforma de hardware (MCU, GPU, FPGA) impacta drasticamente o desempenho.

O trabalho propõe uma análise comparativa sistemática para avaliar a viabilidade e os trade-offs da execução de uma CNN de classificação de imagens multiespectrais em três paradigmas de hardware embarcado:

- Microcontrolador com Acelerador (MAX78000): Foco em ultra-baixo consumo.
- SoC com GPU (NVIDIA Jetson): Foco em alto desempenho e flexibilidade.
- SoC com FPGA (Xilinx Pynq/FINN): Foco em eficiência e latência customizada.

A solução não é apenas implementar o modelo, mas analisar quantitativamente as métricas de acurácia, latência e consumo energético, fornecendo um panorama claro de qual arquitetura é mais adequada para diferentes cenários de sensoriamento remoto embarcado.

1.2 OBJETIVOS

Esta seção formaliza os objetivos do trabalho, conforme descrito a seguir.

1.2.1 Objetivo Geral

Analisar o desempenho de diferentes computadores embarcados na execução de modelo de inteligência artificial (TinyML) para sensoriamento remoto na borda.

1.2.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- 1) Identificar técnicas de TinyML em tecnologias de circuito e processador para sistemas embarcados com restrições de operação;
- 2) Investigar o desempenho do modelo em TinyML embarcado em cada plataforma alvo;
- 3) Analisar limitações de hardware e software de cada solução para inferência local;
- 4) Disponibilizar implementações funcionais em cada plataforma embarcada alvo.

1.2.3 Justificativa

A relevância deste Trabalho de Conclusão de Curso reside na carência de estudos comparativos diretos que avaliem o desempenho de diferentes paradigmas de hardware de borda sob condições idênticas. Conforme destacado na introdução, persistem lacunas na literatura quanto à comparação de plataformas.

Embora existam estudos focados em otimizações para plataformas específicas por exemplo, a aceleração de CNNs em FPGAs para CubeSats ou a implementação de detecção de objetos no MAX78000, faltam análises que comparem, utilizando o mesmo modelo de rede neural e o mesmo conjunto de dados (SAT-6), as métricas essenciais de acurácia, latência (tempo de inferência) e consumo energético.

Esta análise comparativa é crucial para engenheiros de sistemas embarcados e de computação, pois fornece dados quantitativos para a tomada de decisão no projeto. A escolha entre um MCU de baixo consumo, uma GPU embarcada ou um FPGA reconfigurável envolve trade-offs complexos entre custo, eficiência energética, throughput e complexidade de desenvolvimento. O trabalho visa fornecer os subsídios técnicos para essa escolha em aplicações de Sensoriamento Remoto na borda.

Os resultados obtidos ao longo deste trabalho demonstraram que tal comparação é não apenas possível, mas revela diferenças marcantes entre as plataformas. Enquanto a Jetson oferece acurácia superior e menor latência, a MAX78000 destaca-se pela eficiência energética

extrema, e a PYNQ-Z2 evidencia o potencial do fluxo FINN para geração de aceleradores customizados em FPGA cada uma com vantagens distintas a depender do cenário de aplicação.

1.3 METODOLOGIA

A metodologia deste trabalho caracteriza-se como uma pesquisa aplicada, de natureza experimental e comparativa. O estudo foi conduzido com o objetivo de avaliar a execução de modelos de inteligência artificial em três plataformas embarcadas distintas, utilizando como estudo de caso a classificação de imagens multiespectrais do dataset SAT-6. Inicialmente, foi realizada uma revisão bibliográfica e documental sobre sensoriamento remoto, redes neurais convolucionais, Edge AI, TinyML, quantização e aceleração em hardware. Também foram consultadas as documentações técnicas das plataformas e ferramentas utilizadas, incluindo NVIDIA Jetson Orin Nano, MAX78000, PYNQ-Z2, TensorFlow/Keras, Brevitas, QONNX e FINN.

Em seguida, o dataset SAT-6 foi preparado para a tarefa de classificação, considerando suas seis classes de cobertura terrestre. Os dados foram organizados e adaptados conforme as exigências de cada fluxo de desenvolvimento, uma vez que as plataformas avaliadas possuem diferentes restrições de formato, representação numérica e implantação. Na etapa de desenvolvimento dos modelos, foram utilizadas redes neurais convolucionais compatíveis com a tarefa de classificação. Na Jetson Orin Nano, o modelo foi executado em ambiente embarcado com TensorFlow/Keras. Na MAX78000, a rede foi quantizada e convertida para firmware compatível com o acelerador CNN interno da placa. Na PYNQ-Z2, foi empregado um modelo CNN quantizado com Brevitas, exportado para QONNX e processado pelo framework FINN para geração do acelerador em FPGA.

Após a implantação, as plataformas foram avaliadas experimentalmente por meio de métricas de acurácia, matriz de confusão, latência, vazão e consumo energético, conforme a disponibilidade de medição em cada caso. Por fim, os resultados foram comparados considerando os compromissos entre desempenho, consumo, complexidade de desenvolvimento e adequação das plataformas a aplicações de sensoriamento remoto embarcado.

1.4 ESTRUTURA DO TRABALHO

Este trabalho está organizado em seis capítulos. O Capítulo 1 apresenta a introdução, a contextualização do tema, o problema de pesquisa, a solução proposta, os objetivos, a metodologia adotada e a estrutura do documento.

O Capítulo 2 apresenta a fundamentação teórica e os trabalhos relacionados, abordando os principais conceitos necessários ao desenvolvimento do trabalho, como inteligência artificial, redes neurais convolucionais, Edge AI, TinyML, quantização, aceleração em hardware,

plataformas embarcadas, FPGAs e o framework FINN. O Capítulo 3 descreve o projeto desenvolvido, incluindo a visão geral da solução, as premissas, os requisitos, o escopo, o dataset SAT-6 e os fluxos planejados para as plataformas NVIDIA Jetson Orin Nano, MAX78000 e PYNQ-Z2 com FINN.

O Capítulo 4 apresenta a implementação da solução nas plataformas embarcadas, detalhando a preparação dos dados, o treinamento dos modelos, os processos de quantização, conversão e implantação, além das adaptações realizadas para cada placa. O Capítulo 5 apresenta os resultados experimentais obtidos, considerando métricas como acurácia, matriz de confusão, latência, vazão e consumo energético, conforme a disponibilidade de medição em cada plataforma. Também é realizada a comparação entre as soluções avaliadas, destacando os compromissos entre desempenho, consumo, complexidade de desenvolvimento e adequação ao sensoriamento remoto embarcado. Por fim, o Capítulo 6 apresenta as conclusões do trabalho, retomando os objetivos propostos, as contribuições alcançadas, as limitações encontradas e as possibilidades de trabalhos futuros.

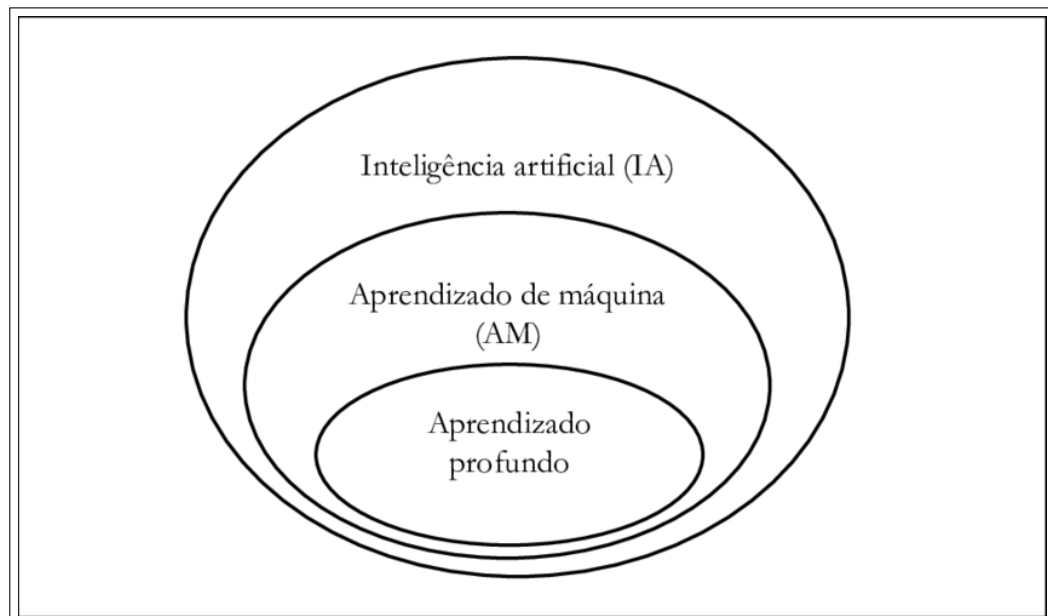
2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo estabelece os fundamentos conceituais necessários para o desenvolvimento da solução proposta. A estrutura está organizada em duas seções principais. A primeira, "Aprendizado de Máquina", aborda os princípios que regem os algoritmos de aprendizado, desde suas classificações fundamentais até a especialização em Aprendizado Profundo e sua aplicação em dispositivos de borda com recursos restritos, conhecida como TinyML. A segunda seção, "Sistemas Embarcados", explora os conceitos de hardware que viabilizam a execução desses algoritmos, detalhando as tecnologias de processamento e as arquiteturas de aceleração, como FPGAs, GPUs e coprocessadores dedicados (NPUs), que são cruciais para a implementação de Inteligência Artificial em ambientes com severas limitações de energia e computação.

2.1 A RELAÇÃO ENTRE INTELIGÊNCIA ARTIFICIAL, APRENDIZADO DE MÁQUINA E APRENDIZADO PROFUNDO

Para compreender o escopo deste trabalho, é fundamental distinguir os conceitos de Inteligência Artificial (IA), Aprendizado de Máquina (ML) e Aprendizado Profundo (Deep Learning). Esses termos são frequentemente usados de forma intercambiável, mas representam áreas com diferentes níveis de especificidade, como ilustrado na Figura abaixo.

Figura 1 – Relação entre Inteligência Artificial, Aprendizado de Máquina e Aprendizado Profundo.

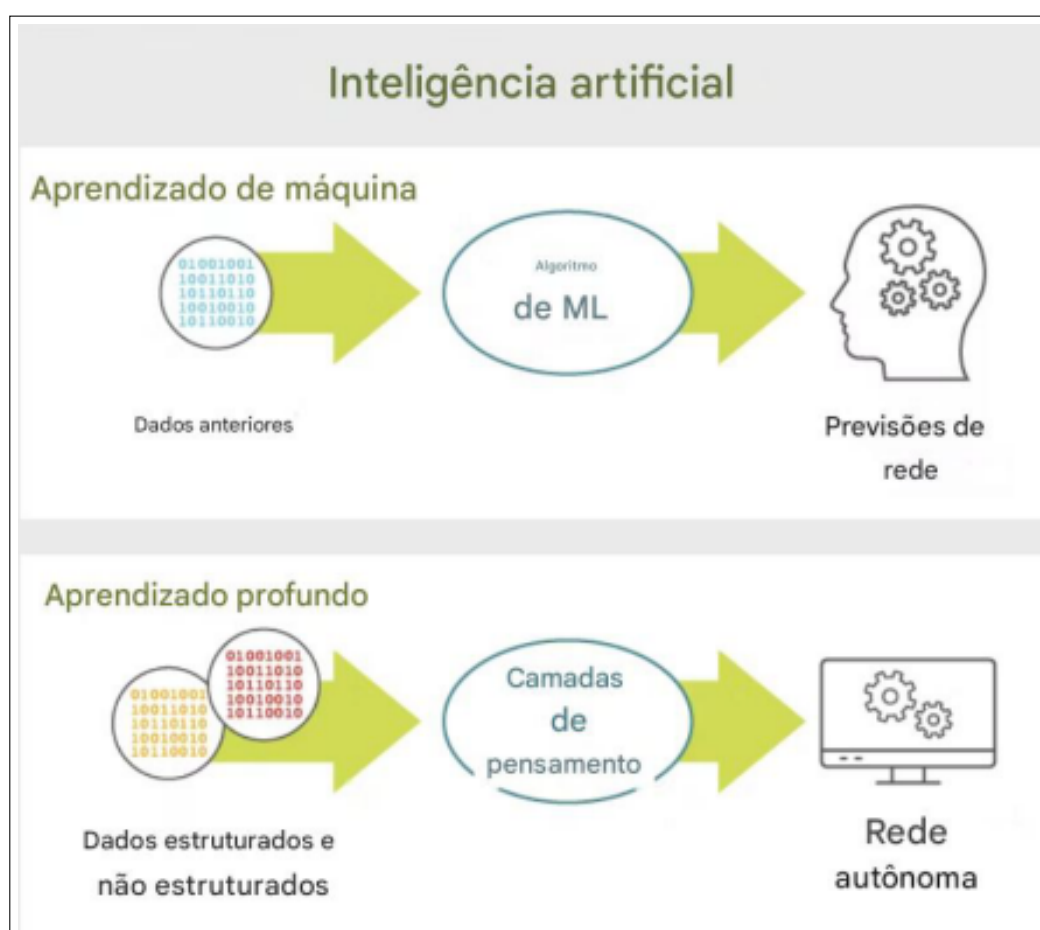


Fonte: Avelar (2025).

A ciência da computação engloba a Inteligência Artificial, cujo objetivo é criar máquinas capazes de simular a inteligência humana, executando tarefas como raciocínio, aprendizado,

percepção e resolução de problemas. O Aprendizado de Máquina (Machine Learning - ML) é um subcampo da IA que se concentra no desenvolvimento de algoritmos que permitem aos sistemas computacionais aprender padrões e tomar decisões a partir de dados, sem serem explicitamente programados para cada cenário possível (GOODFELLOW; BENGIO; COURVILLE, 2016). Em vez de seguir um conjunto fixo de regras, um modelo de ML ajusta seus parâmetros internos com base em exemplos, aprimorando iterativamente seu desempenho em uma tarefa específica. Essa capacidade de generalização para dados nunca vistos é o que confere poder e flexibilidade a essa tecnologia (NIELSEN, 2015). Finalmente, o Aprendizado Profundo (Deep Learning) é um subconjunto especializado do ML, caracterizado pelo uso de redes neurais artificiais com múltiplas camadas profundas para aprender representações de dados em níveis crescentes de abstração.

Figura 2 – Aprendizado de Máquina x Aprendizado profundo.



Fonte: (MACEDO, 2025).

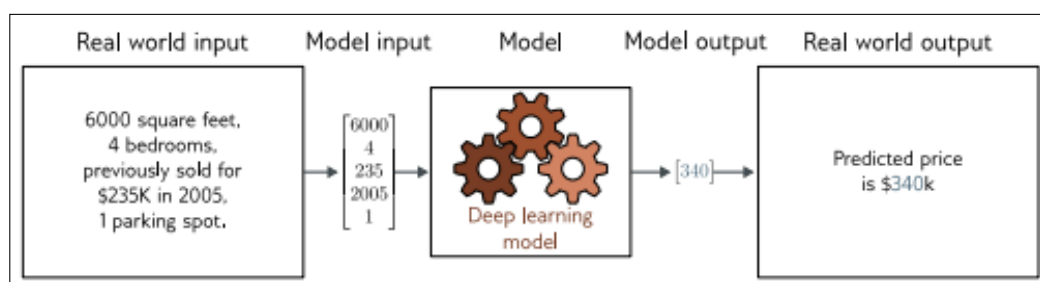
2.1.1 Tipos de Aprendizado de Máquina:

Os algoritmos de Aprendizado de Máquina são classificados em três categorias principais, que se distinguem pela natureza dos dados de treinamento e pelo tipo de feedback fornecido ao modelo durante o processo de aprendizagem (Google Cloud, 2024a; IBM, 2024b). Essa

distinção é crucial, pois determina a abordagem mais adequada para resolver um determinado problema, abaixo seguem tais categorias:

- **Aprendizado Supervisionado:** Nesta abordagem, o algoritmo é treinado com um conjunto de dados de treinamento que é completamente rotulado, ou seja, cada entrada de dados é pareada com uma saída correta correspondente (Google Cloud, 2024a). O objetivo do modelo é aprender uma função de mapeamento que possa prever com precisão a saída para novos dados de entrada que não faziam parte do treinamento. O aprendizado supervisionado abrange duas classes principais de problemas: classificação, onde o objetivo é atribuir uma categoria a uma entrada (por exemplo, classificar um e-mail como "spam" ou "não spam"), e regressão, onde o objetivo é prever um valor contínuo (por exemplo, estimar o preço de uma casa com base em suas características).

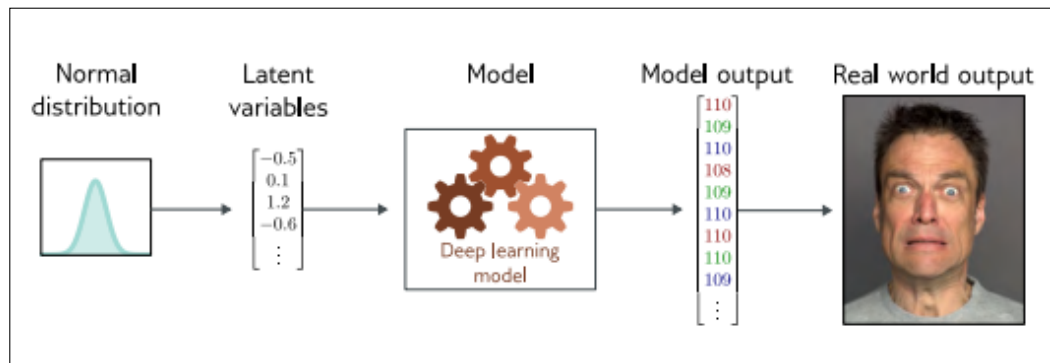
Figura 3 – Aprendizado Supervisionado.



Fonte: Tiwari (2024).

- **Aprendizado Não Supervisionado:** Em contraste com o aprendizado supervisionado, esta modalidade lida com dados que não possuem rótulos pré-definidos (IBM, 2024a). O objetivo do algoritmo é explorar os dados por conta própria para descobrir padrões, estruturas ou agrupamentos ocultos. As tarefas mais comuns incluem o agrupamento (clustering), que visa agrupar dados semelhantes (por exemplo, segmentar clientes em grupos com comportamentos de compra parecidos), e a redução de dimensionalidade, que busca simplificar o conjunto de dados, preservando suas características mais importantes (IBM, 2024a).

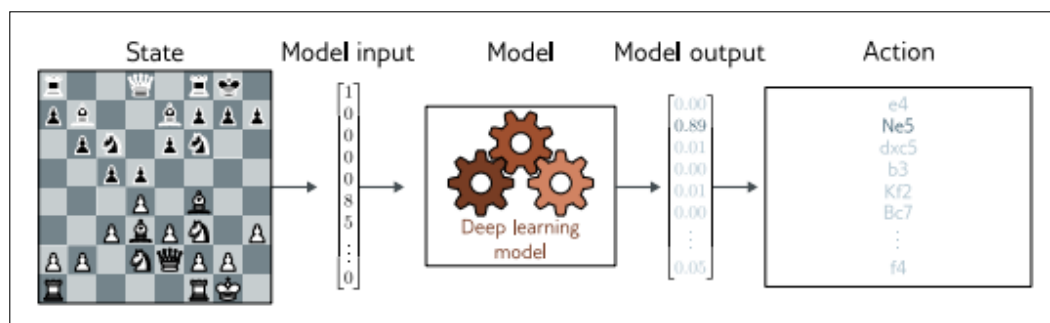
Figura 4 – Não Supervisionado.



Fonte: Tiwari (2024).

- **Aprendizado por Reforço:** Este paradigma de aprendizado é inspirado no comportamento humano e animal, onde um agente (o modelo) aprende a tomar decisões interagindo com um ambiente dinâmico (IBM, 2024b). O agente executa ações no ambiente e, em resposta, recebe um feedback na forma de recompensas (sinais positivos) ou penalidades (sinais negativos) (Oracle, 2024). O objetivo do agente é aprender uma política, ou seja, uma estratégia de tomada de decisão, que maximize a recompensa acumulada ao longo do tempo. Essa abordagem é fundamental para o treinamento de sistemas autônomos, como robôs que aprendem a navegar em um ambiente, algoritmos para jogos estratégicos e sistemas de otimização em tempo real (PEDRO, 2024).

Figura 5 – Aprendizado por Reforço.



Fonte: Tiwari (2024).

Dentro de aprendizado de máquina, o Aprendizado Profundo (Deep Learning) se destaca por utilizar redes neurais artificiais com múltiplas camadas de processamento para aprender representações complexas dos dados. Em redes neurais profundas, as camadas aprendem características em níveis crescentes de abstração e, treinadas de ponta a ponta com grandes volumes de dados, alcançam o estado da arte em visão, fala e PLN (LE-CUN; BENGIO; HINTON, 2015). Esse treinamento é computacionalmente intensivo (retropropagação em milhões de parâmetros), o que historicamente foi viabilizado por *GPUs* devido ao paralelismo massivo (NICKOLLS; DALLY, 2010; HWU; KIRK, 2022).

2.1.2 Aplicações de Aprendizado de Máquina

O impacto do Aprendizado de Máquina é vasto, impulsionando inovações em praticamente todos os setores da indústria e da ciência, desde otimizações em processos de negócios até avanços que transformam a vida cotidiana (Google Cloud, 2024b; Iberdrola, 2024). Algumas das aplicações mais proeminentes incluem:

- **Deteção de Fraudes e Ameaças:** No setor financeiro, algoritmos de ML são essenciais para identificar transações fraudulentas em tempo real, analisando padrões de gastos e detectando anomalias que fogem do comportamento usual do cliente (Google Cloud, 2024b; Alteryx, 2024).
- **Veículos Autônomos:** Carros autônomos dependem intensamente de modelos de deep learning para processar dados de sensores (câmeras, LiDAR, radar), permitindo a detecção de pedestres, outros veículos e sinais de trânsito para uma navegação segura (Iberdrola, 2024).
- **Diagnóstico por Imagem na Medicina:** Redes Neurais Convolucionais (CNNs) são usadas para analisar imagens médicas, como raios-X e ressonâncias magnéticas, auxiliando radiologistas na detecção precoce de doenças como câncer e outras anomalias.
- **Processamento de Linguagem Natural (PLN):** Assistentes virtuais como Siri e Alexa, além de ferramentas de tradução automática, utilizam modelos de ML para compreender e gerar linguagem humana, permitindo uma interação mais natural entre humanos e máquinas.

2.2 INTELIGÊNCIA ARTIFICIAL NA BORDA

Tradicionalmente, os modelos de IA eram executados em servidores de nuvem potentes. No entanto, a crescente proliferação de dispositivos da Internet das Coisas (IoT) e a necessidade de respostas em tempo real impulsionaram o surgimento da IA na Borda (Edge AI) e, mais especificamente, do Tiny Machine Learning (TinyML) (ATLANTICO, 2024; Birchwood University, 2025). A Edge AI representa uma mudança de paradigma fundamental: em vez de enviar dados brutos para a nuvem para serem processados, o processamento e a inferência do modelo de IA ocorrem localmente, diretamente no dispositivo onde os dados são gerados na "borda" da rede.

2.2.1 Edge AI e TinyML: Conceitos e Vantagens

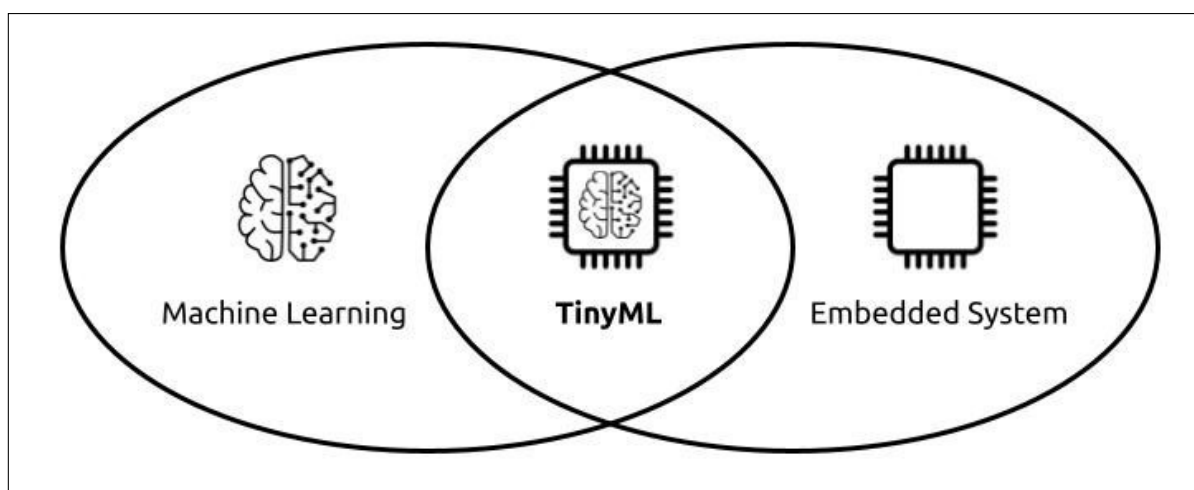
A execução de modelos de inteligência artificial em dispositivos embarcados tem se tornado uma alternativa importante aos sistemas baseados exclusivamente em computação em

nuvem. Nesse contexto, o conceito de *Edge AI* refere-se à execução de algoritmos de inteligência artificial diretamente em dispositivos localizados na borda da rede, como smartphones, câmeras inteligentes, gateways, sistemas embarcados industriais e placas de desenvolvimento. Em vez de enviar continuamente os dados para servidores remotos, o processamento ocorre próximo à fonte de geração dos dados, reduzindo a dependência de conexão com a internet e diminuindo o tempo de resposta do sistema.

O *TinyML*, por sua vez, pode ser entendido como uma área mais específica dentro desse contexto. Enquanto o *Edge AI* pode abranger dispositivos com maior capacidade computacional, como placas com GPU embarcada, o *TinyML* concentra-se na execução de modelos de aprendizado de máquina em microcontroladores e dispositivos com recursos extremamente restritos de memória, processamento e energia. Esses sistemas normalmente operam com consumo reduzido, sendo adequados para aplicações alimentadas por bateria, sensores autônomos e dispositivos de baixo custo (WARDEN; SITUNAYAKE, 2019; KHAN et al., 2024).

A Figura 6 ilustra o *TinyML* como uma área situada na interseção entre aprendizado de máquina e sistemas embarcados. Essa representação é adequada porque o *TinyML* não depende apenas do desenvolvimento de modelos de IA, mas também da adaptação desses modelos às limitações físicas do hardware, como memória, capacidade de processamento, consumo energético e formato numérico suportado pelo dispositivo.

Figura 6 – TinyML = Machine Learning + Sistemas Embarcados.



Fonte: Dabwan et al. (2024).

As principais vantagens da execução local de modelos de IA são a baixa latência, a maior privacidade dos dados, a redução no consumo de banda de rede e a maior confiabilidade em cenários sem conexão constante. A baixa latência ocorre porque a decisão é tomada diretamente no dispositivo, sem depender do envio dos dados para a nuvem. A privacidade é

favorecida porque informações sensíveis podem permanecer no próprio equipamento. Além disso, a redução na transmissão de dados diminui o uso de rede e pode contribuir para maior eficiência energética do sistema (SHI et al., 2016; WARDEN; SITUNAYAKE, 2019).

Apesar dessas vantagens, a implementação de modelos de IA em dispositivos embarcados apresenta desafios significativos. Modelos de *deep learning* geralmente possuem grande quantidade de parâmetros, realizam muitas operações matemáticas e demandam memória considerável. Esses requisitos entram em conflito com as limitações de microcontroladores, FPGAs de baixo custo e demais plataformas embarcadas com restrições de memória, frequência de operação, capacidade de processamento e consumo energético. Dessa forma, torna-se necessário aplicar técnicas de otimização para reduzir o tamanho e a complexidade dos modelos, mantendo uma acurácia aceitável para a aplicação (GHOLAMI et al., 2021; WARDEN; SITUNAYAKE, 2019).

Entre as principais técnicas utilizadas para viabilizar a execução de redes neurais em dispositivos restritos, destacam-se a quantização, a poda e a destilação de conhecimento.

- **Quantização:** reduz a precisão numérica dos pesos e ativações do modelo. Em vez de utilizar números em ponto flutuante de 32 bits, o modelo passa a operar com representações mais compactas, como inteiros de 8 bits ou até precisões menores. Essa redução diminui o tamanho do modelo, reduz o uso de memória e permite operações mais eficientes em hardware embarcado (JACOB et al., 2018; GHOLAMI et al., 2021).
- **Poda:** remove pesos, conexões, neurônios ou canais considerados pouco relevantes para o resultado final da rede. A poda pode ser não estruturada, quando remove pesos individuais, ou estruturada, quando remove blocos inteiros, como filtros ou canais. A poda estruturada tende a ser mais vantajosa em hardware, pois gera reduções mais regulares e fáceis de explorar computacionalmente (BLALOCK et al., 2020).
- **Destilação de conhecimento:** utiliza um modelo maior, chamado de professor, para auxiliar o treinamento de um modelo menor, chamado de aluno. O modelo aluno aprende não apenas a partir dos rótulos reais, mas também a partir das saídas produzidas pelo modelo professor. Com isso, o modelo menor pode preservar parte do desempenho do modelo maior, mesmo possuindo menor complexidade (HINTON; VINYALS; DEAN, 2015).

2.2.2 Treinamento ciente de quantização

A quantização pode ser aplicada de diferentes formas. Uma abordagem simples é a quantização pós-treinamento, ou *Post-Training Quantization* (PTQ), na qual um modelo previamente treinado em ponto flutuante é convertido para uma representação de precisão inferior

após o treinamento. Embora essa técnica seja prática, ela pode comprometer a acurácia, principalmente quando são utilizadas precisões reduzidas ou quando o modelo é sensível a pequenas variações numéricas (GHOLAMI et al., 2021). Uma alternativa mais robusta é o treinamento ciente de quantização, conhecido como *Quantization-Aware Training* (QAT). Nessa abordagem, os efeitos da quantização são simulados durante o próprio treinamento da rede. Assim, o modelo aprende a lidar com as limitações numéricas que estarão presentes na execução embarcada. Em vez de treinar todo o modelo em ponto flutuante e apenas depois convertê-lo, o QAT insere simulações de quantização nos pesos e ativações, aproximando o comportamento do treinamento ao comportamento esperado no hardware final (JACOB et al., 2018).

Essa técnica é especialmente importante quando o modelo será executado em dispositivos com suporte limitado a operações em ponto flutuante ou quando o hardware foi projetado para operar com dispersão de inteiros de precisão. Ao considerar a quantização durante o treinamento, o modelo tende a se tornar mais robusto a arredondamentos, saturações e perdas de precisão. Com isso, reduz-se a diferença entre a acurácia obtida no ambiente de treinamento e a acurácia observada na execução real do dispositivo (JACOB et al., 2018; GHOLAMI et al., 2021). No presente trabalho, o QAT foi relevante. Na MAX78000, a rede precisou ser treinada e convertida para uma forma compatível com o acelerador CNN interno, que executa inferência com representação numérica reduzida. Esse processo foi essencial para permitir que o modelo fosse sintetizado em firmware e executado diretamente na placa. A validação real em hardware demonstrou a importância dessa etapa, uma vez que diferentes checkpoints apresentaram comportamentos distintos no computador e na placa, evidenciando que maior acurácia em ambiente de treinamento nem sempre garante melhor desempenho no hardware embarcado.

No fluxo de desenvolvimento para a PYNQ-Z2, a técnica de QAT também se mostrou fundamental. A rede convolucional processada pelo FINN foi treinada previamente com o Brevitas, adotando pesos e ativações de baixa precisão. Após o treinamento, o modelo foi exportado para o formato QONNX e utilizado na geração do acelerador. Neste contexto, a quantização transcende a simples compressão do modelo, tornando-se uma premissa obrigatória para a síntese de uma arquitetura de hardware viável. A redução da precisão dos dados diminui drasticamente o consumo de memória, simplifica a lógica aritmética das operações e viabiliza um alto grau de paralelismo espacial na FPGA. (AMD Research, 2024b; AMD Research, 2024a). Portanto, o QAT representa uma etapa essencial para aproximar o modelo treinado do comportamento real do hardware. Em aplicações de *Edge AI* e *TinyML*, essa técnica contribui para compactar o tamanho dos modelos, otimizando o consumo computacional e preservar a acurácia em dispositivos com recursos limitados. No caso deste trabalho, sua utilização foi diretamente relacionada à viabilização da inferência embarcada na MAX78000 e da geração do acelerador quantizado na PYNQ-Z2 com FINN.

2.2.3 Sistemas Embarcados e Computação de Borda

Sistemas embarcados são sistemas computacionais projetados para executar funções dedicadas e específicas dentro de um dispositivo ou sistema maior (BERTOLETI, 2025; CUGNASCA, 2021). Diferentemente de computadores de propósito geral, eles são otimizados para atender a requisitos rigorosos de custo, tamanho, consumo de energia e, frequentemente, operação em tempo real. A arquitetura de um sistema embarcado é uma integração coesa de hardware e software. O núcleo do hardware é tipicamente um microcontrolador (MCU) ou um microprocessador (MPU), que combina uma unidade central de processamento (CPU) com memória (RAM para dados voláteis e Flash para armazenamento do programa) e periféricos de entrada e saída (I/O) em um único chip (CUGNASCA, 2021). O software, conhecido como firmware, é o programa que controla o hardware e implementa a funcionalidade dedicada do dispositivo.

2.2.4 Tecnologias de Processadores e Circuitos

A escolha da tecnologia de processamento é um fator fundamental no projeto de sistemas embarcados, pois influencia diretamente o desempenho, o consumo energético, o custo, a flexibilidade e a complexidade de desenvolvimento. Em aplicações de inteligência artificial embarcada, essa escolha torna-se ainda mais relevante, uma vez que a execução de redes neurais pode exigir grande volume de operações matemáticas, acesso frequente à memória e suporte a formatos numéricos reduzidos.

Microprocessadores, ou MPUs, como os baseados na família ARM Cortex-A, são projetados para alto desempenho computacional e normalmente são utilizados em sistemas de elevada complexibilidade, como smartphones, gateways, computadores embarcados e placas com sistema operacional Linux. Esses processadores geralmente dependem de componentes externos, como memória RAM e armazenamento, mas entregam ampla capacidade de processamento e flexibilidade de software. Plataformas como a NVIDIA Jetson seguem essa abordagem, combinando CPU, GPU e outros aceleradores em um único sistema em chip, ou *System-on-Chip* (SoC), permitindo a execução de modelos de inteligência artificial com desempenho otimizado (NVIDIA, 2024).

Microcontroladores, ou MCUs, como os baseados na família ARM Cortex-M, são projetados para baixo custo, baixo consumo energético e integração de periféricos em um único chip. Em geral, esses dispositivos possuem memória, interfaces de comunicação, temporizadores, conversores analógico-digitais e outros recursos integrados. Por essa razão, são amplamente utilizados em aplicações de *TinyML*, nas quais o modelo de aprendizado de máquina precisa operar em um ambiente com recursos computacionais e energéticos extremamente limitados (WARDEN; SITUNAYAKE, 2019). A MAX78000 representa esse tipo de abordagem, pois combina um microcontrolador com um acelerador CNN dedicado, permitindo a execução

de inferência neural com consumo reduzido (Analog Devices, 2024).

Além das CPUs e MCUs, outro tipo importante de tecnologia utilizada em inteligência artificial embarcada são as GPUs. As unidades de processamento gráfico são capazes de executar inúmeras operações em paralelo, o que as torna adequadas para cargas de trabalho envolvendo matrizes, convoluções e redes neurais profundas. Em plataformas embarcadas, GPUs integradas permitem acelerar a inferência de modelos de IA sem depender exclusivamente da CPU. Essa característica é explorada em plataformas como a Jetson, que utiliza GPU embarcada para executar redes neurais com baixa latência. Outra alternativa é o uso de FPGAs, ou *Field-Programmable Gate Arrays*. Diferentemente de CPUs e GPUs, que executam instruções sobre uma arquitetura previamente definida, os FPGAs permitem configurar circuitos digitais personalizados na lógica programável. Isso possibilita criar aceleradores específicos para determinada aplicação, explorando paralelismo em nível de hardware e obtendo maior previsibilidade de latência e vazão. Em SoCs FPGA, como os dispositivos da família Zynq, há a combinação entre processadores ARM e lógica programável no mesmo chip. Nesse modelo, o processador pode ser usado para controle da aplicação, enquanto a lógica programável executa o acelerador de inferência (AMD/Xilinx, 2024).

Em termos de tecnologia de circuito, os processadores e aceleradores são fabricados por meio de processos de semicondutores, como CMOS, em diferentes nós tecnológicos, por exemplo 28 nm, 16 nm ou 7 nm. Em geral, nós menores permitem elevada densidade de transistores, consumo atenuado por operação e ganhos expressivos de desempenho. Entretanto, também inflacionam o custo de fabricação e a complexidade do projeto. Dessa forma, a escolha entre CPU, GPU, MCU, acelerador dedicado ou FPGA depende do equilíbrio desejado entre desempenho, consumo, custo, flexibilidade e facilidade de desenvolvimento. No contexto deste trabalho, essa diversidade tecnológica é representada pelas três plataformas avaliadas. A Jetson representa uma solução baseada em SoC com GPU embarcada, voltada a alta performance. A MAX78000 representa uma solução baseada em microcontrolador com acelerador CNN dedicado, voltada a eficiência energética. A PYNQ-Z2 representa uma solução baseada em SoC FPGA, na qual a lógica programável pode ser configurada para implementar aceleradores de redes neurais quantizadas.

2.2.5 Framework FINN para aceleração de redes quantizadas em FPGA

O FINN é um framework desenvolvido originalmente pela Xilinx, atualmente AMD/Xilinx, com o objetivo de automatizar a geração de aceleradores de redes neurais quantizadas em FPGA. O trabalho original de Umuroglu et al. apresenta o FINN como uma abordagem para construção de aceleradores rápidos e escaláveis para redes neurais binarizadas, explorando arquiteturas em fluxo de dados e recursos customizáveis por camada (UMUROGLU et al., 2017). Diferentemente de plataformas baseadas em GPU, nas quais a inferência é executada sobre uma

arquitetura de processamento mais genérica, o FINN busca gerar uma arquitetura de hardware especializada para a rede neural utilizada. No fluxo atual, o FINN recebe como entrada uma rede neural quantizada em formato ONNX/QONNX, geralmente obtida a partir de treinamento eficiente de quantização com a biblioteca Brevitas. A partir desse modelo, o framework aplica transformações sucessivas na representação da rede, preparando as camadas para implementação em hardware. O processo inclui etapas de otimização, geração de componentes em HLS e RTL, integração no Vivado e criação dos arquivos necessários para execução em placas baseadas em FPGA (AMD Research, 2024b; AMD Research, 2024a).

Uma característica importante do FINN é a adoção de uma arquitetura *dataflow*, na qual as camadas da rede são implementadas como blocos de hardware conectados em fluxo. Nessa abordagem, os dados percorrem o acelerador camada por camada, reduzindo a necessidade de controle por software durante a inferência. Essa organização permite explorar paralelismo espacial e temporal, tornando possível obter latência e vazão mais previsíveis quando comparadas a execuções puramente sequenciais em processadores de propósito geral. Em dispositivos baseados em SoC FPGA, como a PYNQ-Z2, o acelerador gerado pelo FINN é implementado na lógica programável do dispositivo, enquanto o processador ARM é utilizado para controle, envio de dados e leitura dos resultados. Essa integração normalmente ocorre por meio de interfaces AXI e de arquivos de projeto gerados pelo fluxo de síntese, como o *bitstream*, o arquivo de descrição de hardware e o driver Python utilizado no ambiente PYNQ. Dessa forma, o FINN permite que uma rede neural quantizada seja transformada em um acelerador dedicado, executando a inferência diretamente no FPGA.

No contexto deste trabalho, o FINN foi utilizado na implementação da plataforma PYNQ-Z2. O modelo treinado com Brevitas foi exportado para QONNX e processado pelo FINN até a geração do acelerador, incluindo síntese, simulação e geração dos arquivos de implantação, baseada em hardware reconfigurável, complementando a comparação com a Jetson, que utiliza GPU embarcada, e com a MAX78000, que utiliza um acelerador CNN dedicado com foco em consumo.

2.2.6 Aceleração em Hardware para IA Embarcada

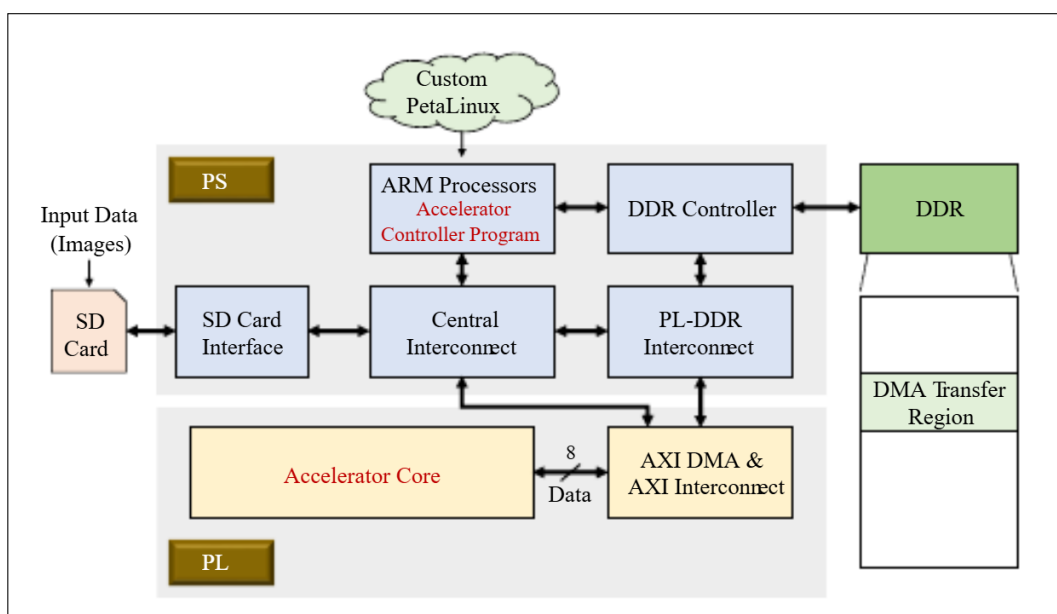
A execução de modelos de *deep learning*, especialmente redes neurais convolucionais e redes profundas, envolve grande quantidade de operações matemáticas, como multiplicações de matrizes, convoluções, somas acumuladas e funções de ativação. Embora processadores de propósito geral sejam flexíveis e adequados para uma ampla variedade de tarefas, sua arquitetura nem sempre é a mais eficiente para cargas de trabalho altamente paralelizáveis, como as encontradas em redes neurais. Por esse motivo, arquiteturas especializadas de hardware passaram a ser amplamente utilizadas para acelerar a inferência de IA, reduzindo latência, consumo energético e tempo de processamento (SZE et al., 2017). Em sistemas embarcados, a escolha da

arquitetura de aceleração depende do equilíbrio entre desempenho, consumo, custo, flexibilidade e facilidade de desenvolvimento. Entre as principais alternativas utilizadas em aplicações de IA embarcada estão os FPGAs, as GPUs embarcadas e os aceleradores neurais dedicados integrados a SoCs ou microcontroladores.

FPGA

Os FPGAs, ou *Field-Programmable Gate Arrays*, são circuitos integrados cuja lógica interna pode ser reconfigurada após a fabricação. Diferentemente de CPUs e GPUs, que executam instruções sobre uma arquitetura previamente definida, os FPGAs permitem implementar circuitos digitais específicos para determinada aplicação. Essa característica possibilita a criação de aceleradores customizados para redes neurais, explorando paralelismo em nível de hardware e mitigando a sobrecarga de execução por software. Em plataformas do tipo SoC-FPGA, como os dispositivos da família Zynq, há a integração entre processadores ARM e lógica programável no mesmo chip. Nessa arquitetura, o processador pode ser utilizado para controle da aplicação, gerenciamento de memória e comunicação com o sistema operacional, enquanto a lógica programável implementa o acelerador de inferência. A Figura 7 apresenta um diagrama de sistema baseado em SoC-FPGA, ilustrando a comunicação entre processador, memória, barramento AXI e núcleo acelerador.

Figura 7 – Diagrama de um sistema SoC-FPGA baseado na plataforma Zynq, ilustrando a integração entre CPU e lógica reconfigurável para aceleração de redes neurais.



Fonte: Hamanaka et al. (2023).

A principal vantagem dos FPGAs está na possibilidade de configurar uma arquitetura sob medida para o modelo de IA utilizado. Essa flexibilidade permite ajustar paralelismo,

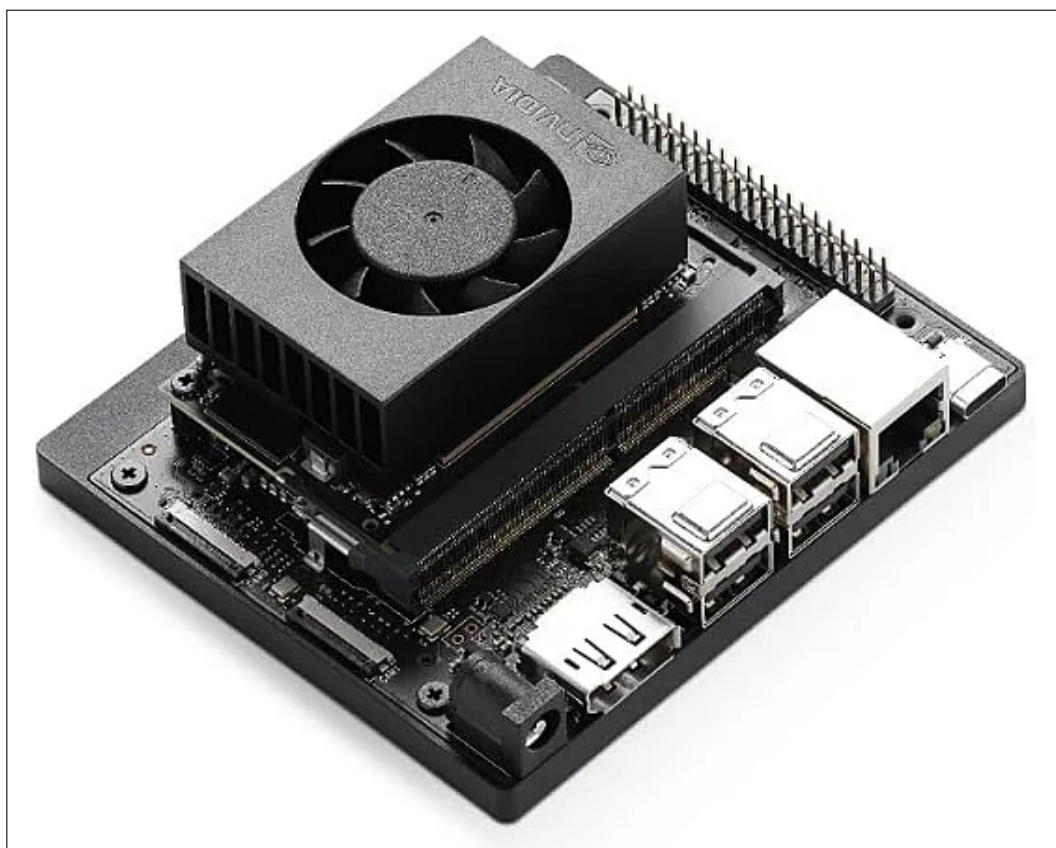
largura de dados, buffers e organização do fluxo de execução. Em redes quantizadas, essa característica é especialmente vantajosa, pois o hardware pode ser adaptado para operar com precisões reduzidas, como 8 bits, 4 bits, 2 bits ou até representações binárias. Frameworks como o FINN exploram essa propriedade para transformar redes neurais quantizadas em aceleradores especializados para FPGA (UMUROGLU et al., 2017; AMD Research, 2024b).

2.2.6.1 GPU embarcada

As GPUs, ou *Graphics Processing Units*, são arquiteturas projetadas para executar grande quantidade de operações em paralelo. Inicialmente desenvolvidas para processamento gráfico, passaram a ser amplamente utilizadas em aplicações de aprendizado profundo devido à semelhança entre as operações de redes neurais e operações matriciais altamente paralelizáveis. Em comparação com CPUs, caracterizada por uma quantidade reduzida de núcleos altamente complexos e otimizados para controle e tarefas sequenciais, as GPUs integram uma vasta matriz de processadores simplificados, adequados para executar a mesma operação sobre grandes volumes de dados.

Em aplicações de IA, as GPUs são especialmente eficientes em operações como convoluções e multiplicações de matrizes, que aparecem com frequência em redes neurais. GPUs modernas também incluem unidades especializadas, como os *Tensor Cores*, projetadas para acelerar operações matriciais em precisões reduzidas, como FP16, INT8 e outros formatos utilizados em inferência e treinamento de redes neurais. A Figura 8 apresenta o kit de desenvolvimento Jetson Orin Nano, utilizado neste trabalho como representante de plataforma embarcada baseada em GPU. Essa plataforma combina CPU ARM e GPU NVIDIA em um sistema embarcado, permitindo a execução de modelos de IA com bibliotecas de alto nível, como TensorFlow, Keras e ferramentas do ecossistema NVIDIA.

Figura 8 – Kit de desenvolvimento Jetson Orin Nano.



Fonte: NVIDIA (2024).

No contexto deste trabalho, a Jetson foi utilizada para executar uma rede neural convolucional em ambiente embarcado com maior capacidade computacional. Essa plataforma serviu como referência de alto desempenho, permitindo comparar a execução baseada em GPU com as demais abordagens avaliadas, como o microcontrolador com acelerador CNN dedicado e o SoC-FPGA.

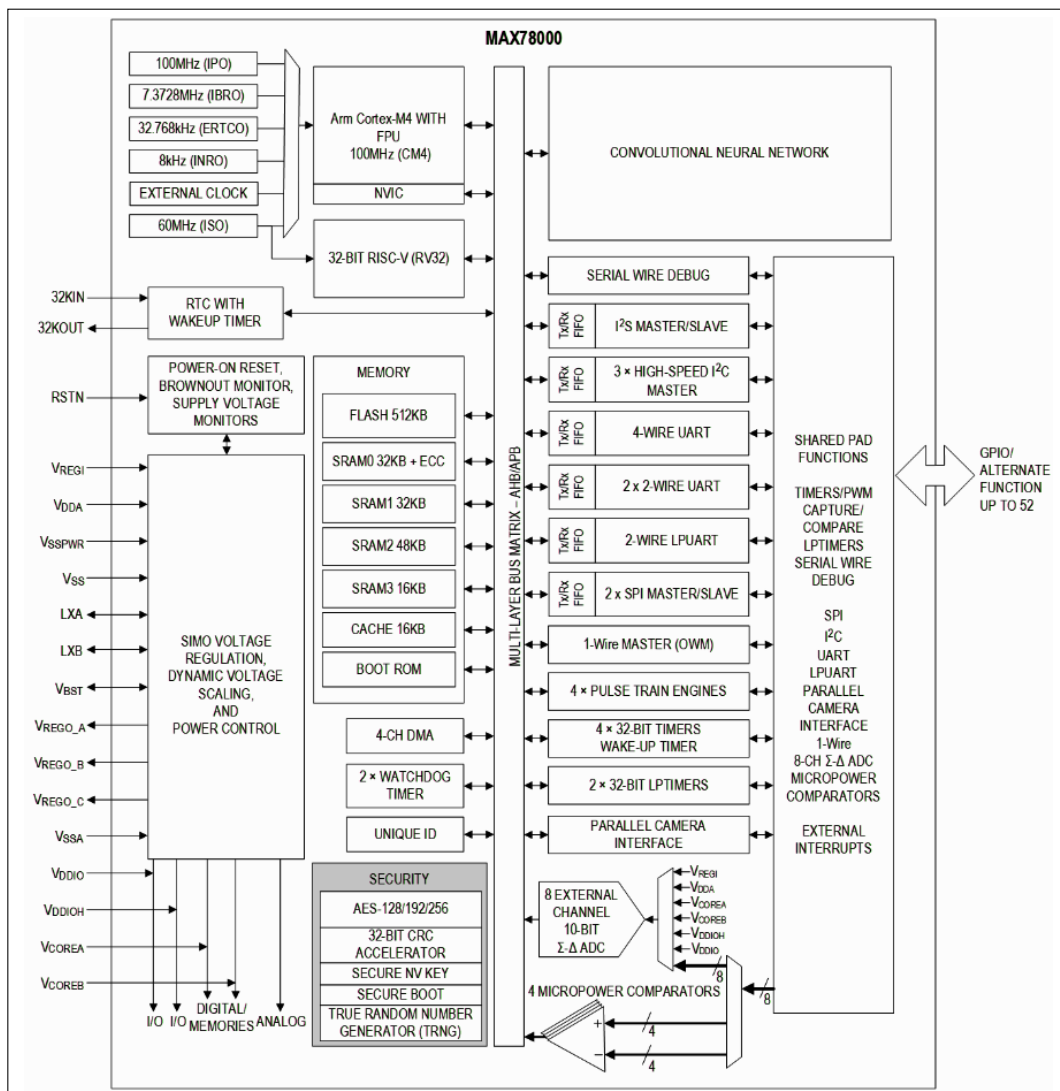
2.2.6.2 Aceleradores neurais dedicados em SoCs e microcontroladores

Além de GPUs e FPGAs, outra abordagem importante para IA embarcada é o uso de aceleradores neurais dedicados, frequentemente integrados a SoCs, microcontroladores ou circuitos especializados. Esses aceleradores são projetados para executar operações típicas de redes neurais com máxima eficiência quando comparados a uma CPU de propósito geral, atenuando o consumo energético e tempo de inferência. De modo geral, aceleradores neurais dedicados exploram dois princípios: aritmética de baixa precisão e paralelismo em hardware. A aritmética de baixa precisão utiliza representações numéricas compactas, como INT8 ou precisões menores, abreviando o uso de memória e o custo das operações. Já o paralelismo em hardware permite executar múltiplas operações simultaneamente, acelerando etapas como convoluções, multiplicações acumuladas e processamento de ativações (SZE et al., 2017; GHOLAMI et al.,

2021). Em algumas arquiteturas de aceleradores, o processamento é organizado em estruturas como arrays sistólicos, nos quais dados e pesos percorrem uma matriz de elementos de processamento de forma coordenada. Esse tipo de arquitetura permite expressiva reutilização de dados e redução de acessos à memória externa, fator importante porque a movimentação de dados pode consumir mais energia do que as próprias operações aritméticas (KUNG, 1982; SZE et al., 2017). Entretanto, nem todo acelerador neural utiliza necessariamente array sistólico; a organização interna depende do projeto do fabricante e do tipo de aplicação alvo.

A Figura 9 apresenta o diagrama de blocos do microcontrolador MAX78000, utilizado neste trabalho como representante de uma plataforma de ultra baixo consumo com acelerador neural integrado. O dispositivo combina núcleos de processamento de propósito geral, memória, periféricos e um acelerador CNN dedicado, permitindo executar inferência de redes neurais diretamente no microcontrolador.

Figura 9 – Microcontrolador MAX78000 com acelerador neural integrado



Fonte: Maxim Integrated / Analog Devices (2021).

No contexto deste trabalho, a MAX78000 foi utilizada para avaliar uma abordagem de *TinyML*, na qual a inferência é realizada em um dispositivo de recursos limitados e consumo reduzido. Diferentemente da Jetson, que utiliza GPU embarcada, e da PYNQ-Z2, que utiliza lógica reconfigurável em FPGA, a MAX78000 emprega um acelerador CNN dedicado integrado ao microcontrolador. Essa característica torna a plataforma adequada para aplicações em que consumo energético, tamanho físico e autonomia são fatores críticos.

2.3 TRABALHOS RELACIONADOS

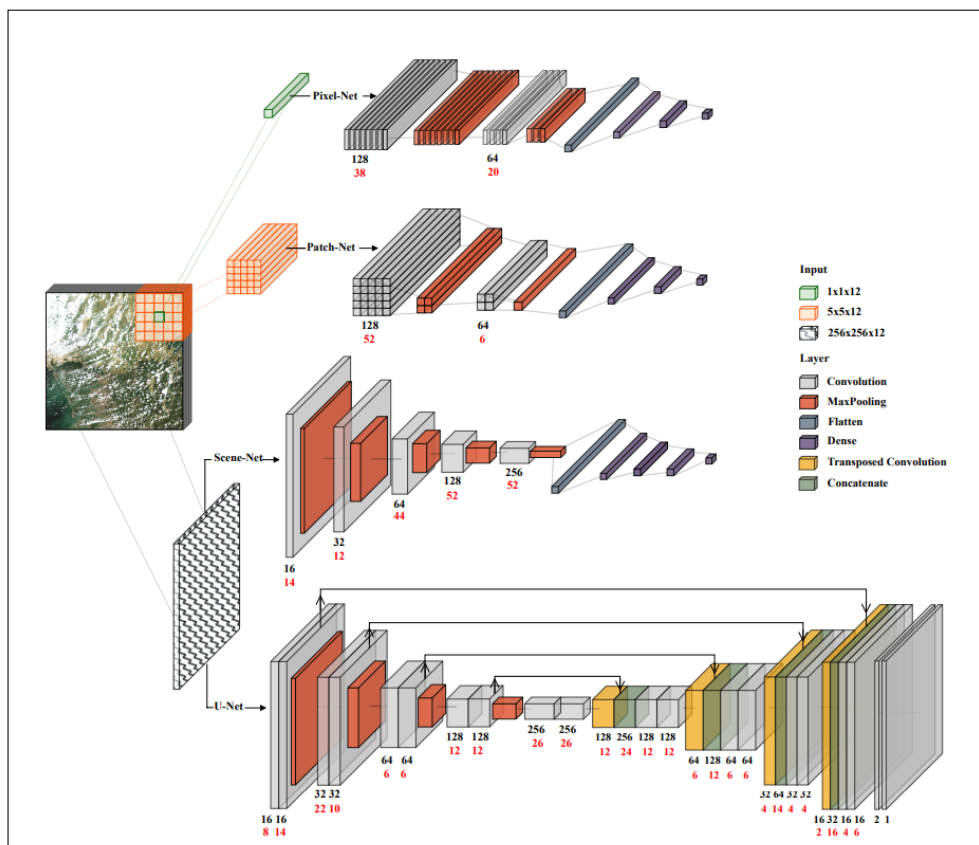
Esta seção apresenta trabalhos relacionados à implementação de modelos de inteligência artificial em plataformas embarcadas e aceleradores de hardware. A seleção dos estudos considerou aplicações que envolvem restrições de desempenho, consumo energético, memória e implantação em dispositivos de borda. Foram priorizados trabalhos que utilizam FPGAs, microcontroladores com aceleradores neurais dedicados e fluxos de otimização de modelos, como quantização, poda, co-design hardware/software e treinamento ciente de quantização. Os trabalhos analisados abrangem aplicações em sensoriamento remoto, visão computacional e processamento de áudio. Essa diversidade é relevante porque o presente trabalho compara três abordagens distintas de inferência embarcada: uma plataforma com GPU embarcada, representada pela Jetson Orin Nano; uma plataforma TinyML com acelerador CNN dedicado, representada pela MAX78000; e uma plataforma SoC-FPGA, representada pela PYNQ-Z2 com o fluxo FINN.

2.3.1 Aceleração de CNNs em FPGA para detecção de nuvens em CubeSats

O trabalho de Cratere et al. (2025) investiga a aceleração de redes neurais convolucionais em FPGA para detecção de nuvens a bordo de CubeSats. A motivação central consiste em otimizar o volume de dados transmitidos para a estação terrestre, uma vez que imagens com alta cobertura de nuvens apresentam utilidade limitada para determinadas aplicações de observação da Terra. Ao realizar a detecção a bordo, o sistema pode selecionar previamente as imagens mais relevantes, reduzindo a demanda de comunicação e ampliando para a autonomia da missão.

Para atender às restrições de tamanho, peso e potência de nanossatélites, os autores implementaram quatro arquiteturas de CNN, denominadas Pixel-Net, Patch-Net, Scene-Net e U-Net, utilizando o fluxo Vitis AI em uma plataforma com DPU da Xilinx. A estratégia de otimização combinou poda estruturada de canais e quantização INT8, reduzindo significativamente o número de parâmetros e operações dos modelos, com baixa perda de acurácia. Os resultados reportados indicam inferência em tempo real, com taxas de até 57,14 FPS para o modelo Scene-Net e consumo aproximado de 2,5 W.

Figura 10 – Arquiteturas CNN para detecção de nuvens em CubeSats.



Fonte: Cratere et al. (2025).

Esse trabalho é relevante para a presente pesquisa por demonstrar o uso de FPGA em aplicações de sensoriamento remoto com restrições embarcadas. Entretanto, ele se diferencia do presente trabalho por utilizar uma aplicação específica de detecção de nuvens, baseada em CNNs otimizadas para DPU, enquanto este trabalho realiza classificação de imagens do dataset SAT-6 e compara diferentes plataformas de execução embarcada.

2.3.2 BED: sistema de detecção de objetos para dispositivos de borda

O projeto BED, apresentado por Wang et al. (2022), explora o uso do microcontrolador MAX78000 para detecção de objetos em dispositivos de borda. O trabalho é relevante por demonstrar um fluxo completo de desenvolvimento, desde o treinamento do modelo até sua implantação em um dispositivo de ultra baixo consumo. O modelo utilizado é inspirado na arquitetura YOLOv1 e foi adaptado para as restrições do acelerador CNN integrado ao MAX78000. Para isso, os autores empregaram treinamento ciente de quantização, permitindo que o modelo fosse treinado considerando previamente as limitações numéricas do hardware. Após o treinamento, o modelo foi convertido para representação de 8 bits e sintetizado com as ferramentas do fabricante, gerando código executável para o microcontrolador.

Os resultados reportados indicam acurácia de 96,3% para classificação de 12 palavras-chave e consumo de 251 μ J por inferência. Esse estudo demonstra o potencial do MAX78000 para aplicações de ultra baixo consumo. No entanto, vale ressaltar que sua aplicação está relacionada ao processamento de áudio, enquanto o presente trabalho avalia imagens multiespectrais e compara a MAX78000 com outras arquiteturas embarcadas.

2.3.4 HAViTHI: acelerador baseado em Transformer para classificação de imagens hiperespectrais

O trabalho de Viel (VIEL, 2023), intitulado *HAViTHI: Acelerador em Hardware Baseado em Vision Transformer para Classificação de Imagens Hiperespectrais*, apresenta uma proposta de acelerador em hardware para classificação de imagens hiperespectrais. A tese parte do problema de que imagens hiperespectrais possuem um volume massivo de dados e exigem processamento eficiente, especialmente em aplicações de sensoriamento remoto e sistemas espaciais. A proposta utiliza uma arquitetura baseada em Transformer para classificação por pixel de imagens hiperespectrais. O trabalho considera que, embora arquiteturas Transformer já sejam amplamente utilizadas em classificação hiperespectral no nível espectral-espacial, sua exploração como acelerador em hardware para esse tipo de aplicação ainda é limitada. Além disso, a tese também destaca a necessidade de confiabilidade em aceleradores voltados a ambientes espaciais.

Um aspecto importante do HAViTHI é a busca por compromisso entre acurácia, tempo de computação, memória demandada, recursos de hardware e confiabilidade. Para isso, a tese realiza uma análise comparativa entre arquiteturas 1D-CNN, 2D-CNN, 3D-CNN, LSTM e Transformer, considerando classificação em nível espectral. Os resultados discutidos indicam que arquiteturas 1D e Transformer podem reduzir demanda de memória e tempo de processamento em comparação com abordagens 2D e 3D, embora arquiteturas espaciais e espectral-espaciais possam apresentar desempenho superior em cenários com forte desequilíbrio de classes.

Esse trabalho está diretamente relacionado à presente pesquisa por abordar aceleradores de hardware para imagens de sensoriamento remoto, uso de redes neurais em plataformas restritas e preocupação com desempenho, memória e confiabilidade. Entretanto, diferencia-se do presente trabalho em três pontos principais. Primeiro, o HAViTHI concentra-se em imagens hiperespectrais e classificação por pixel, enquanto este trabalho utiliza o dataset SAT-6, composto por imagens multiespectrais classificadas em seis classes. Segundo, o HAViTHI propõe uma arquitetura de acelerador baseada em Transformer, enquanto este trabalho avalia empiricamente três plataformas distintas de inferência embarcada. Terceiro, o presente trabalho inclui resultados reais de execução em hardware na Jetson, na MAX78000 e na PYNQ-Z2 com FINN, permitindo uma comparação prática entre GPU embarcada, acelerador CNN dedicado e FPGA.

2.3.5 Análise Comparativa

A análise conjunta dos trabalhos relacionados evidencia diferentes estratégias para viabilizar inteligência artificial em dispositivos embarcados. Em FPGAs, observa-se maior flexibilidade arquitetural e possibilidade de criação de aceleradores customizados, como no trabalho de Cratere et al. (2025) e no fluxo FINN utilizado neste trabalho. Em microcontroladores com aceleradores neurais dedicados, como o MAX78000, observa-se maior eficiência energética, como demonstrado por Wang et al. (2022) e Ulkar e Okman (2021). Já o trabalho de (VIEL, 2023) contribui com uma perspectiva voltada ao projeto de aceleradores para imagens hiperespectrais e aplicações espaciais, reforçando a importância de considerar simultaneamente acurácia, desempenho, memória, recursos de hardware e confiabilidade.

O presente trabalho diferencia-se dos estudos analisados por realizar uma comparação experimental entre três plataformas de natureza distinta, utilizando o mesmo problema de classificação de imagens. A Jetson Orin Nano representa uma abordagem baseada em GPU embarcada, com maior capacidade computacional e menor latência. A MAX78000 representa uma abordagem TinyML com acelerador CNN dedicado e foco em ultra baixo consumo. A PYNQ-Z2 representa uma abordagem baseada em FPGA, utilizando o framework FINN para geração de um acelerador a partir de um modelo quantizado.

Tabela 1 – Comparativo entre trabalhos relacionados e o presente trabalho.

Característica	Ulkar e Okman (2021)	Pathak et al. (2022)	Viel (2023)	Cratere et al. (2025)	Este trabalho
Aplicação principal	Reconhecimento de palavras-chave	Detecção de objetos em dispositivo de borda	Classificação de imagens hiperespectrais	Detecção de nuvens em CubeSats	Classificação de imagens multiespectrais do SAT-6
Tipo de dado	Áudio	Imagens de câmera	Imagens hiperespectrais	Imagens de sensoriamento remoto	Imagens multiespectrais $28 \times 28 \times 4$
Hardware alvo	Microcontrolador MAX78000	Microcontrolador MAX78000	Acelerador em hardware para FPGA	FPGA com DPU Xilinx	Jetson Orin Nano, MAX78000 e PYNQ-Z2
Modelo ou arquitetura	CNN para KWS	Modelo inspirado em YOLOv1	Arquitetura baseada em Transformer/ViT	Pixel-Net, Patch-Net, Scene-Net e U-Net	CNN na Jetson, MAX78000 e na PYNQ-Z2/FINN
Framework ou ferramentas	Ferramentas MAX78000 e NAS/OFA	PyTorch e ferramentas MAX78000	Modelagem de acelerador em hardware	Vitis AI	TensorFlow/Keras, ai8x/MaximSDK, Brevitas/QONNX/FINN
Técnicas de otimização	Co-design hardware/software e NAS	QAT e quantização	Redução de memória, análise arquitetural e confiabilidade	Poda estruturada de canais e quantização INT8	Quantização, QAT, validação HIL e geração de acelerador com FINN
Resultado principal	96,3% de acurácia em 12 classes e $251 \mu\text{J}$ por inferência	Sistema funcional de detecção de objetos em microcontrolador	Proposta de acelerador para HSI com foco em acurácia, tempo, recursos e confiabilidade	Até 57,14 FPS e consumo aproximado de 2,5 W	99,34% na Jetson; 94,70% na MAX78000; 93,50% na PYNQ-Z2/FINN
Consumo ou eficiência	Ultra baixo consumo por inferência	Foco em operação de baixo consumo	Foco em redução de recursos e eficiência em hardware	Aproximadamente 2,5 W	MAX78000 com aproximadamente 60 mW medidos; Jetson com 6,539 W médios
Foco diferencial	Eficiência energética extrema em áudio TinyML	Sistema de visão computacional ponta a ponta no MAX78000	Acelerador para HSI com preocupação em confiabilidade	Processamento a bordo para redução de transmissão em CubeSats	Comparação entre GPU embarcada, MCU TinyML e FPGA

Fonte: Autor (2026).

3 PROJETO

Este capítulo apresenta o projeto desenvolvido para avaliação comparativa de inferência embarcada aplicada à classificação de imagens multiespectrais do dataset SAT-6. O objetivo do projeto é analisar o comportamento de modelos de inteligência artificial executados em três plataformas de hardware distintas, representando diferentes paradigmas de aceleração na borda: uma plataforma com GPU embarcada, uma plataforma TinyML com acelerador CNN dedicado e uma plataforma SoC-FPGA com acelerador gerado por fluxo de síntese. A Seção 3.1 apresenta a visão geral do projeto e as premissas adotadas. Em seguida, são descritos os requisitos funcionais e não funcionais, a delimitação do escopo, a preparação dos modelos e os fluxos de implantação em cada plataforma. Por fim, são apresentados o plano de verificação, a metodologia experimental e os critérios utilizados para comparação dos resultados.

3.1 VISÃO GERAL E PREMISSAS

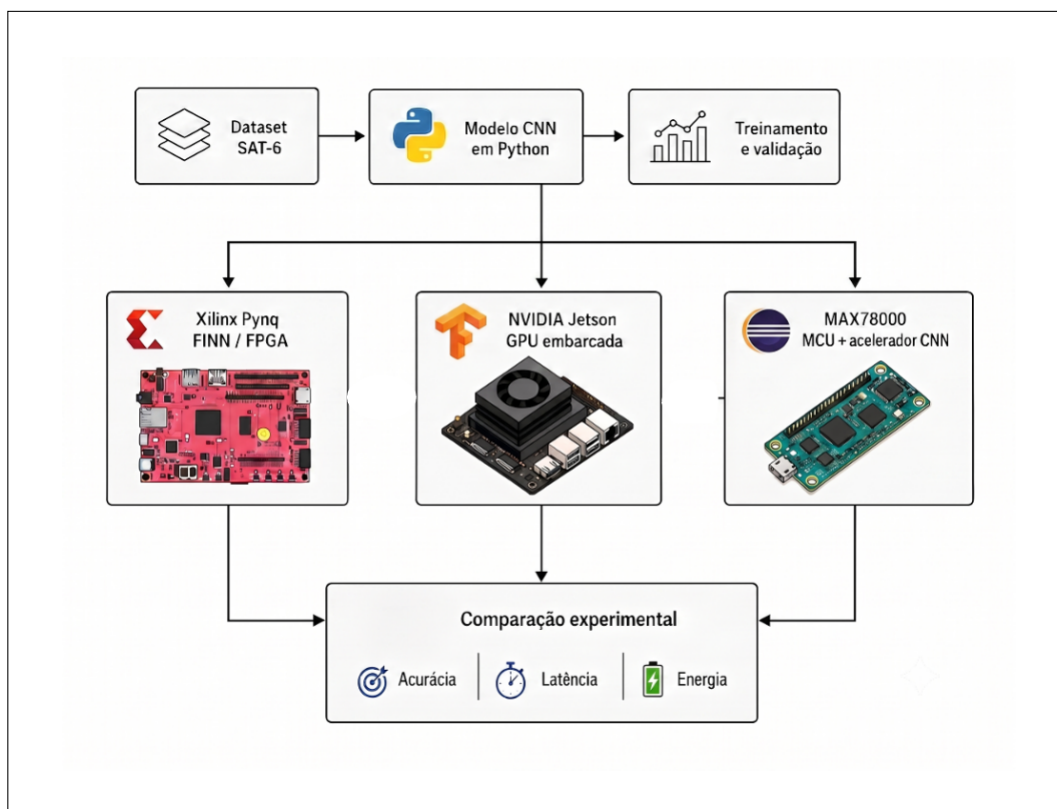
Este trabalho propõe o desenvolvimento e a avaliação de uma solução de inferência embarcada para classificação de imagens multiespectrais de sensoriamento remoto, utilizando o dataset SAT-6. A análise é realizada considerando três plataformas distintas: NVIDIA Jetson Orin Nano, MAX78000 e PYNQ-Z2. Cada uma dessas plataformas representa uma abordagem diferente para execução ou aceleração de modelos de inteligência artificial em dispositivos embarcados. A NVIDIA Jetson Orin Nano representa uma plataforma baseada em *System-on-Chip* com GPU embarcada, adequada para execução de modelos de aprendizado profundo com maior capacidade computacional. A MAX78000 representa uma plataforma TinyML baseada em microcontrolador com acelerador CNN dedicado, voltada à inferência de ultra baixo consumo. A PYNQ-Z2 representa uma plataforma SoC-FPGA, utilizada como alvo para o estudo de geração de aceleradores em FPGA por meio do fluxo FINN.

Embora as três abordagens estejam relacionadas ao mesmo problema de classificação, os fluxos de implementação foram adaptados às características de cada ecossistema de hardware e software. Na Jetson, foi utilizada uma rede neural convolucional implementada com TensorFlow/Keras. Na MAX78000, foi utilizada uma CNN quantizada e convertida para execução no acelerador CNN interno da placa. Para a PYNQ-Z2, foi adotado o fluxo baseado em PyTorch, Brevitas, QONNX e FINN, seguindo uma topologia CNN quantizada aplicada ao dataset SAT-6. Nesse fluxo, a rede é treinada com quantização em pesos e ativações, exportada para o formato QONNX e processada pelo FINN para geração e análise de um acelerador em FPGA.

O fluxo geral do projeto é apresentado na Figura 13. Inicialmente, o dataset SAT-6 é preparado para treinamento e validação dos modelos. Em seguida, são definidos modelos convolucionais compatíveis com cada fluxo de implantação ou geração de acelerador. Após essa

etapa, cada plataforma ou ferramenta recebe um processo específico de conversão, compilação, síntese ou simulação. Por fim, os resultados são avaliados por meio de métricas como acurácia, latência, consumo energético e, no caso do fluxo FINN, utilização de recursos e estimativas de desempenho obtidas no processo de geração do acelerador.

Figura 13 – Visão geral do projeto proposto.



Fonte: Autor (2026).

A comparação entre as abordagens permite observar os compromissos envolvidos entre desempenho, consumo, complexidade de implementação e adequação das ferramentas utilizadas. Dessa forma, o projeto não se limita à avaliação da acurácia dos modelos, mas também considera aspectos práticos da implantação embarcada e da geração de aceleradores, como adaptação do modelo, compatibilidade com ferramentas, conversão de formatos, geração de firmware, síntese em FPGA, simulação e análise de desempenho.

3.1.1 Premissas

As premissas que norteiam o desenvolvimento deste trabalho são apresentadas a seguir:

- O trabalho utiliza o dataset SAT-6 como base para classificação de imagens multiespectrais de sensoriamento remoto.

- As imagens utilizadas possuem dimensão reduzida e quatro canais espectrais, permitindo avaliar a execução e a aceleração de inferência em plataformas embarcadas com diferentes capacidades computacionais.
- O treinamento e a preparação dos modelos são realizados em ambiente computacional externo às placas, utilizando ferramentas em Python.
- A execução nas plataformas embarcadas é voltada à inferência, não ao treinamento dos modelos.
- A Jetson Orin Nano utiliza um fluxo baseado em TensorFlow/Keras para execução de uma CNN em uma plataforma com GPU embarcada.
- A MAX78000 utiliza um fluxo baseado nas ferramentas do fabricante para conversão, síntese e execução de uma CNN quantizada no acelerador CNN interno.
- No fluxo FINN, as etapas consideradas incluem preparação do dataset, treinamento da CNN quantizada, exportação para QONNX, aplicação das transformações do FINN, geração do acelerador, simulação e análise dos relatórios de desempenho e utilização de recursos.
- A abordagem com PYNQ-Z2 concentra-se na reprodução do fluxo FINN para CNN quantizada, sem considerar a execução física final na placa.
- As métricas utilizadas na avaliação incluem acurácia de classificação, latência de inferência e consumo energético, conforme a disponibilidade e as características de medição de cada plataforma.
- A comparação considera não apenas o desempenho final, mas também a complexidade de implantação, as limitações de cada ferramenta e a adequação de cada abordagem à inferência embarcada.

3.2 ANÁLISE DE REQUISITOS

Com base nas premissas descritas na seção anterior, os Quadro 1 e Quadro 2 descrevem os requisitos funcionais e não funcionais, respectivamente.

Quadro 1 – Requisitos funcionais

Requisito	Descrição
RF01	A solução deve realizar a classificação de imagens multiespectrais do dataset SAT-6, considerando suas seis classes de cobertura terrestre.
RF02	O sistema deve receber como entrada amostras de imagens multiespectrais com quatro canais, compatíveis com o formato utilizado no treinamento e na inferência dos modelos.
RF03	Devem ser desenvolvidos a partir de redes neurais convolucionais, compatíveis com as características das abordagens avaliadas, considerando CNN em TensorFlow/Keras para a Jetson Orin Nano, CNN quantizada para a MAX78000 e CNN quantizada com PyTorch/Brevitas, QONNX e FINN para o fluxo da PYNQ-Z2.
RF04	A solução deve permitir a execução da inferência nas plataformas Jetson Orin Nano e MAX78000, além da geração, transformação, simulação e análise do acelerador CNN quantizado no fluxo FINN voltado à PYNQ-Z2.
RF05	Devem ser coletadas métricas de avaliação, incluindo acurácia de classificação, latência de inferência e consumo energético, conforme a disponibilidade e as características de medição de cada abordagem. No caso do fluxo FINN, também devem ser considerados resultados de simulação, estimativas de desempenho e utilização de recursos de hardware.
RF06	Os arquivos, códigos e resultados utilizados no treinamento, implantação e avaliação devem ser organizados de forma a permitir a rastreabilidade e a reprodutibilidade dos experimentos.

Fonte: Autor (2026).

Quadro 2 – Requisitos não funcionais

Requisito	Descrição
RNF01	O treinamento, a preparação dos dados e a avaliação inicial dos modelos devem ser realizados em Python, utilizando bibliotecas de aprendizado de máquina e aprendizado profundo compatíveis com cada fluxo de desenvolvimento.
RNF02	Os modelos destinados à execução embarcada ou à geração de aceleradores devem empregar técnicas de otimização e quantização quando exigido ou favorecido pela plataforma ou ferramenta utilizada.
RNF03	A implementação na Jetson Orin Nano deve utilizar um ambiente compatível com execução embarcada de modelos de aprendizado profundo, explorando a GPU.
RNF04	A implementação na MAX78000 deve utilizar firmware em linguagem C e as ferramentas do ecossistema Analog Devices/Maxim, explorando o acelerador CNN integrado ao microcontrolador.
RNF05	O fluxo voltado à PYNQ-Z2 deve utilizar PyTorch, Brevitas, QONNX e FINN para treinamento, quantização, exportação, transformação, geração e simulação de um acelerador em FPGA a partir de uma CNN quantizada.
RNF06	A avaliação deve diferenciar claramente os resultados obtidos em hardware real, software, simulação, síntese e estimativas de ferramenta, evitando tratar resultados simulados ou estimados como execução física na placa.
RNF07	A comparação entre as abordagens deve considerar não apenas os resultados de acurácia, mas também latência, consumo energético, complexidade de implantação, utilização de recursos de hardware e limitações de cada fluxo.

Fonte: Autor (2026).

3.3 DELIMITAÇÃO DE ESCOPO

Este trabalho delimita-se à implementação e avaliação de modelos de inteligência artificial aplicados à classificação de imagens multiespectrais do dataset SAT-6, considerando três abordagens de computação embarcada: NVIDIA Jetson Orin Nano, MAX78000 e PYNQ-Z2 com FINN. O objetivo não é propor um novo dataset, nem desenvolver uma nova arquitetura de rede neural do estado da arte, mas avaliar a viabilidade prática de aplicação de redes neurais convolucionais em plataformas com características distintas de processamento. O escopo contempla a preparação dos dados, o treinamento dos modelos, a aplicação de técnicas de quantização ou otimização quando necessárias, a adaptação dos modelos aos respectivos fluxos de desenvolvimento e a coleta de resultados experimentais ou estimados conforme a natureza de cada plataforma.

Na Jetson Orin Nano, o foco está na execução de uma rede neural convolucional em

uma plataforma com GPU embarcada. Na MAX78000, o foco está na execução de uma CNN quantizada no acelerador neural integrado ao microcontrolador. Na abordagem com PYNQ-Z2, o foco está na reprodução do fluxo FINN para geração e análise de um acelerador em FPGA a partir de uma CNN quantizada, utilizando PyTorch, Brevitas, QONNX e as transformações do FINN.

A comparação entre as abordagens é realizada com base em métricas como acurácia, latência ou tempo de inferência e consumo energético, conforme a disponibilidade de medição em cada caso. No fluxo FINN, também são considerados os resultados de síntese, simulação e utilização de recursos de hardware, como LUTs, FFs, BRAMs, DSPs e estimativas de desempenho geradas pela ferramenta. Também são consideradas as dificuldades práticas de implantação e reprodução dos fluxos, como compatibilidade de ferramentas, adaptação do modelo, conversão de dados, geração de firmware, exportação para QONNX, transformação do modelo e geração do acelerador.

Não fazem parte do escopo deste trabalho a integração com satélites, veículos aéreos não tripulados ou sistemas reais de aquisição de imagens em campo. Também não são abordados novos métodos de pré-processamento de imagens, novas técnicas de compressão de dados, treinamento diretamente nas placas, desenvolvimento manual de hardware em RTL ou criação de uma nova arquitetura neural inédita. No caso da PYNQ-Z2, esta etapa do trabalho não contempla a execução física final na placa, concentrando-se na reprodução do fluxo de CNN quantizada com FINN e na análise dos resultados gerados pelo processo de síntese e simulação. O foco está na avaliação experimental e comparativa de abordagens para inferência embarcada aplicadas ao mesmo problema de classificação.

3.4 MODELOS E ARQUITETURAS ADOTADAS

Esta seção apresenta as arquiteturas e modelos considerados para a classificação de imagens multiespectrais do dataset SAT-6. Inicialmente, são discutidas arquiteturas CNN de referência utilizadas em trabalhos relacionados com o SAT-6 e com aceleração em FPGA. Em seguida, são descritas as escolhas adotadas para cada abordagem avaliada neste trabalho, mantendo o uso de redes neurais convolucionais como base para a tarefa de classificação.

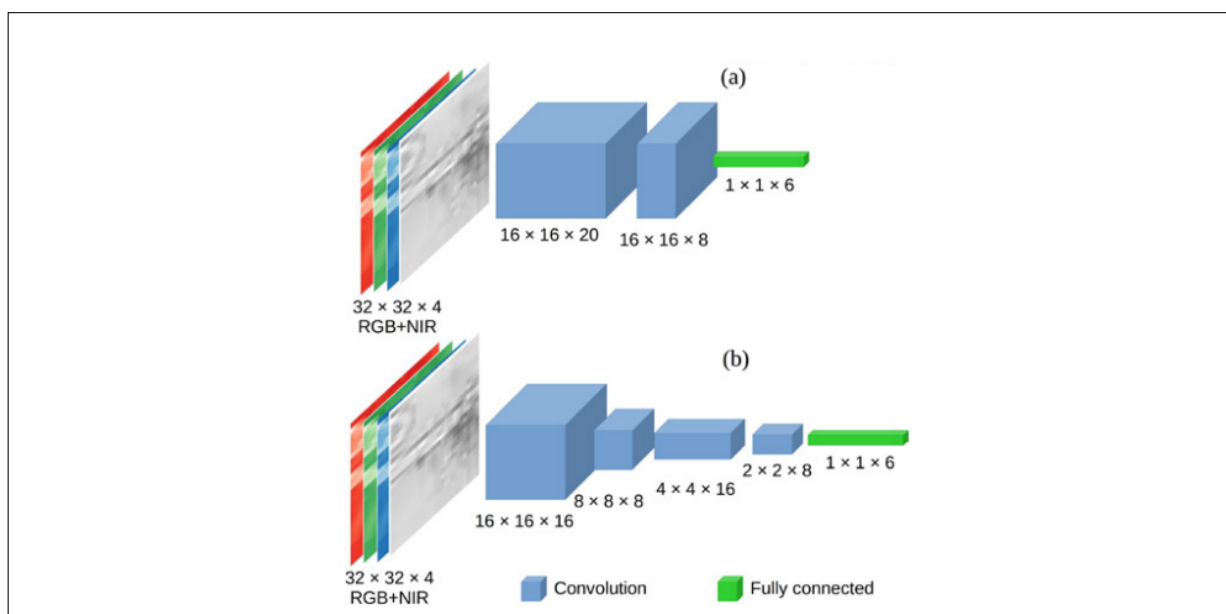
3.4.1 Arquiteturas CNN de referência para o SAT-6

O dataset SAT-6 é composto por imagens multiespectrais associadas a seis classes de cobertura terrestre. Por conter informação espacial e espectral em cada amostra, redes neurais convolucionais são adequadas para esse tipo de problema, pois permitem explorar padrões locais presentes nas imagens, como texturas, bordas, regiões homogêneas e variações entre bandas espectrais. Em trabalhos relacionados, arquiteturas CNN leves foram avaliadas para

classificação de imagens aéreas em plataformas embarcadas e aceleradores em FPGA. Ely et al. (2024) apresentam duas topologias CNN para classificação de imagens utilizando o framework FINN em APSoCs baseados em SRAM. Essas topologias foram avaliadas considerando diferentes níveis de quantização, paralelismo e uso de recursos, permitindo analisar o impacto da profundidade da rede e das configurações do acelerador sobre o desempenho do sistema.

A Figura 14 apresenta duas topologias utilizadas como referência. A primeira topologia é composta por duas camadas convolucionais, sendo uma convolução inicial 3×3 seguida por uma convolução 1×1 , além de uma camada final totalmente conectada. A segunda topologia é composta por quatro camadas convolucionais 3×3 com *stride* 2, seguidas por uma camada totalmente conectada responsável pela classificação final em seis classes.

Figura 14 – Topologias CNN de referência para classificação de imagens com o framework FINN.



Fonte: Adaptado de Ely et al. (2024).

O Quadro 3 resume a segunda topologia, utilizada como referência principal para o fluxo CNN com FINN. A entrada considerada nessa topologia é uma imagem multiespectral com quatro canais no formato $32 \times 32 \times 4$. Esse formato corresponde à adequação das amostras do SAT-6 ao fluxo de CNN utilizado no FINN, preservando as quatro bandas espectrais de entrada. A saída da rede corresponde às seis classes do dataset SAT-6.

Quadro 3 – Arquitetura CNN de referência para o dataset SAT-6

Camada	Descrição	Saída
Entrada	Imagem multiespectral com quatro canais	$32 \times 32 \times 4$
Layer 1	Convolução 3×3 com <i>stride</i> 2	$16 \times 16 \times 16$
Layer 2	Convolução 3×3 com <i>stride</i> 2	$8 \times 8 \times 8$
Layer 3	Convolução 3×3 com <i>stride</i> 2	$4 \times 4 \times 16$
Layer 4	Convolução 3×3 com <i>stride</i> 2	$2 \times 2 \times 8$
Saída	Camada totalmente conectada com seis saídas	6 classes

Fonte: Adaptado de Ely et al. (2024).

A arquitetura apresentada no Quadro 3 é especialmente relevante para este trabalho por representar uma CNN leve, compatível com classificação de imagens multiespectrais e adequada ao fluxo de geração de aceleradores com o FINN. No caso da abordagem voltada à PYNQ-Z2, essa topologia serve como base para o fluxo de treinamento com PyTorch e Brevitás, exportação para QONNX, transformações do FINN, geração do acelerador, simulação e análise dos recursos de hardware.

3.4.2 Modelos utilizados nas plataformas avaliadas

Embora o problema de classificação seja o mesmo para as três abordagens avaliadas, os modelos aplicados demandaram ajustes conforme as características de cada hardware e de cada fluxo de desenvolvimento. Essa decisão foi necessária considerando as plataformas avaliadas possuem diferentes restrições de memória, formatos numéricos, ferramentas de conversão, mecanismos de aceleração e níveis de suporte à quantização.

Para a NVIDIA Jetson Orin Nano, foi utilizada uma rede neural convolucional implementada em Python com TensorFlow/Keras. Essa plataforma possui elevada capacidade computacional e GPU embarcada, permitindo a execução de modelos de aprendizado profundo com ampla flexibilidade de software. Nesse caso, a CNN foi utilizada como modelo de classificação para avaliar o desempenho da inferência em uma plataforma embarcada de alto poder computacional.

Na MAX78000, foi utilizada uma CNN quantizada e convertida para execução no acelerador CNN interno da placa. Esse fluxo exigiu adaptação do modelo às restrições do microcontrolador, geração de firmware em linguagem C e validação em hardware real. A escolha por uma CNN nessa plataforma é coerente com o próprio acelerador dedicado do MAX78000, projetado para executar operações típicas de redes convolucionais com baixo consumo energético.

Na abordagem voltada à PYNQ-Z2, foi adotado um fluxo baseado em CNN quantizada, seguindo o processo de treinamento, exportação e geração de acelerador utilizado como referência no fluxo SAT-6 CNN com FINN. Nesse caso, a rede é definida em PyTorch com camadas quantizadas da biblioteca Brevitas, preservando a estrutura espacial e espectral das imagens. Após o treinamento, o modelo é exportado para o formato QONNX e processado pelo FINN, que aplica transformações sobre o grafo da rede e gera uma arquitetura de acelerador em FPGA para análise por meio de síntese e simulação.

Dessa forma, a comparação realizada neste trabalho considera três abordagens baseadas em redes neurais convolucionais aplicadas ao mesmo problema de classificação. A Jetson representa uma execução baseada em GPU embarcada, a MAX78000 representa uma execução em microcontrolador com acelerador CNN dedicado, e a PYNQ-Z2 representa o estudo de geração de acelerador em FPGA a partir de uma CNN quantizada com Brevitas, QONNX e FINN.

3.4.3 Treinamento e preparação dos modelos

O processo de treinamento e preparação dos modelos foi definido conforme as características de cada plataforma alvo. Embora o problema de classificação seja o mesmo, cada abordagem apresenta restrições e ferramentas próprias, exigindo adaptações no modelo, no formato dos dados, na quantização e no fluxo de implantação ou geração do acelerador. A estratégia adotada neste trabalho considera três abordagens distintas. Na Jetson Orin Nano, foi utilizado um fluxo baseado em Python e TensorFlow/Keras, adequado à execução de uma CNN em uma plataforma embarcada com GPU. Na MAX78000, foi utilizado um fluxo voltado à execução de uma CNN quantizada no acelerador CNN interno do microcontrolador. Na abordagem com PYNQ-Z2, foi utilizado um fluxo baseado em PyTorch, Brevitas, QONNX e FINN para treinar, quantizar, exportar, transformar e analisar uma CNN quantizada voltada à geração de acelerador em FPGA. Essa separação fez-se necessária diante da impossibilidade de um fluxo unificado de treinamento e implantação atender as particularidades, às três plataformas. Dessa forma, o trabalho avalia como diferentes paradigmas de hardware e ferramentas de desenvolvimento permitem aplicar redes neurais convolucionais à classificação de imagens multiespectrais, considerando suas próprias restrições de desenvolvimento, formatos numéricos e estratégias de otimização.

3.4.3.1 Preparação para NVIDIA Jetson Orin Nano

A plataforma NVIDIA Jetson Orin Nano foi utilizada como ambiente embarcado com aceleração por GPU. Diferentemente da MAX78000, cujo fluxo exige maior adequação ao acelerador CNN dedicado, a Jetson oferece maior flexibilidade de software, permitindo a execução de modelos de aprendizado profundo com bibliotecas de alto nível. Neste trabalho, o modelo utilizado na Jetson foi preparado em Python com TensorFlow/Keras. O conjunto de dados SAT-6 foi carregado a partir dos arquivos disponíveis, reorganizado para o formato de imagem

multiespectral e normalizado conforme o padrão de entrada exigido pelo modelo. As amostras possuem originalmente dimensão $28 \times 28 \times 4$, correspondendo a 28 pixels de altura, 28 pixels de largura e quatro bandas espectrais. A execução na Jetson teve como objetivo estabelecer uma referência de desempenho em uma plataforma de borda com maior robustez de hardware. Essa avaliação permitiu comparar a acurácia, a latência e o consumo energético da abordagem baseada em GPU com as demais abordagens avaliadas no trabalho.

3.4.3.2 Preparação para PYNQ-Z2 com FINN/Brevitas

A preparação para a abordagem com PYNQ-Z2 utilizou o fluxo PyTorch/Brevitas, voltado à geração de aceleradores de redes neurais quantizadas em FPGA. Diferentemente do fluxo anteriormente considerado, esta etapa foi baseada em uma rede neural convolucional quantizada, mantendo a estrutura espacial das imagens multiespectrais do SAT-6. O fluxo adotado segue a organização do estudo de referência para SAT-6 CNN com FINN. Inicialmente, o dataset é preparado para uso em uma arquitetura convolucional. Embora as amostras originais do SAT-6 possuam dimensão $28 \times 28 \times 4$, o fluxo de referência utiliza entrada $32 \times 32 \times 4$, adequando as imagens ao formato esperado pelas topologias CNN analisadas. Essa adaptação preserva os quatro canais espectrais e permite aplicar as camadas convolucionais definidas para o processo de geração de hardware.

A rede convolucional é definida em PyTorch com apoio da biblioteca Brevitas, que permite utilizar camadas quantizadas durante o treinamento. Essa característica é importante para o FINN, pois o framework é voltado à geração de aceleradores para redes neurais quantizadas. Após o treinamento, o modelo é exportado para o formato QONNX, que representa a rede quantizada em uma forma compatível com as transformações realizadas pelo FINN. Em seguida, o FINN processa o modelo QONNX por meio de etapas de transformação, otimização, especialização de operadores, geração de componentes de hardware, síntese e simulação. O objetivo dessa etapa é avaliar a viabilidade de geração de um acelerador em FPGA para uma CNN quantizada aplicada ao SAT-6, considerando resultados como acurácia em software, latência estimada, uso de recursos de hardware e comportamento do fluxo de síntese e simulação. Nesta fase do trabalho, não é considerada a execução física final na PYNQ-Z2.

3.4.3.3 Preparação para MAX78000

A MAX78000 possui um acelerador CNN integrado ao microcontrolador, voltado à execução eficiente de redes neurais em aplicações de baixo consumo. Por esse motivo, o fluxo de preparação do modelo para essa plataforma exige adaptação às restrições do hardware e às ferramentas do ecossistema Analog Devices/Maxim. Neste trabalho, foi utilizada uma CNN quantizada e convertida para execução no acelerador CNN interno da placa. O processo envolveu treinamento e preparação do modelo em ambiente externo, conversão para representação compatível com a plataforma, geração de código em linguagem C e compilação do firmware

para execução na placa. A validação foi realizada em hardware real, permitindo coletar resultados de inferência diretamente da MAX78000. O objetivo dessa etapa foi avaliar uma abordagem TinyML de ultra baixo consumo, verificando a viabilidade de executar classificação de imagens multiespectrais em um microcontrolador com acelerador neural dedicado. Essa plataforma foi especialmente relevante para analisar o compromisso entre acurácia, consumo energético e restrições de implantação.

3.4.4 Conjunto de dados

O conjunto de dados selecionado para este trabalho é o SAT-6, originalmente proposto no contexto do DeepSat. Esse dataset é composto por imagens de sensoriamento remoto extraídas do programa NAIP (*National Agriculture Imagery Program*) e organizadas em patches de 28×28 pixels. Cada amostra possui quatro bandas espectrais: vermelho, verde, azul e infravermelho próximo, representadas respectivamente por R, G, B e NIR. O SAT-6 possui 405.000 imagens rotuladas, divididas em 324.000 imagens para treinamento e 81.000 imagens para teste. As amostras são classificadas em seis classes de cobertura terrestre: *barren land*, *trees*, *grassland*, *roads*, *buildings* e *water bodies*. Essas características tornam o dataset adequado para avaliar modelos de classificação de imagens multiespectrais em plataformas embarcadas (BASU et al., 2015). A descrição deste conjunto pode ser observada no Quadro 4, a seguir:

Quadro 4 – Descrição do conjunto de dados SAT-6

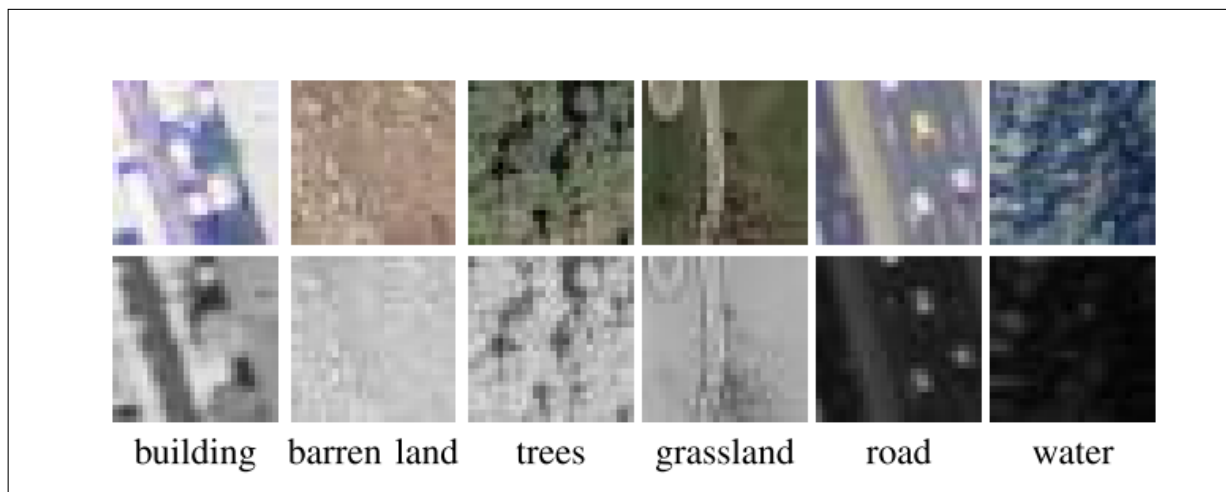
Característica	Descrição
Nome	SAT-6
Origem	DeepSat, com imagens extraídas do <i>National Agriculture Imagery Program</i> (NAIP)
Dimensão das amostras	Patches de 28×28 pixels
Bandas espectrais	4 bandas: vermelho, verde, azul e infravermelho próximo (RGB + NIR)
Total de imagens	405.000 imagens
Conjunto de treinamento	324.000 imagens
Conjunto de teste	81.000 imagens
Número de classes	6 classes
Classes	<i>barren land</i> , <i>trees</i> , <i>grassland</i> , <i>roads</i> , <i>buildings</i> e <i>water bodies</i>

Fonte: Adaptado de Basu et al. (2015).

A Figura 15 apresenta amostras representativas das seis classes do dataset SAT-6. Cada

coluna representa uma categoria de cobertura terrestre, sendo exibida uma composição RGB e uma representação associada ao canal de infravermelho próximo. Essa visualização evidencia a natureza multiespectral do conjunto de dados, característica importante para a tarefa de classificação.

Figura 15 – Exemplos de imagens RGB e NIR do SAT-6.



Fonte: Adaptado de Ely et al. (2024).

3.5 FLUXOS DE IMPLEMENTAÇÃO NAS PLATAFORMAS EMBARCADAS

Esta seção descreve os fluxos de implantação utilizados para executar a inferência dos modelos nas três plataformas avaliadas: PYNQ-Z2, NVIDIA Jetson Orin Nano e MAX78000. Embora o problema de classificação seja o mesmo, cada plataforma possui um ecossistema próprio de ferramentas, formatos de modelo, restrições de hardware e etapas de validação.

A aplicação de modelos de inteligência artificial em plataformas embarcadas não consiste apenas em treinar uma rede neural e executá-la diretamente no dispositivo. Em cada caso, foi necessário adaptar o modelo ao ambiente de execução ou ao fluxo de geração de hardware, converter formatos, preparar os dados de entrada, aplicar quantização quando necessário e coletar resultados compatíveis com cada plataforma. Dessa forma, os fluxos apresentados a seguir descrevem as etapas adotadas para transformar os modelos treinados em soluções adequadas à inferência embarcada ou à geração de aceleradores em FPGA.

3.5.1 Fluxo de implementação na PYNQ-Z2 com FINN

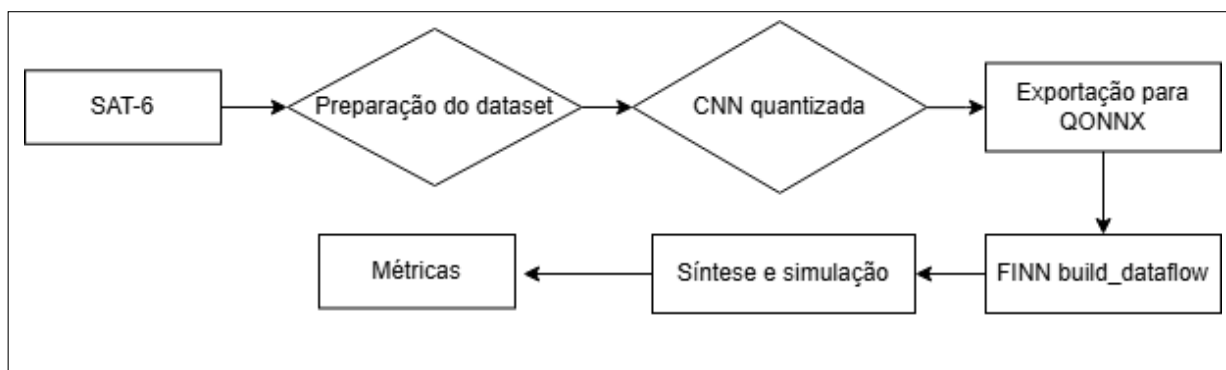
O fluxo associado à PYNQ-Z2 foi baseado na reprodução do processo de geração de acelerador para CNN quantizada aplicado ao dataset SAT-6, seguindo a estrutura disponibilizada no repositório de referência utilizado neste trabalho. Esse fluxo utiliza PyTorch e Brevitas para definição e treinamento da rede neural convolucional quantizada, QONNX como formato

intermediário de representação do modelo e FINN para transformação, síntese e simulação do acelerador em FPGA. O processo teve início com a preparação do dataset SAT-6. As amostras originais, compostas por imagens multiespectrais de 28×28 pixels com quatro bandas, foram organizadas para uso no fluxo convolucional de referência. Nesse fluxo, as imagens são adequadas ao formato $32 \times 32 \times 4$, preservando os quatro canais espectrais e permitindo a aplicação das topologias CNN utilizadas no estudo com FINN.

Em seguida, foi realizado o treinamento da CNN quantizada em ambiente Python, utilizando PyTorch e Brevitas. A arquitetura considerada é composta por camadas convolucionais quantizadas, funções de ativação quantizadas e uma camada final totalmente conectada responsável pela classificação nas seis classes do SAT-6. A utilização de quantização em pesos e ativações é uma etapa central do fluxo, pois o FINN explora modelos de baixa precisão para gerar arquiteturas de hardware especializadas e eficientes em FPGA. Após o treinamento, o modelo foi exportado para o formato QONNX. Esse formato representa a rede neural quantizada de maneira compatível com as transformações aplicadas pelo FINN. A partir do modelo QONNX, o FINN realizou uma sequência de etapas de preparação e transformação do grafo, incluindo limpeza da representação, inferência de tipos de dados, adequação de operadores, conversão de camadas convolucionais para estruturas compatíveis com o fluxo de hardware e especialização das camadas para implementação em arquitetura *dataflow*.

Dessa forma, a abordagem com PYNQ-Z2 neste trabalho concentra-se na reprodução do fluxo de geração de acelerador com FINN para uma CNN quantizada aplicada ao SAT-6. O objetivo é documentar e avaliar o processo de transformação de uma rede neural convolucional quantizada em uma arquitetura de hardware para FPGA, permitindo comparação com as demais plataformas a partir das métricas e relatórios disponíveis no fluxo de síntese e simulação.

Figura 16 – Fluxo de implementação na PYNQ-Z2 com FINN.



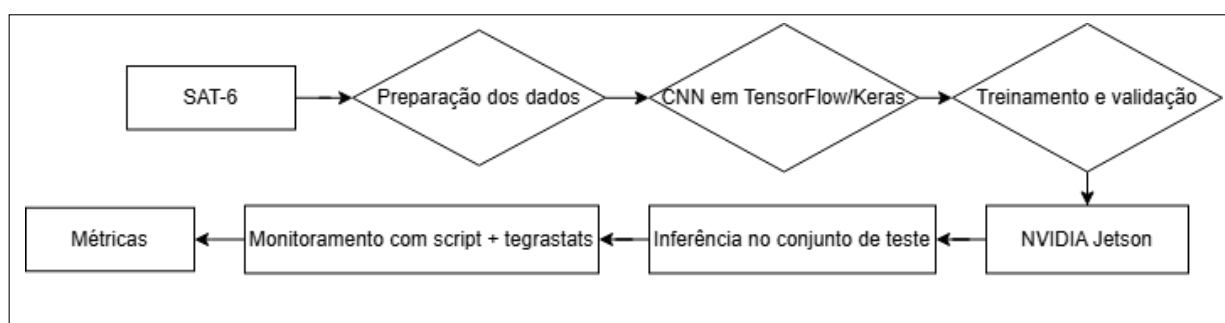
Fonte: Autor (2026).

3.5.2 Fluxo de implementação na NVIDIA Jetson Orin Nano

A NVIDIA Jetson Orin Nano foi utilizada como plataforma embarcada com GPU integrada. O fluxo de implementação nessa plataforma foi mais flexível em relação às demais, pois a Jetson permite executar modelos de aprendizado profundo em um ambiente Linux com suporte a bibliotecas de alto nível. O processo iniciou com a preparação do dataset SAT-6 e do modelo em Python. As amostras foram organizadas no formato esperado pela rede neural e normalizadas conforme a entrada definida no treinamento. O modelo utilizado na Jetson foi uma rede neural convolucional implementada com TensorFlow/Keras, adequada para a tarefa de classificação de imagens multiespectrais.

Após o treinamento e validação do modelo, a execução foi realizada na Jetson para avaliar a inferência em ambiente embarcado com GPU. O conjunto de teste foi utilizado para medir a acurácia de classificação e gerar a matriz de confusão. A latência foi medida diretamente no script de inferência, considerando o tempo necessário para processar as amostras de teste. A medição de consumo foi realizada com auxílio das ferramentas de monitoramento disponíveis na plataforma, como o `tegrastats`, permitindo acompanhar o comportamento da placa durante a execução. Essa etapa permitiu comparar a abordagem baseada em GPU com as demais soluções embarcadas avaliadas neste trabalho.

Figura 17 – Fluxo de implementação na NVIDIA Jetson Orin Nano.



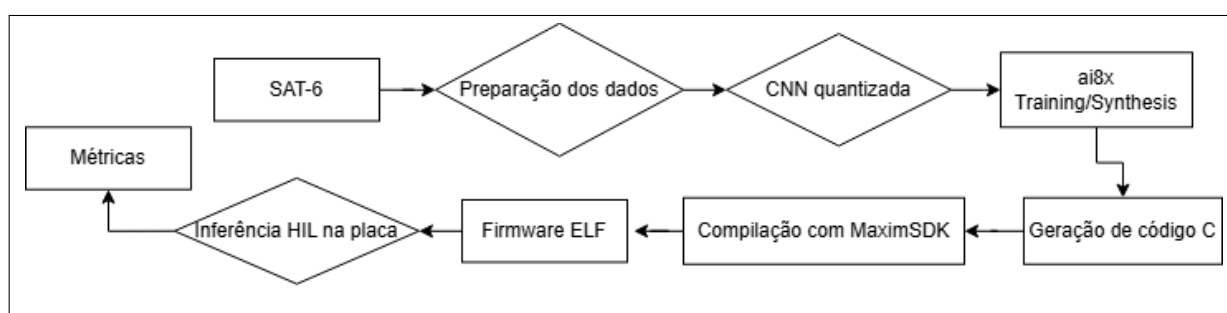
Fonte: Autor (2026).

3.5.3 Fluxo de implementação na MAX78000

A implementação na MAX78000 utilizou o acelerador CNN integrado ao microcontrolador. Essa plataforma possui restrições mais rígidas de memória, operadores suportados e formato numérico, exigindo um fluxo de conversão específico para transformar o modelo treinado em firmware executável. O fluxo teve início com o treinamento e a preparação de uma CNN quantizada em ambiente externo. Após a definição do modelo compatível com o acelerador da MAX78000, foram utilizadas as ferramentas do ecossistema Analog Devices/Maxim para gerar os arquivos necessários à execução da rede na placa. Esse processo incluiu a geração dos pesos quantizados, dos arquivos em linguagem C e das funções responsáveis por configurar e acionar o acelerador CNN interno.

Em seguida, o projeto foi compilado com o MaximSDK, gerando o arquivo executável para a MAX78000. O firmware foi gravado na placa e a inferência foi validada em hardware real por meio de um fluxo de teste *hardware-in-the-loop*. Durante a execução, amostras do SAT-6 foram enviadas para a placa, processadas pelo acelerador CNN e comparadas com os respectivos rótulos reais. A validação da MAX78000 permitiu calcular a acurácia obtida diretamente em hardware. Para a análise energética, foi utilizado um modo de execução contínua de inferência, permitindo medir o consumo da placa durante a operação. Dessa forma, a avaliação da MAX78000 concentrou-se principalmente na acurácia real em hardware e no consumo energético medido durante a inferência.

Figura 18 – Fluxo de implementação na MAX78000.



Fonte: Autor (2026).

Um ponto importante desse fluxo é que a implementação embarcada não executa o modelo em Python. O Python foi utilizado no computador hospedeiro para preparação dos dados, automação dos testes e geração das métricas. A inferência na MAX78000 foi realizada pelo firmware em linguagem C, executando a rede quantizada no acelerador CNN interno do microcontrolador.

3.6 PLANO DE VERIFICAÇÃO

O plano de verificação tem como objetivo validar se a solução desenvolvida atende aos requisitos definidos para a classificação de imagens multiespectrais do dataset SAT-6 em abordagens de computação embarcada. A verificação considera tanto aspectos funcionais, relacionados à classificação correta das amostras, quanto aspectos não funcionais, relacionados ao desempenho, consumo energético, compatibilidade com os fluxos de implementação, geração de aceleradores e reprodutibilidade dos experimentos.

A verificação foi organizada considerando três abordagens avaliadas: NVIDIA Jetson Orin Nano, MAX78000 e PYNQ-Z2 com FINN. Cada uma delas possui características próprias de hardware e software, portanto os procedimentos de teste foram adaptados aos respectivos fluxos. Na Jetson, a verificação concentrou-se na execução do modelo em ambiente com

GPU embarcada. Na MAX78000, a validação ocorreu por meio da execução do firmware na placa e coleta das previsões em hardware real. Na abordagem com PYNQ-Z2, a verificação concentrou-se na reprodução do fluxo FINN para CNN quantizada, considerando treinamento, exportação para QONNX, transformação do modelo, síntese, simulação e análise dos relatórios gerados.

As métricas utilizadas para verificação incluem acurácia de classificação, latência ou tempo de inferência e consumo energético, conforme a disponibilidade de medição em cada abordagem. No caso do fluxo FINN, também foram considerados resultados associados à geração do acelerador, como latência estimada, frequência de operação, simulação e utilização de recursos de hardware, incluindo LUTs, FFs, BRAMs e DSPs. Além disso, foram observados aspectos práticos de implementação e reprodução, como geração de firmware, exportação para QONNX, execução do fluxo `build_dataflow`, compatibilidade dos dados de entrada e organização dos artefatos produzidos.

O Quadro 5 apresenta os testes utilizados para verificar os requisitos funcionais da solução, enquanto o Quadro 6 apresenta os testes associados aos requisitos não funcionais.

Quadro 5 – Verificação dos requisitos funcionais

Requisito	Tipo de teste	Resultado esperado
RF01	Execução da classificação de imagens multiespectrais do dataset SAT-6.	O sistema deve classificar as amostras do SAT-6 em uma das seis classes de cobertura terrestre.
RF02	Verificação do formato de entrada utilizado em cada abordagem.	As amostras devem ser processadas com quatro canais espectrais, respeitando o formato exigido por cada modelo e por cada fluxo de desenvolvimento.
RF03	Validação dos modelos adotados para cada abordagem.	A Jetson Orin Nano, a MAX78000 e o fluxo FINN voltado à PYNQ-Z2 devem utilizar modelos baseados em CNN, respeitando as restrições de cada ferramenta e plataforma.
RF04	Verificação dos fluxos de execução ou geração de acelerador.	A inferência deve ser executada na Jetson Orin Nano e na MAX78000, enquanto o fluxo da PYNQ-Z2 deve gerar e analisar um acelerador CNN quantizado por meio do FINN.
RF05	Coleta dos resultados experimentais e dos relatórios de ferramenta.	Devem ser obtidos resultados de acurácia, latência ou tempo de inferência e consumo energético, conforme a disponibilidade de cada abordagem.

Fonte: Autor (2026).

Quadro 6 – Verificação dos requisitos não funcionais

Requisito	Tipo de teste	Resultado esperado
RNF01	Execução dos fluxos de treinamento, preparação de dados e validação inicial em Python.	Os modelos e scripts devem ser executáveis em ambiente Python, utilizando bibliotecas compatíveis com cada fluxo de desenvolvimento.
RNF02	Aplicação de quantização e otimização quando exigido pela plataforma ou ferramenta.	Os modelos destinados à MAX78000 e ao fluxo FINN devem ser preparados em formatos compatíveis com baixa precisão numérica e com as restrições dos respectivos aceleradores.
RNF03	Verificação do fluxo de execução na Jetson Orin Nano.	O modelo CNN deve ser executado em ambiente embarcado.
RNF04	Verificação do fluxo de firmware da MAX78000.	O firmware deve ser compilado, gravado e executado na placa, utilizando o acelerador CNN integrado ao microcontrolador.
RNF05	Verificação do fluxo FINN voltado à PYNQ-Z2.	A CNN quantizada deve ser treinada, exportada para QONNX e processada pelo FINN, permitindo a geração, síntese, simulação e análise do acelerador em FPGA.
RNF06	Separação entre resultados experimentais, simulação, síntese e estimativas de ferramenta.	Os resultados devem ser diferenciados conforme sua origem, evitando misturar medições reais em hardware com resultados obtidos por software.
RNF07	Análise comparativa entre as abordagens.	A comparação deve considerar acurácia, latência ou tempo de inferência, consumo energético, complexidade de implantação, limitações de cada fluxo e, no caso do FINN, utilização de recursos de hardware.

Fonte: Autor (2026).

3.7 METODOLOGIA EXPERIMENTAL

Esta seção apresenta os materiais e métodos utilizados para a execução experimental do trabalho. A metodologia foi estruturada para permitir a comparação entre três plataformas embarcadas com características distintas: uma plataforma com GPU embarcada, uma plataforma TinyML com acelerador CNN dedicado e uma plataforma SoC-FPGA com acelerador gerado por FINN.

A avaliação experimental foi conduzida a partir do mesmo problema de classificação,

utilizando o dataset SAT-6. Entretanto, os fluxos de treinamento, conversão, implantação e validação foram adaptados conforme as restrições e ferramentas de cada plataforma. Essa abordagem permitiu analisar não apenas a acurácia dos modelos, mas também aspectos práticos de implantação, latência, vazão, consumo energético e complexidade de desenvolvimento.

3.7.1 Materiais

Para a execução do trabalho, foram utilizados recursos de hardware para desenvolvimento, plataformas embarcadas alvo e ferramentas de software específicas para treinamento, conversão, síntese, compilação e validação dos modelos. O ambiente de desenvolvimento foi utilizado para preparação dos dados, treinamento dos modelos, organização dos arquivos, execução de scripts de validação e, no caso da PYNQ-Z2, execução do fluxo FINN com ferramentas AMD/Xilinx. As plataformas embarcadas foram utilizadas para validação da inferência em hardware real e coleta das métricas experimentais.

3.7.2 Hardware

O Quadro 7 apresenta os principais recursos de hardware utilizados no desenvolvimento e na validação experimental.

Quadro 7 – Hardware utilizado no desenvolvimento e validação experimental

Item	Descrição
Computador de desenvolvimento	Utilizado para preparação dos dados, treinamento dos modelos, execução de scripts, organização dos resultados e execução dos ambientes de desenvolvimento necessários aos fluxos das plataformas.
NVIDIA Jetson Orin Nano	Plataforma embarcada com GPU integrada, utilizada para execução e avaliação da inferência com modelo de rede neural em ambiente Linux embarcado.
MAX78000	Microcontrolador com acelerador CNN integrado, utilizado para validação de inferência TinyML em hardware real.
PYNQ-Z2	Plataforma SoC-FPGA baseada em Zynq, utilizada para execução do acelerador gerado pelo fluxo FINN.
Instrumentos de medição	Ferramentas e equipamentos utilizados para coleta de consumo energético, incluindo monitoramento por software quando disponível e medição elétrica externa nos casos aplicáveis.

Fonte: Autor (2026).

3.7.3 Software e ferramentas

O ambiente de software foi organizado de acordo com o ecossistema de cada plataforma. De forma geral, a linguagem Python foi utilizada na preparação dos dados, treinamento dos modelos, validação inicial e geração de resultados. Além disso, foram aplicadas ferramentas específicas para conversão, síntese, compilação e execução embarcada.

No fluxo da NVIDIA Jetson Orin Nano, empregaram-se instrumentos compatíveis com o ambiente Jetson, incluindo JetPack, suporte à GPU integrada e bibliotecas de aprendizado profundo. O modelo foi preparado em Python com TensorFlow/Keras e executado na plataforma para avaliação da inferência. A coleta de métricas incluiu scripts de validação e ferramentas de monitoramento da própria Jetson, como o `tegrastats`, utilizado para acompanhar o comportamento da plataforma durante a execução.

No fluxo da MAX78000, foram utilizadas ferramentas do ecossistema Analog Devices/-Maxim para conversão do modelo, geração de código em linguagem C, compilação do firmware e gravação na placa. O processo incluiu o uso de ferramentas de síntese e compilação compatíveis com o acelerador CNN integrado ao microcontrolador. A validação foi realizada por meio de execução em hardware real, permitindo a coleta das predições e a construção das métricas de classificação.

No fluxo voltado à PYNQ-Z2, foram utilizadas as bibliotecas PyTorch e Brevitas para definição, treinamento e quantização da CNN aplicada ao dataset SAT-6. Após o treinamento, o modelo foi exportado para o formato QONNX e processado pelo framework FINN. O FINN foi responsável pelas etapas de transformação do grafo, adequação dos operadores, geração da arquitetura *dataflow*, síntese, simulação e geração dos relatórios de análise do acelerador. Nesta etapa, o fluxo foi utilizado para avaliar a geração de hardware para uma CNN quantizada, sem considerar a execução física final na placa.

O Quadro 8 resume as principais ferramentas utilizadas em cada etapa do trabalho.

Quadro 8 – Software e ferramentas utilizados

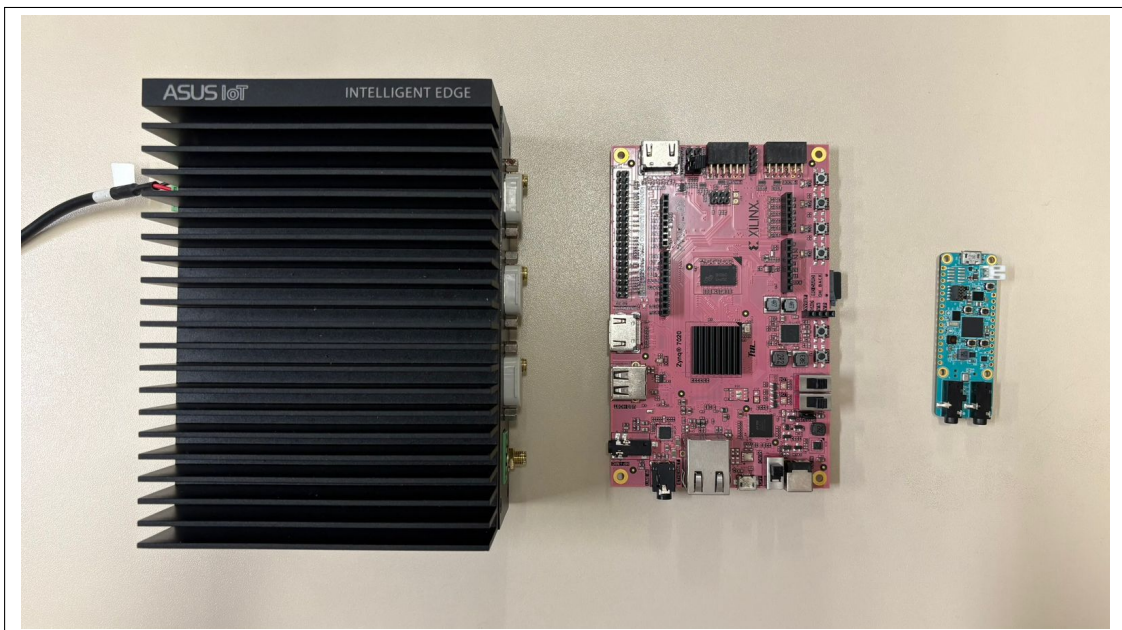
Ferramenta	Finalidade no trabalho
Python	Preparação dos dados, treinamento, validação, scripts de inferência e geração de métricas.
TensorFlow/Keras	Desenvolvimento e execução do modelo utilizado no fluxo da Jetson Orin Nano.
PyTorch	Treinamento do modelo quantizado utilizado no fluxo FINN da PYNQ-Z2.
Brevitas	Treinamento ciente de quantização e exportação de modelo compatível com o fluxo QONNX/FINN.
FINN	Geração do acelerador em FPGA a partir do modelo quantizado, incluindo transformações, síntese, simulação e arquivos de implantação.
Vivado/Vitis HLS 2023.2	Ferramentas AMD/Xilinx utilizadas no fluxo de síntese e implementação associado ao FINN.
MaximSDK e ferramentas Analog Devices/Maxim	Compilação, geração e gravação do firmware utilizado na MAX78000.
tegrastats	Monitoramento da Jetson durante a execução, incluindo acompanhamento de uso da plataforma e consumo.
Git	Organização e controle de versão dos arquivos do projeto.

Fonte: Autor (2026).

4 IMPLEMENTAÇÃO

Neste capítulo, apresenta-se a implementação prática da solução proposta para classificação de imagens multiespectrais do dataset SAT-6 em abordagens de computação embarcada. A implementação foi conduzida a partir do projeto metodológico definido no capítulo anterior e contemplou três fluxos distintos: uma plataforma com GPU embarcada, representada pela NVIDIA Jetson Orin Nano; uma plataforma TinyML com acelerador CNN dedicado, representada pela MAX78000; e uma abordagem SoC-FPGA, representada pelo fluxo FINN com alvo na PYNQ-Z2.

Figura 19 – Plataformas embarcadas utilizadas nos experimentos.



Fonte: Autor (2026).

A implementação não consistiu apenas no treinamento de modelos de classificação. Para cada abordagem, foi necessário preparar os dados, adaptar o modelo ao respectivo ambiente, converter formatos, gerar artefatos compatíveis e validar os resultados de acordo com as possibilidades de cada fluxo. Na Jetson Orin Nano, o foco esteve na execução de uma rede neural convolucional em ambiente Linux embarcado com suporte à GPU. Na MAX78000, o foco foi a conversão da CNN quantizada para firmware executável no acelerador CNN integrado. No fluxo associado à PYNQ-Z2, o foco foi a reprodução do processo de geração de acelerador em FPGA a partir de uma CNN quantizada, utilizando PyTorch, Brevitas, QONNX e FINN.

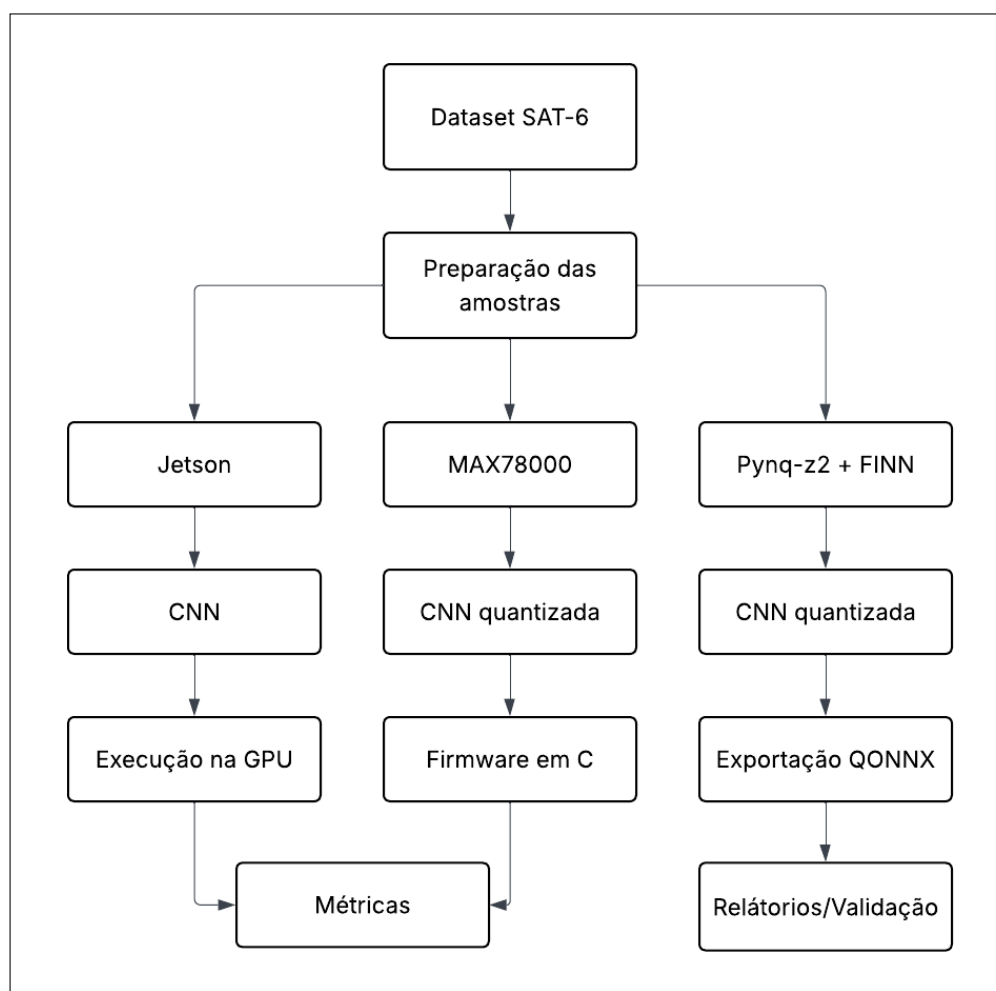
Dessa forma, este capítulo descreve os procedimentos adotados para preparação das amostras, implementação dos modelos, adaptação aos fluxos de cada plataforma e validação funcional dos experimentos. Os resultados quantitativos de acurácia, latência ou tempo de

inferência, consumo energético e, no caso do FINN, síntese, simulação e utilização de recursos de hardware são apresentados posteriormente no capítulo de resultados.

4.1 VISÃO GERAL DA IMPLEMENTAÇÃO

A implementação foi organizada em quatro etapas principais. A primeira correspondeu à preparação do conjunto de dados SAT-6, incluindo carregamento dos arquivos, reorganização das amostras e adequação do formato de entrada para cada fluxo. A segunda etapa envolveu a definição e o treinamento dos modelos de inteligência artificial, considerando as restrições de cada plataforma ou ferramenta. A terceira etapa consistiu na conversão, compilação, síntese ou geração dos artefatos necessários, por meio de ferramentas específicas como TensorFlow/Keras, ecossistema Analog Devices/Maxim e PyTorch/Brevitas/QONNX/FINN. A quarta etapa correspondeu à execução dos testes, validações, simulações e coleta dos dados necessários para avaliação.

Figura 20 – Visão geral do fluxo de implementação nas plataformas avaliadas.



Fonte: Autor (2026).

A Figura 20 apresenta uma visão geral do fluxo de implementação adotado nas três plataformas avaliadas. Todas as abordagens partiram do dataset SAT-6 e da preparação das amostras, porém cada plataforma exigiu um processo próprio de adaptação e execução. Na Jetson Orin Nano, o modelo CNN foi executado em GPU embarcada. Na MAX78000, a CNN quantizada foi convertida para firmware em linguagem C e executada no acelerador CNN interno. Na PYNQ-Z2, a CNN quantizada foi exportada para QONNX e processada no fluxo FINN, permitindo a geração e análise de aceleradores em FPGA.

4.2 CONJUNTO DE DADOS E PREPARAÇÃO DAS AMOSTRAS

O conjunto de dados empregado foi o SAT-6, definido no projeto como base experimental deste trabalho. Esse dataset é composto por patches de imagens de sensoriamento remoto, cada um com dimensões de 28×28 pixels e quatro bandas espectrais: vermelho, verde, azul e infravermelho próximo. As amostras estão distribuídas em seis classes de cobertura do solo: *Barren Land*, *Trees*, *grassland*, *Roads*, *Buildings* e *water bodies*.

Os arquivos utilizados foram armazenados em formato CSV, separados entre dados e rótulos de treinamento e teste. Nesse formato, cada imagem é armazenada de forma linearizada, com 3136 valores, correspondentes ao produto entre altura, largura e quantidade de canais:

$$28 \times 28 \times 4 = 3136 \quad (4.1)$$

Essa representação linearizada corresponde apenas à forma de armazenamento dos dados nos arquivos CSV. Para os modelos convolucionais, as amostras foram reorganizadas novamente no formato de imagem multiespectral, preservando a estrutura espacial e os quatro canais espectrais. Na etapa de preparação, as amostras foram reorganizadas de acordo com o formato exigido por cada fluxo. Para a Jetson Orin Nano, os dados foram reconstruídos no formato de imagem multiespectral e normalizados conforme o padrão de entrada do modelo em TensorFlow/Keras. Para a MAX78000, a preparação considerou as restrições do firmware e do acelerador CNN integrado, adequando as amostras ao formato esperado pela aplicação embarcada.

No fluxo associado à PYNQ-Z2 com FINN, as amostras também foram tratadas como imagens multiespectrais, e não como vetores de atributos para uma rede totalmente conectada. Como as topologias CNN de referência utilizadas no fluxo FINN trabalham com entrada $32 \times 32 \times 4$, as imagens originais do SAT-6 foram adequadas a esse formato, preservando os quatro canais espectrais. Essa etapa permitiu empregar a estrutura convolucional necessária para o treinamento da CNN quantizada com Brevitas e posterior exportação para QONNX.

4.3 MODELOS IMPLEMENTADOS

Os modelos implementados foram definidos considerando as restrições e possibilidades de cada plataforma. Embora o problema de classificação permaneça invariável, a arquitetura e o fluxo de preparação divergem entre os cenários, pois cada hardware ou ferramenta possui requisitos específicos de formato, operadores suportados, precisão numérica e mecanismos de aceleração.

Na Jetson Orin Nano, foi adotada uma rede neural convolucional para classificação das imagens multiespectrais do SAT-6. A escolha por uma CNN é adequada para esse tipo de dado, pois a estrutura espacial das imagens contém padrões relevantes para distinguir classes como estradas, áreas construídas, vegetação e corpos d'água. Na MAX78000, também foi utilizada uma CNN, porém adaptada e quantizada para execução no acelerador CNN interno da placa. A arquitetura foi mantida compacta para se adequar às restrições de memória e aos operadores suportados pelo ecossistema da plataforma. No fluxo voltado à PYNQ-Z2, foi utilizada uma CNN quantizada definida em PyTorch com apoio da biblioteca Brevitas. Essa rede foi baseada em camadas convolucionais quantizadas, ativações quantizadas e uma camada final totalmente conectada responsável pela classificação nas seis classes do SAT-6. Após o treinamento, o modelo foi exportado para QONNX e processado pelo FINN para geração e análise de um acelerador em FPGA.

Dessa forma, a implementação comparou três abordagens práticas baseadas em redes neurais convolucionais aplicadas ao mesmo problema de classificação: CNN em uma plataforma com GPU embarcada, CNN quantizada em um microcontrolador com acelerador dedicado e CNN quantizada em um fluxo SoC-FPGA com geração de acelerador por FINN.

4.4 IMPLEMENTAÇÃO NA PLATAFORMA MAX78000

A implementação na plataforma MAX78000 teve como objetivo avaliar a execução de inferência embarcada em um microcontrolador de ultra baixo consumo com acelerador CNN integrado. Diferentemente da Jetson Orin Nano, que utiliza uma GPU embarcada em ambiente Linux, e da PYNQ-Z2, que utiliza lógica programável em FPGA, a MAX78000 executa a inferência por meio de um acelerador neural dedicado, integrado ao próprio microcontrolador. Nesta etapa, foi utilizada uma rede neural convolucional quantizada para classificação de imagens multiespectrais do dataset SAT-6. O fluxo de implementação envolveu a adaptação do modelo às restrições da plataforma, a geração de código em linguagem C, a compilação do firmware, a gravação na placa e a validação da inferência por meio de testes *hardware-in-the-loop*.

4.4.1 Adequação do modelo ao acelerador CNN

A conversão do modelo gerou arquivos em linguagem C responsáveis por representar e controlar a rede neural no acelerador CNN da MAX78000. Entre esses arquivos, destaca-se o `cnn.h`, gerado automaticamente para a rede `sat6`, contendo as chamadas utilizadas para habilitar, configurar, executar e descarregar os resultados do acelerador. O Código 4.1 apresenta um trecho dessa interface.

Código 4.1 – Interface gerada para controle da CNN na MAX78000

```
1 int cnn_enable(uint32_t clock_source, uint32_t clock_divider);  
2 int cnn_init(void);  
3 int cnn_load_weights(void);  
4 int cnn_load_bias(void);  
5 int cnn_configure(void);  
6 int cnn_start(void);  
7 int cnn_unload(uint32_t *out_buf);
```

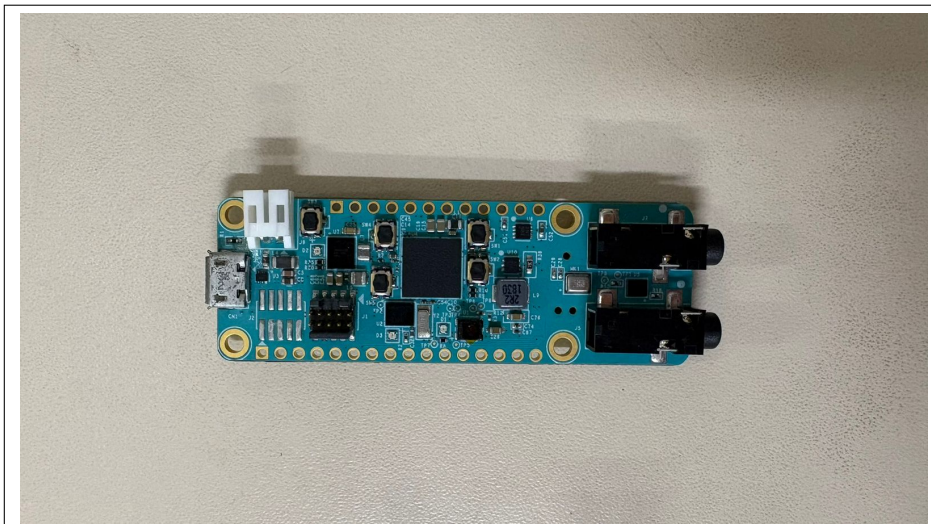
Esse trecho evidencia que o modelo treinado e quantizado foi convertido para uma interface de firmware específica da plataforma. As funções geradas permitem inicializar o acelerador, carregar os pesos e vieses da rede, configurar a arquitetura, iniciar a inferência e recuperar os valores de saída produzidos pela CNN.

4.4.2 Geração e gravação do firmware

A implementação embarcada na MAX78000 foi realizada por meio de firmware em linguagem C. O firmware gerado é responsável por inicializar a placa, configurar o acelerador CNN, organizar os dados de entrada na região de memória esperada, acionar a inferência e disponibilizar os resultados de saída. Durante esse processo, foram utilizados arquivos gerados a partir do modelo quantizado, juntamente às ferramentas de compilação compatíveis com o ecossistema da MAX78000. A compilação resultou em um binário executável, posteriormente gravado na memória da placa. Adotou-se um rigoroso controle de rastreabilidade para assegurar a correspondência entre o firmware gravado e a respectiva rede neural. Dado que múltiplos ciclos de treinamento, conversão e compilação geram diversos arquivos intermediários, fez-se indispensável auditar a integridade dos pesos, os artefatos do projeto compilado e o arquivo binário efetivamente transferido para a placa. Esse procedimento evitou quaisquer discrepâncias entre a arquitetura e o modelo executado.

A Figura 21 apresenta a plataforma MAX78000 utilizada na validação embarcada.

Figura 21 – Plataforma MAX78000 utilizada na validação embarcada.



Fonte: Autor (2026).

4.4.3 Validação por teste *hardware-in-the-loop*

No firmware embarcado, a placa permanece aguardando o envio de uma amostra pelo computador hospedeiro. Ao receber o comando de início, a MAX78000 confirma a comunicação, recebe os dados da imagem pela UART, carrega a entrada na memória do acelerador, executa a inferência e retorna a classe prevista. O Código 4.2 apresenta um trecho reduzido dessa rotina.

Código 4.2 – Execução da inferência HIL na MAX78000

```

1 #define IMG_BYTES (28 * 28 * 4)
2 #define SAT6_NUM_CLASSES 6
3
4 cnn_enable(MXC_S_GCR_PCLKDIV_CNNCLKSEL_PCLK,
5 MXC_S_GCR_PCLKDIV_CNNCLKDIV_DIV1);
6 cnn_init();
7 cnn_load_weights();
8 cnn_load_bias();
9 cnn_configure();
10
11 while (1) {
12     if (uart_getc() != 'S') {
13         continue;
14     }
15
16     uart_putc('K');
17     uart_read_bytes((uint8_t *) image_buf32, IMG_BYTES);
18     load_cnn_input();
19     cnn_time = 0;
20     cnn_start();

```

```

21 while (cnn_time == 0) {
22     __WFI();
23 }
24 cnn_unload(cnn_out_words);
25 uint16_t *cnn_out_raw = (uint16_t *) cnn_out_words;
26 for (int i = 0; i < SAT6_NUM_CLASSES; i++) {
27     cnn_out_buf[i] = (int8_t) (cnn_out_raw[i] & 0xff);
28 }
29 send_result(argmax_int16(cnn_out_buf, SAT6_NUM_CLASSES));
30 }

```

Esse trecho resume a validação *hardware-in-the-loop*. O computador envia uma imagem do SAT-6 pela comunicação serial, a MAX78000 executa a inferência no acelerador CNN e retorna a classe prevista. A função `cnn_start` aciona o processamento, enquanto `cnn_unload` recupera as saídas da rede. Em seguida, a função `argmax_int16` seleciona a classe com maior valor de saída, permitindo ao script no computador comparar a predição com o rótulo real da amostra.

4.4.4 Registro das predições e preparação das métricas

Durante a validação HIL, as predições retornadas pela MAX78000 foram armazenadas pelo script executado no computador hospedeiro. Para cada amostra avaliada, foram registrados o rótulo real, a classe prevista pela placa e, quando disponível, os valores de saída da rede neural. Esses registros foram utilizados posteriormente para calcular as métricas de avaliação, como acurácia, matriz de confusão, acurácia por classe e erros por classe. A apresentação quantitativa desses resultados é realizada no capítulo de resultados, enquanto esta seção limita-se à descrição do processo de implementação e validação funcional da plataforma. Além da validação de classificação, a MAX78000 também foi empregada em um modo de inferência contínua para medição de consumo energético. Esse procedimento permitiu avaliar o comportamento da placa durante operação repetitiva do acelerador CNN. A descrição detalhada da medição e os valores obtidos são apresentados na seção de resultados energéticos.

4.4.5 Resumo da implementação na MAX78000

O Quadro 9 resume os principais elementos da implementação realizada na plataforma MAX78000.

Quadro 9 – Resumo da implementação na MAX78000

Elemento	Descrição
Plataforma	MAX78000, microcontrolador com acelerador CNN integrado.
Tipo de modelo	Rede neural convolucional quantizada para classificação de imagens multiespectrais.
Dataset	SAT-6, composto por imagens de $28 \times 28 \times 4$ pixels e seis classes de cobertura terrestre.
Formato de execução	Firmware em linguagem C gravado diretamente na placa.
Acelerador utilizado	Acelerador CNN interno da MAX78000.
Entrada do modelo	Amostras do SAT-6 preparadas no formato esperado pelo firmware embarcado.
Validação	Teste <i>hardware-in-the-loop</i> com envio de amostras por comunicação serial e retorno da classe prevista.
Saída registrada	Rótulo real, classe prevista pela placa e dados utilizados para cálculo das métricas.
Métricas preparadas	Acurácia, latência, acurácia por classe e consumo energético.

Fonte: Autor (2026).

4.5 IMPLEMENTAÇÃO NA PLATAFORMA NVIDIA JETSON ORIN NANO

A implementação na NVIDIA Jetson Orin Nano teve como objetivo avaliar a execução de inferência embarcada em uma plataforma com GPU integrada. Diferentemente da MAX78000, que exige a conversão do modelo para firmware e execução em um acelerador CNN dedicado, a Jetson oferece um ambiente Linux mais flexível, permitindo a utilização de bibliotecas de aprendizado profundo em Python e suporte à aceleração por GPU. Nesta etapa, a referida plataforma executou a inferência de uma rede neural convolucional voltada à classificação de imagens multiespectrais do dataset SAT-6. O escopo da implementação abrangeu a preparação do ambiente, a organização dos dados de entrada, a execução do modelo em hardware, a mensuração da latência e o perfilamento do consumo energético durante os ensaios.

A Figura 22 apresenta a plataforma utilizada na etapa de implementação e validação da inferência embarcada com GPU.

Figura 22 – Plataforma utilizada na implementação com NVIDIA Jetson Orin Nano.



Fonte: Autor (2026).

4.5.1 Preparação do ambiente

A Jetson Orin Nano foi configurada com o ambiente NVIDIA JetPack, que fornece os componentes necessários para execução de aplicações aceleradas por GPU na plataforma. Durante a preparação, foram verificados o funcionamento do sistema operacional embarcado, o suporte à GPU, as bibliotecas de aprendizado profundo e as ferramentas auxiliares de monitoramento. O ambiente de execução foi preparado em Python, utilizando bibliotecas compatíveis com o modelo adotado no trabalho. Também foram utilizadas ferramentas de monitoramento da própria plataforma, especialmente o `tegrastats`, que permite acompanhar informações como uso da GPU, temperatura e consumo estimado dos blocos de alimentação da placa.

Essa etapa foi importante para garantir que a inferência fosse executada no ambiente correto e que as métricas de desempenho fossem coletadas de forma padronizada. A configuração da plataforma também permitiu verificar a estabilidade da execução antes da realização dos testes com o conjunto de imagens do SAT-6.

4.5.2 Preparação das amostras para inferência

As amostras do dataset SAT-6 foram carregadas a partir dos arquivos de dados e reorganizadas no formato esperado pelo modelo convolucional. Cada imagem possui dimensão original de $28 \times 28 \times 4$, correspondente a 28 pixels de altura, 28 pixels de largura e quatro bandas espectrais. Na preparação dos dados, as imagens foram reconstruídas como tensores multiespectrais, preservando os quatro canais de entrada. Em seguida, os valores foram normalizados conforme o padrão utilizado pelo modelo. Essa padronização foi necessária para garantir que

os dados usados na inferência embarcada fossem compatíveis com aqueles utilizados durante a etapa de treinamento e validação.

Após essa preparação, as amostras do conjunto de teste foram utilizadas para avaliar o comportamento do modelo na Jetson. Para cada imagem processada, foi registrada a classe prevista, permitindo posteriormente o cálculo da acurácia, da matriz de confusão e das demais métricas de avaliação.

4.5.3 Execução da inferência na Jetson

A execução da inferência na NVIDIA Jetson Orin Nano foi realizada diretamente no ambiente embarcado da plataforma, utilizando o modelo CNN treinado para classificação das seis classes do SAT-6. Inicialmente, o modelo exportado pelo fluxo TensorFlow/Keras foi carregado no script de inferência. Como etapa de validação, foi verificado se o modelo possuía entrada compatível com o formato das imagens do SAT-6, isto é, 28×28 pixels com quatro canais espectrais, conforme apresentado no Código 4.3.

Código 4.3 – Carregamento e validação do modelo TensorFlow/Keras na Jetson

```
1 import tensorflow as tf
2
3 model = tf.keras.models.load_model("sat6_tf_28_best.keras", compile=False)
4
5 print("Input shape:", model.input_shape)
6 print("Output shape:", model.output_shape)
7
8 if model.input_shape[1:] != (28, 28, 4):
9     raise ValueError(f"Modelo incorreto para SAT-6: {model.input_shape}")
```

Essa verificação foi necessária porque o SAT-6 utiliza imagens multiespectrais com quatro canais, diferentemente de imagens RGB convencionais. Dessa forma, o trecho assegura que o modelo carregado corresponde à arquitetura esperada antes da execução das inferências na placa.

Em seguida, os arquivos de teste do SAT-6 foram lidos e as amostras, originalmente armazenadas de forma linearizada, foram reconstruídas no formato $28 \times 28 \times 4$, preservando os quatro canais espectrais do dataset. Após a reconstrução das imagens, os dados foram normalizados conforme o mesmo padrão utilizado no treinamento. Essa etapa garantiu que as entradas processadas na Jetson mantivessem a mesma escala usada na validação do modelo. Com os dados preparados, o script executou a inferência sobre o conjunto de teste, registrando para cada amostra a classe prevista e o respectivo rótulo real. Para cada predição, a saída da rede foi interpretada a partir da classe com maior valor no vetor de saída. Em seguida, as predições foram

comparadas com os rótulos verdadeiros para cálculo da acurácia. Além disso, o script registrou os tempos de execução da inferência, permitindo calcular o tempo médio por imagem. Os testes também foram realizados com diferentes tamanhos de lote, a fim de observar o impacto do *batch size* no desempenho da GPU.

Além da execução em Python, o modelo também foi exportado no formato `SavedModel`, permitindo sua chamada por meio da API C do TensorFlow. Esse trecho representa a integração do modelo em uma aplicação em linguagem C, mantendo as dimensões de entrada do SAT-6 e o número de classes da tarefa de classificação, conforme apresentado no Código 4.4.

Código 4.4 – Chamada do modelo TensorFlow `SavedModel` por meio da API C

```

1 #define SAT6_HEIGHT 28
2 #define SAT6_WIDTH 28
3 #define SAT6_CHANNELS 4
4 #define FEATURE_COUNT (SAT6_HEIGHT * SAT6_WIDTH * SAT6_CHANNELS)
5 #define NUM_CLASSES 6
6
7 const char* model_path = "sat6_savedmodel_for_c/";
8 // ...
9 TF_Session* session = TF_LoadSessionFromSavedModel(...);
10
11 TF_Output input_op = { TF_GraphOperationByName(graph, input_op_name), 0 };
12 TF_Output output_op = { TF_GraphOperationByName(graph, output_op_name), 0
13     };
14
15 TF_SessionRun(
16     session, NULL,
17     &input_op, &input_tensor, 1,
18     &output_op, &output_tensor, 1,
19     // ...
20 );

```

No trecho em C, são definidas as dimensões da entrada e o número de classes de saída. Em seguida, o modelo salvo é carregado, as operações de entrada e saída são obtidas a partir do grafo computacional e a inferência é executada por meio da função `TF_SessionRun`. Esse fluxo demonstra a possibilidade de utilizar o modelo treinado em TensorFlow/Keras fora do ambiente Python, preservando a compatibilidade com a estrutura de dados utilizada no experimento. Durante os testes, a ferramenta `tegrastats` foi executada em paralelo para acompanhar o comportamento da Jetson, incluindo informações de uso da plataforma e consumo energético. Assim, a etapa de execução na Jetson gerou os dados necessários para avaliar a acurácia, o tempo de inferência e o consumo energético da abordagem baseada em GPU embarcada.

4.5.4 Medição de latência e consumo de energia

A latência de inferência foi medida diretamente no script de execução, registrando o tempo necessário para processar as amostras do conjunto de teste. A partir dessas medições, foram calculadas estatísticas como latência média, valor mínimo, valor máximo e, quando aplicável, desvio padrão. O consumo energético foi acompanhado por meio da ferramenta `tegrastats`, executada em paralelo ao script de inferência. Essa ferramenta permitiu registrar informações de consumo da plataforma durante a execução, incluindo a potência total aproximada associada ao campo `VDD_IN`, além de dados auxiliares como uso da GPU e temperatura. Para estimar a energia consumida por imagem, foi utilizada a relação entre potência média e tempo médio de inferência:

$$E_{\text{imagem}} = P_{\text{media}} \times t_{\text{inferencia}} \quad (4.2)$$

em que E_{imagem} representa a energia estimada por imagem, P_{media} corresponde à potência média observada durante a inferência e $t_{\text{inferencia}}$ representa a latência média por imagem. Os valores finais de acurácia, matriz de confusão, latência, vazão e consumo energético são apresentados no capítulo de resultados, de forma comparativa com as plataformas MAX78000 e PYNQ-Z2.

4.5.5 Resumo da implementação na Jetson

O Quadro 10 resume os principais elementos da implementação realizada na plataforma Jetson Orin Nano.

Quadro 10 – Resumo da implementação na NVIDIA Jetson Orin Nano

Elemento	Descrição
Plataforma	NVIDIA Jetson Orin Nano com GPU integrada.
Tipo de modelo	Rede neural convolucional para classificação de imagens multiespectrais.
Dataset	SAT-6, composto por imagens de $28 \times 28 \times 4$ pixels e seis classes de cobertura terrestre.
Ambiente de execução	Linux embarcado com JetPack e bibliotecas de aprendizado profundo em Python.
Entrada do modelo	Imagens multiespectrais reconstruídas e normalizadas conforme o padrão do modelo.
Medição de latência	Realizada no script de inferência, considerando o tempo de processamento das amostras avaliadas.
Medição de consumo	Monitoramento com <code>tegrastats</code> , utilizando a potência média durante a inferência.
Saída registrada	Classe prevista para cada amostra, tempos de inferência e dados para cálculo das métricas.

Fonte: Autor (2026).

4.6 FLUXO FINN NA PLATAFORMA PYNQ-Z2

O fluxo FINN voltado à plataforma PYNQ-Z2 foi utilizado como uma etapa complementar deste trabalho, com o objetivo de investigar a geração e a análise de aceleradores em FPGA para classificação de imagens multiespectrais do dataset SAT-6. Diferentemente da NVIDIA Jetson Orin Nano, que utiliza GPU embarcada, e da MAX78000, que possui acelerador CNN integrado ao microcontrolador, a PYNQ-Z2 representa uma abordagem baseada em hardware reconfigurável, utilizando a lógica programável do FPGA como alvo para aceleração de redes neurais quantizadas.

Nesta etapa, foi reproduzido o fluxo `sat6_cnn`, baseado em CNNs quantizadas, PyTorch, Brevitas, QONNX e FINN. O processo envolveu a preparação do dataset, definição e treinamento das redes convolucionais quantizadas, exportação dos modelos para QONNX, transformação dos grafos pelo FINN, execução do fluxo `build_dataflow` e análise dos relatórios de desempenho e área gerados. Para a etapa de análise do fluxo FINN, foram consideradas diferentes configurações de topologia e quantização: `t1w2_500fps`, `t1w4_500fps`, `t1w8_500fps`, `t2w2_500fps`, `t2w4_500fps` e `t2w8_500fps`. Essa análise permitiu observar o impacto da arquitetura convolucional e da largura de palavra sobre a vazão estimada e o uso de recursos do FPGA.

A execução física na PYNQ-Z2 foi tratada como validação complementar e foi realizada apenas com a configuração `t2w8_500fps`. As métricas obtidas nessa execução, como acurácia, latência e vazão real, são apresentadas no capítulo de resultados. Nesta seção, são descritas as etapas de preparação, geração e análise dos artefatos do fluxo FINN.

4.6.1 Organização do fluxo `sat6_cnn`

O fluxo utilizado foi organizado a partir da estrutura `sat6_cnn`, composta por notebooks e scripts voltados à preparação do dataset, definição das redes neurais, treinamento, exportação dos modelos e geração dos aceleradores com FINN. A organização sequencial do fluxo permitiu acompanhar separadamente cada etapa necessária para transformar CNNs quantizadas treinadas em software em representações compatíveis com implementação em FPGA. De forma geral, o fluxo pode ser resumido pelas seguintes etapas: preparação do dataset SAT-6, definição e treinamento das CNNs quantizadas, exportação para QONNX, transformação dos grafos pelo FINN, execução do `build_dataflow` e análise dos relatórios de área e desempenho. Essa estrutura foi mantida para garantir rastreabilidade entre os modelos treinados, os grafos exportados, as transformações aplicadas e os artefatos gerados ao final do processo.

4.6.2 Preparação do dataset para o fluxo FINN

O dataset SAT-6 é composto por imagens multiespectrais originalmente representadas com dimensão $28 \times 28 \times 4$, sendo quatro canais correspondentes às bandas vermelho, verde, azul e infravermelho próximo. Nos arquivos originais, cada amostra é armazenada de forma linearizada, totalizando 3136 valores por imagem:

$$28 \times 28 \times 4 = 3136 \quad (4.3)$$

No fluxo adotado, essa representação linearizada foi utilizada apenas como formato inicial de armazenamento dos dados. Para o treinamento e utilização das CNNs quantizadas, as amostras foram reconstruídas como imagens multiespectrais, preservando a relação espacial entre os pixels e os quatro canais espectrais. Como a arquitetura de referência utilizada no fluxo `sat6_cnn` opera com entrada no formato $32 \times 32 \times 4$, as imagens originais foram adequadas a esse formato. Essa adaptação manteve os quatro canais espectrais e permitiu utilizar operações convolucionais sobre as imagens. Assim, os dados não foram tratados como vetores para uma rede totalmente conectada, mas como imagens multiespectrais compatíveis com CNNs.

4.6.3 Definição das CNNs quantizadas

A configuração utilizada como referência no fluxo FINN foi baseada na topologia `t2`, descrita em Python com o apoio da biblioteca Brevitas. O Código 4.5 apresenta um trecho

reduzido da definição da CNN quantizada, destacando o uso de camadas convolucionais quantizadas, ativações quantizadas e camada linear final.

Código 4.5 – CNN quantizada com Brevitas para o fluxo FINN

```

1 import torch.nn as nn
2 import brevitass.nn as qnn
3
4 class t2_quantizedCNN(nn.Module):
5 def **init**(self, bit_quantization):
6 super(t2_quantizedCNN, self).**init**()
7
8     self.conv1 = qnn.QuantConv2d(
9         4, 16,
10        kernel_size=3, stride=2, padding=1,
11        bias=False,
12        weight_bit_width=bit_quantization,
13        weight_quant=CommonWeightQuant,
14    )
15
16    self.relu1 = qnn.QuantReLU(
17        act_quant=CommonUintActQuant,
18        bit_width=bit_quantization,
19        return_quant_tensor=True
20    )
21
22    # Demais camadas convolucionais quantizadas seguem a mesma estrutura
23
24    self.fc1 = qnn.QuantLinear(
25        8*2*2, 6,
26        bias=False,
27        weight_bit_width=bit_quantization,
28        weight_quant=CommonWeightQuant,
29    )
30
31 def forward(self, x):
32     x = self.relu1(self.conv1(x))
33     # Aplicacao sequencial das demais camadas quantizadas
34     x = x.view(x.size(0), -1)
35     x = self.fc1(x)
36     return x

```

Esse trecho resume a estrutura principal da rede utilizada no fluxo FINN. A entrada da primeira convolução possui quatro canais, compatíveis com as bandas do SAT-6, enquanto a camada final possui seis saídas, correspondentes às classes do dataset. O parâmetro `bit_quantization` controla a largura de palavra usada na quantização, permitindo gerar as dife-

rentes configurações avaliadas no trabalho.

4.6.4 Treinamento e validação inicial em software

O treinamento das CNNs quantizadas foi realizado em ambiente Python, utilizando PyTorch e Brevitas. As amostras preparadas do SAT-6 foram utilizadas para ajustar os pesos das redes, enquanto os rótulos correspondentes às seis classes foram empregados como referência para o cálculo do erro durante o treinamento. Após o treinamento, os modelos foram validados inicialmente em software. Essa etapa teve como objetivo verificar se as CNNs quantizadas apresentavam comportamento coerente antes da exportação para QONNX. A validação em software também serviu como referência para as etapas posteriores do fluxo, uma vez que diferenças de formato, escala de entrada ou representação numérica podem alterar o comportamento do modelo após a exportação e transformação. Os resultados quantitativos relacionados ao desempenho dos modelos são apresentados no capítulo de resultados. Nesta seção, o objetivo principal é descrever a preparação dos modelos para o fluxo FINN.

4.6.5 Exportação dos modelos para QONNX

Após a validação inicial em software, o modelo treinado foi carregado a partir dos pesos salvos em PyTorch e exportado para o formato QONNX. Esse formato foi utilizado como representação intermediária para permitir que as informações de quantização fossem interpretadas pelas etapas posteriores do fluxo FINN. O Código 4.6 apresenta o trecho responsável pelo carregamento dos pesos e pela exportação do modelo.

Código 4.6 – Exportacao do modelo quantizado para QONNX

```

1 from brevitass.export import export_qonnx
2 from qonnx.util.cleanup import cleanup as qonnx_cleanup
3 from qonnx.core.modelwrapper import ModelWrapper
4
5 def make_onnx(cnv, quant, topology):
6     cnv.load_state_dict(
7         torch.load(f"./pytorch_models/sat6-cnn-t{topology}w{quant}.pt")
8     )
9
10    onnx_output_filename = (
11        f"./hardware_onnxs/sat6-cnn-t{topology}w{quant}.onnx"
12    )
13
14    export_onnx_path = (
15        build_dir + f"/end2end_cnv_t{topology}w{quant}_export.onnx"
16    )
17
18    export_qonnx(cnv, torch.randn(1, 4, 32, 32), export_onnx_path)
19    qonnx_cleanup(export_onnx_path, out_file=export_onnx_path)
20

```

```

21 model = ModelWrapper(export_onnx_path)
22
23 return onnx_output_filename

```

Esse trecho evidencia a transição entre o modelo treinado em PyTorch/Brevitas e a representação QONNX utilizada pelo FINN. A entrada fictícia `torch.randn(1, 4, 32, 32)` define o formato esperado pelo modelo no fluxo de exportação, preservando os quatro canais espectrais utilizados no SAT-6.

4.6.6 Transformações e geração dos aceleradores com FINN

Com os modelos em formato QONNX, foi iniciado o processamento no FINN. Essa etapa aplicou transformações sucessivas aos grafos das redes, com o objetivo de preparar os modelos para geração de hardware. Entre as operações realizadas pelo fluxo estão a limpeza do grafo, inferência de tipos de dados, adequação de operadores, conversão de camadas para representações compatíveis e preparação da arquitetura em fluxo de dados. A geração dos aceleradores foi realizada por meio do fluxo `build_dataflow`. Esse processo automatiza diversas etapas necessárias para transformar redes neurais quantizadas em arquiteturas de hardware. Durante a execução, o FINN gera componentes intermediários, organiza as camadas em uma estrutura *dataflow*, aciona ferramentas de síntese e produz os artefatos necessários para análise e uso posterior.

Nesta implementação, o fluxo foi executado para as configurações `t1w2_500fps`, `t1w4_500fps`, `t1w8_500fps`, `t2w2_500fps`, `t2w4_500fps` e `t2w8_500fps`. Essa escolha permitiu comparar, ainda na etapa de geração e análise do FINN, o comportamento de diferentes níveis de quantização e de duas topologias convolucionais. Também foram considerados parâmetros relacionados ao paralelismo da implementação, como configurações de *folding*. Esses parâmetros influenciam a latência estimada, a frequência de operação, o tempo de síntese e o consumo de recursos do FPGA.

4.6.7 Relatórios de desempenho e área

Após a execução do fluxo de geração, foram analisados os relatórios produzidos pelo FINN e pelas ferramentas AMD/Xilinx associadas. Essa etapa incluiu a organização dos resultados de desempenho e de área, utilizados para caracterizar as arquiteturas geradas antes da validação física na placa. Os resultados de desempenho foram analisados a partir do notebook de *performance results*, que reúne informações de vazão para cada configuração de hardware gerada. Essa análise permitiu observar o comportamento das configurações `t1w2_500fps`, `t1w4_500fps`, `t1w8_500fps`, `t2w2_500fps`, `t2w4_500fps` e `t2w8_500fps` no fluxo FINN.

Além disso, foram analisados os resultados de área obtidos a partir do fluxo Vivado/OOC. Esses relatórios apresentam o uso de recursos lógicos e de memória da FPGA, incluindo LUTs, FFs, BRAMs e DSPs, além de informações como WNS e frequência máxima estimada. Esses dados são importantes para avaliar o custo da arquitetura gerada em relação à capacidade disponível no dispositivo alvo. Dessa forma, os notebooks de análise de desempenho e de área foram utilizados como parte da caracterização do fluxo FINN. Os valores quantitativos desses relatórios são apresentados no capítulo de resultados, onde são discutidos em conjunto com a validação complementar realizada na PYNQ-Z2 para a configuração `t2w8_500fps`.

4.6.8 Artefatos gerados pelo fluxo

Ao final do fluxo de geração, foram obtidos os artefatos necessários para análise e possível utilização dos aceleradores na plataforma PYNQ-Z2. Entre os principais arquivos gerados estão os diretórios de saída do FINN, os arquivos de bitstream, os arquivos de descrição de hardware e os drivers Python associados aos aceleradores. Os principais artefatos considerados foram os arquivos `.bit`, responsáveis pela configuração da FPGA, os arquivos `.hwh`, que descrevem a arquitetura de hardware gerada, e os arquivos de driver, como `driver.py`, `driver_base.py` e `validate.py`. Esses arquivos compõem o pacote de *deploy* produzido pelo FINN. Para fins de organização do trabalho, a análise de relatórios foi considerada para todas as configurações geradas no fluxo FINN. Já a etapa de execução física na PYNQ-Z2 foi limitada à configuração `t2w8_500fps`, sendo apresentada no capítulo 5 como validação complementar.

4.6.9 Resumo do fluxo FINN

O Quadro 11 resume os principais elementos do fluxo FINN voltado à PYNQ-Z2.

Quadro 11 – Resumo do fluxo FINN voltado à PYNQ-Z2

Elemento	Descrição
Plataforma alvo	PYNQ-Z2, baseada em SoC-FPGA Xilinx Zynq, utilizada como alvo para geração de aceleradores em FPGA por meio do FINN.
Dataset	SAT-6, composto por imagens multiespectrais com quatro canais e seis classes de saída.
Tipo de modelo	CNNs quantizadas, compostas por camadas convolucionais quantizadas.
Topologias avaliadas	t1 e t2, correspondentes a duas arquiteturas convolucionais consideradas no fluxo <code>sat6_cnn</code> .
Configurações analisadas no fluxo FINN	t1w2_500fps, t1w4_500fps, t1w8_500fps, t2w2_500fps, t2w4_500fps e t2w8_500fps.
Configuração utilizada na validação física	t2w8_500fps, adotada como modelo representativo para execução complementar na PYNQ-Z2.
Entrada do modelo	Imagem multiespectral no formato $32 \times 32 \times 4$, obtida a partir da adaptação das amostras originais $28 \times 28 \times 4$ do SAT-6.
Ferramentas de treinamento	PyTorch e Brevitas, com quantização de pesos e ativações.
Formato intermediário	QONNX, utilizado como entrada para o fluxo FINN.
Geração dos aceleradores	Realizada pelo FINN, incluindo transformações dos modelos, arquitetura <i>dataflow</i> , execução do <code>build_dataflow</code> e geração dos artefatos de <i>deploy</i> .
Relatórios analisados	Relatórios de desempenho do FINN e resultados de área Vivado/OOC, incluindo vazão, LUTs, FFs, BRAMs, DSPs, WNS e frequência máxima estimada.
Métricas apresentadas	Os valores quantitativos de desempenho, área e validação física complementar são apresentados no capítulo de resultados.

Fonte: Autor (2026).

5 TESTES E RESULTADOS

Neste capítulo, são apresentados os resultados obtidos na avaliação experimental das abordagens utilizadas neste trabalho: NVIDIA Jetson Orin Nano, MAX78000 e fluxo FINN voltado à PYNQ-Z2. Os testes tiveram como objetivo avaliar a classificação de imagens multi-espectrais do dataset SAT-6, considerando métricas como acurácia, latência, vazão e consumo energético, conforme a disponibilidade de medição em cada plataforma. A análise foi organizada de acordo com as características de cada abordagem. Na Jetson Orin Nano, avaliou-se a execução de uma CNN em uma plataforma embarcada com GPU integrada. Na MAX78000, avaliou-se a execução de uma CNN quantizada em um microcontrolador com acelerador CNN integrado. No fluxo FINN para a PYNQ-Z2, foram analisados os resultados de geração de aceleradores em FPGA.

É importante destacar que a comparação entre as abordagens não se limita à acurácia final. Cada plataforma possui um fluxo próprio, com diferentes restrições de modelo, precisão numérica, ferramentas de desenvolvimento, formas de aceleração e métodos de medição. Dessa forma, os resultados devem ser interpretados considerando os compromissos entre desempenho, consumo energético, complexidade de implantação, restrições de hardware e fidelidade entre o modelo treinado e sua forma final de execução.

5.1 CRITÉRIOS DE AVALIAÇÃO

As métricas utilizadas para avaliação foram definidas a partir dos objetivos do trabalho e das características das plataformas analisadas. A acurácia foi utilizada para medir a taxa de classificações corretas em relação ao total de amostras avaliadas. A latência ou tempo de inferência foi utilizada para representar o tempo necessário para processar uma amostra ou um conjunto de amostras. A vazão foi utilizada para indicar a quantidade de imagens processadas por segundo. O consumo energético foi avaliado apenas nas plataformas em que houve possibilidade de medição direta ou estimativa a partir de potência média e tempo de execução.

No caso da Jetson Orin Nano, foram considerados os resultados obtidos pela execução direta do modelo no ambiente embarcado. Para a MAX78000, foram considerados os resultados obtidos por validação em hardware real, utilizando o firmware executado na placa. Para o fluxo FINN voltado à PYNQ-Z2, foram considerados os relatórios de desempenho e área gerados pelo fluxo, além da validação física complementar da configuração `t2w8_500fps`.

A acurácia foi calculada pela razão entre o número de classificações corretas e o total de amostras avaliadas:

$$Acuracia = \frac{N_{acertos}}{N_{total}} \times 100 \quad (5.1)$$

A latência média por imagem foi calculada a partir do tempo total de execução dividido pelo número de amostras avaliadas:

$$t_{medio} = \frac{t_{total}}{N_{total}} \quad (5.2)$$

A vazão foi calculada pela razão entre o número total de imagens processadas e o tempo total de execução:

$$Vazao = \frac{N_{total}}{t_{total}} \quad (5.3)$$

Para as plataformas em que houve medição de potência média e tempo médio de inferência, a energia estimada por imagem foi calculada por:

$$E_{imagem} = P_{media} \times t_{medio} \quad (5.4)$$

em que E_{imagem} representa a energia estimada por imagem, P_{media} representa a potência média durante a inferência e t_{medio} representa o tempo médio de processamento por imagem.

Além dessas métricas, no fluxo FINN foram analisadas informações específicas de implementação em FPGA. Nos resultados de desempenho, foi observada a vazão estável das configurações geradas. Nos resultados de área Vivado/OOC, foram analisados LUTs, FFs, BRAMs, DSPs, WNS e frequência máxima estimada. Esses dados permitem avaliar o custo da arquitetura gerada em relação à capacidade disponível no dispositivo alvo e auxiliam na interpretação dos compromissos entre quantização, topologia da CNN e uso de recursos em FPGA.

5.2 RESULTADOS NA NVIDIA JETSON ORIN NANO

A NVIDIA Jetson Orin Nano foi avaliada como plataforma embarcada com GPU integrada. Essa abordagem representa o cenário de maior flexibilidade de software entre as três plataformas avaliadas, permitindo a execução de modelos de aprendizado profundo em ambiente Linux com suporte ao ecossistema NVIDIA.

Nesta etapa, foi utilizado o modelo final exportado no formato TensorFlow/Keras e convertido para execução como *SavedModel*. Os testes foram realizados com 10.000 amostras do conjunto de teste do dataset SAT-6, mantendo a entrada original de $28 \times 28 \times 4$ pixels. As seis classes consideradas foram: *buildings*, *barren land*, *trees*, *grassland*, *roads* e *water bodies*.

5.2.1 Desempenho de classificação

A avaliação no conjunto de teste apresentou 9934 classificações corretas em 10.000 amostras, resultando em acurácia final de 99,34%. Esse resultado indica que a rede neural convolucional apresentou alta capacidade de classificação das imagens multiespectrais do SAT-6 quando executada na plataforma Jetson.

A matriz de confusão obtida é apresentada na Tabela 2. Observa-se que a classe *water bodies* obteve acerto total no conjunto avaliado. As demais classes também apresentaram desempenho elevado, com poucos erros concentrados principalmente entre *buildings* e *roads*, e entre *barren land* e *grassland*.

Tabela 2 – Matriz de confusão obtida na NVIDIA Jetson Orin Nano.

Classe real	0	1	2	3	4	5
0	446	0	0	0	8	0
1	1	2235	1	23	0	0
2	0	1	1785	5	0	0
3	0	20	4	1511	0	0
4	3	0	0	0	248	0
5	0	0	0	0	0	3709

Fonte: Autor (2026).

A correspondência entre os índices das classes segue a organização adotada no dataset SAT-6: classe 0 corresponde a *barren land*; classe 1 a *trees*; classe 2 a *grassland*; classe 3 a *roads*; classe 4 a *buildings*; e classe 5 a *water bodies*.

A Tabela 3 apresenta as métricas por classe. Os resultados mostram valores elevados de precisão, revocação e F1-score para todas as classes, reforçando a estabilidade do modelo no conjunto de teste.

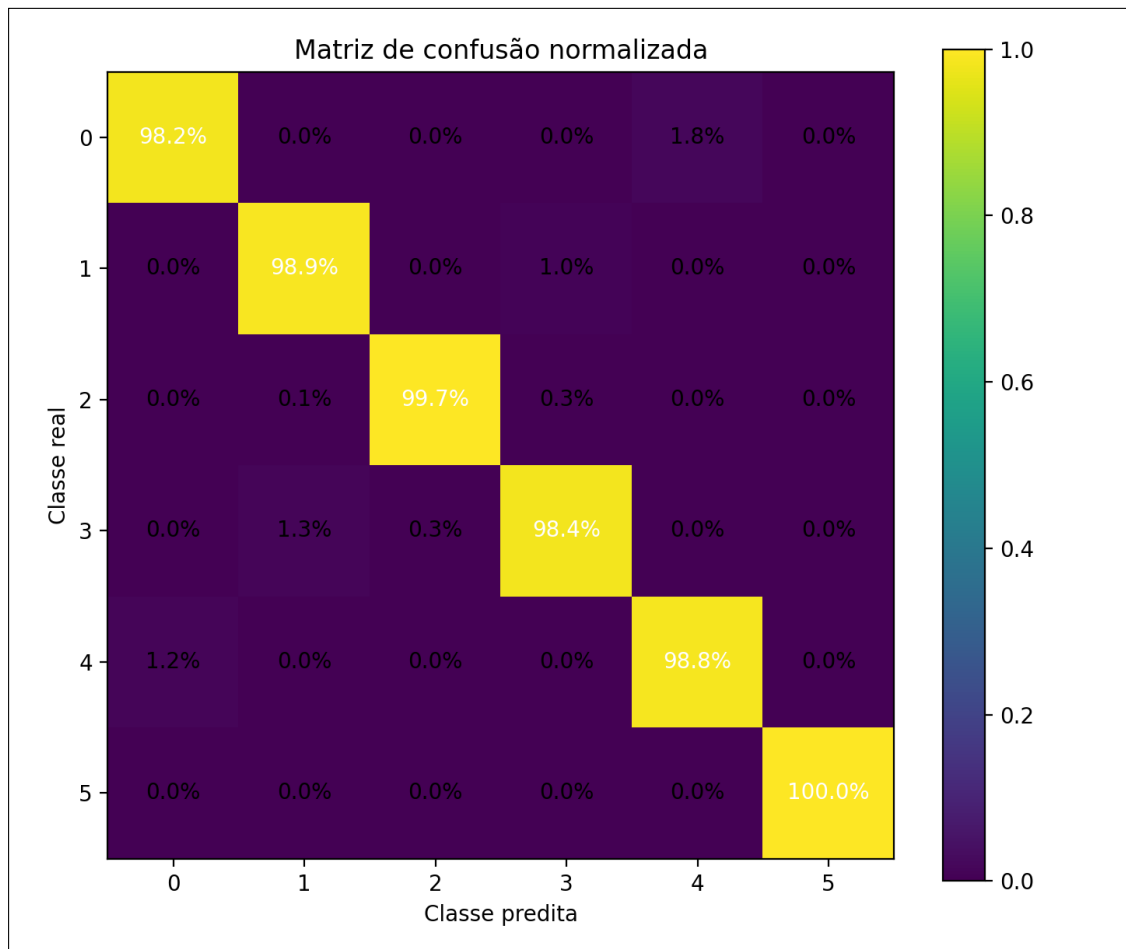
Tabela 3 – Métricas por classe obtidas na NVIDIA Jetson Orin Nano.

Classe	Descrição	Precisão	Revocação	F1-score	Amostras
0	<i>barren land</i>	99,11%	98,24%	98,67%	454
1	<i>trees</i>	99,07%	98,89%	98,98%	2260
2	<i>grassland</i>	99,72%	99,66%	99,69%	1791
3	<i>roads</i>	98,18%	98,44%	98,31%	1535
4	<i>buildings</i>	96,88%	98,80%	97,83%	251
5	<i>water bodies</i>	100,00%	100,00%	100,00%	3709

Fonte: Autor (2026).

Observa-se que a classe *water bodies* obteve acerto total no conjunto avaliado. As demais classes também apresentaram desempenho elevado, com poucos erros concentrados principalmente entre *trees* e *roads*, entre *barren land* e *buildings*, e entre *grassland* e *roads*. Apesar dessas confusões pontuais, a acurácia final de 99,34% indica alta estabilidade do modelo na plataforma Jetson Orin Nano.

Figura 23 – Matriz de confusão normalizada obtida na NVIDIA Jetson Orin Nano.



Fonte: Autor (2026).

5.2.2 Latência, vazão e consumo de energia

Além da acurácia, foram realizados testes de desempenho nos modos de potência 7W e 15W da Jetson, variando o tamanho do lote de inferência. Os testes consideraram batches de 1, 32, 256 e 1024 imagens. Em todos os casos, a acurácia permaneceu em 99,34%, pois o mesmo modelo e o mesmo conjunto de teste foram utilizados. As diferenças observadas estão relacionadas ao tempo de execução, vazão e consumo energético.

A Tabela 4 apresenta o resumo dos resultados obtidos. A configuração 15W com batch 32 apresentou o menor tempo médio por imagem, com latência de 0,665477 ms e vazão estimada de aproximadamente 1502,68 imagens por segundo. Essa mesma configuração também apresentou a menor energia aproximada por imagem, com 4,233038 mJ/imagem.

Tabela 4 – Benchmark de inferência na NVIDIA Jetson Orin Nano.

Modo	Batch	Tempo total (s)	Latência (ms/img)	Vazão (img/s)	VDD_IN médio (W)	Energia (mJ/img)
15W	1	49,622537	4,962254	201,52	5,888038	29,174185
15W	32	6,654767	0,665477	1502,68	6,346327	4,233038
15W	256	6,707321	0,670732	1490,91	6,418981	4,936461
15W	1024	9,935931	0,993593	1006,45	7,316737	8,744077
7W	1	52,447672	5,244767	190,67	5,646180	30,078457
7W	32	13,734433	1,373443	728,10	5,656020	8,230434
7W	256	14,031864	1,403186	712,66	5,771176	8,727112
7W	1024	24,187373	2,418737	413,44	6,213557	16,220130

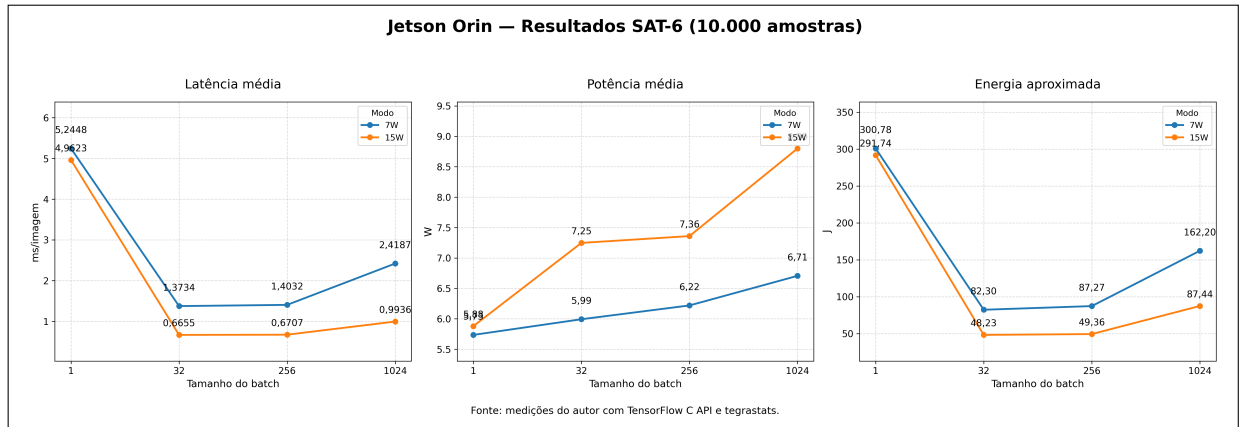
Fonte: Autor (2026).

A análise mostra que o modo 15W apresentou melhor desempenho temporal, especialmente nos batches 32 e 256. O batch 1 apresentou maior latência por imagem devido ao menor aproveitamento do paralelismo da GPU. Já o batch 1024 apresentou aumento de tempo e energia por imagem, indicando que lotes muito grandes não necessariamente resultam no melhor desempenho para esse modelo e configuração.

No modo 7W, a plataforma apresentou menor orçamento energético, porém com aumento da latência e redução da vazão. Ainda assim, o desempenho permaneceu funcional, com acurácia idêntica à do modo 15W. Esse resultado indica que a escolha entre 7W e 15W depende do objetivo da aplicação: o modo 15W favorece maior desempenho, enquanto o modo 7W pode ser considerado em cenários com maior restrição energética.

A Figura 24 apresenta uma visualização consolidada dos resultados de benchmark, comparando os modos de potência e os diferentes tamanhos de batch avaliados.

Figura 24 – Benchmark da NVIDIA Jetson Orin Nano.



Fonte: Autor (2026).

5.2.3 Síntese dos resultados da Jetson

A Tabela 5 apresenta uma síntese dos principais resultados obtidos na NVIDIA Jetson Orin Nano, considerando a classificação das imagens do SAT-6 e o melhor cenário de desempenho observado nos testes de benchmark.

Tabela 5 – Resumo dos resultados obtidos na NVIDIA Jetson Orin Nano.

Métrica	Resultado
Plataforma avaliada	NVIDIA Jetson Orin Nano
Tipo de execução	Inferência em GPU embarcada
Modelo executado	CNN TensorFlow/Keras
Dataset	SAT-6
Amostras avaliadas	10.000
Acertos	9.934
Acurácia final	99,34%
Melhor configuração	Modo 15W, batch 32
Latência média	0,665477 ms/imagem
Vazão	1502,68 imagens/s
Potência média medida em VDD_IN	6,346327 W
Energia estimada por imagem	4,223 mJ/imagem
Eficiência energética	236,78 imagens/J

Fonte: Autor (2026).

Os resultados obtidos demonstram que a Jetson Orin Nano apresentou elevada capacidade de classificação das imagens multiespectrais do SAT-6, alcançando 99,34% de acurácia no conjunto de teste. A matriz de confusão e as métricas por classe indicam comportamento estável do modelo, com destaque para a classe *water bodies*, que obteve 100,00% de precisão, revocação e F1-score.

Em relação ao desempenho computacional, a melhor configuração observada foi o modo 15W com batch 32, que apresentou latência média de 0,665477 ms por imagem e vazão de 1502,68 imagens por segundo. A partir da potência média medida em VDD_IN, de 6,346327 W, estimou-se energia de 4,223 mJ por imagem e eficiência energética de 236,78 imagens/J. Esse resultado mostra que o uso de lotes intermediários permitiu melhor aproveitamento da GPU embarcada, reduzindo a latência por imagem em comparação com a execução unitária.

5.3 RESULTADOS NA MAX78000

A MAX78000 foi avaliada como representante de uma abordagem TinyML baseada em microcontrolador com acelerador CNN integrado. Diferentemente da Jetson Orin Nano, que executa o modelo em um ambiente Linux com bibliotecas de alto nível, a MAX78000 realiza a inferência diretamente em firmware embarcado, utilizando o acelerador neural dedicado da placa. A validação foi realizada por meio de um teste *hardware-in-the-loop* (HIL). Nesse fluxo, um computador hospedeiro executa um script em Python responsável por carregar amostras do conjunto de teste do SAT-6, preparar os dados no formato esperado pela placa, enviá-los por comunicação serial e receber a classe prevista pelo firmware. Em seguida, a predição retornada pela MAX78000 é comparada com o rótulo real da amostra.

Foram avaliadas 1000 imagens reais do conjunto de teste. Todas as respostas retornadas pela placa foram consideradas válidas, não havendo respostas inválidas durante a validação. O resultado obtido foi de 947 acertos em 1000 imagens, correspondendo a uma acurácia total de 94,70%. Esse resultado demonstra que a MAX78000 foi capaz de executar a CNN quantizada em hardware real com desempenho de classificação elevado, mesmo operando em uma plataforma de recursos computacionais restritos e voltada ao ultra baixo consumo.

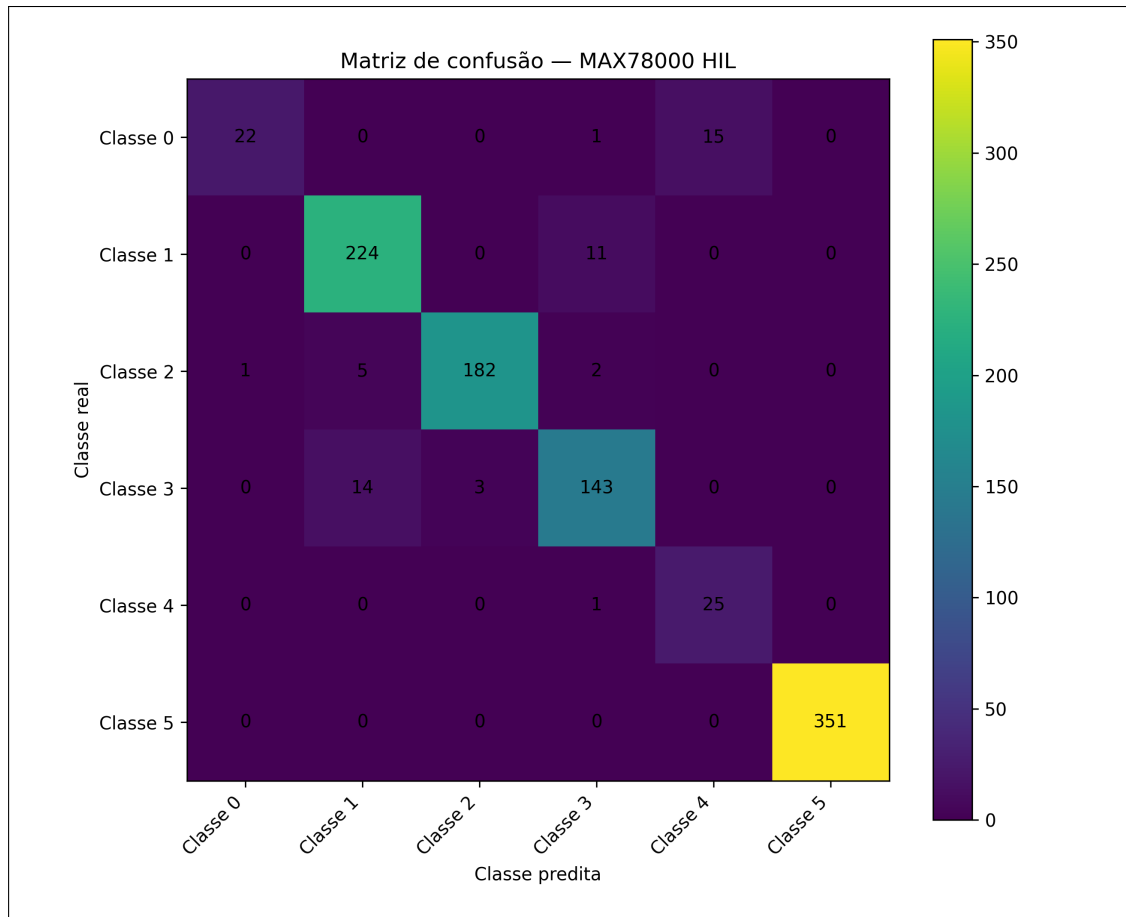
Quadro 12 – Resumo da validação HIL na MAX78000

Métrica	Resultado
Total de imagens avaliadas	1000
Respostas válidas	1000
Respostas inválidas	0
Acertos	947/1000
Acurácia total	94,70%
Acurácia nas respostas válidas	94,70%
Tipo de validação	Execução real em hardware por teste <i>hardware-in-the-loop</i> .

Fonte: Autor (2026).

A Figura 25 apresenta a matriz de confusão obtida na validação HIL. A diagonal principal indica as classificações corretas, enquanto os valores fora da diagonal indicam os erros de classificação entre as classes.

Figura 25 – Matriz de confusão obtida na validação HIL da MAX78000.



Fonte: Autor (2026).

A correspondência entre os índices das classes segue a organização adotada no dataset: classe 0 corresponde a *barren land*; classe 1 a *trees*; classe 2 a *grassland*; classe 3 a *roads*; classe 4 a *buildings*; e classe 5 a *water bodies*.

A Tabela 6 apresenta as métricas por classe. A acurácia por classe representa a proporção de amostras corretamente classificadas dentro de cada classe real, enquanto a precisão indica a proporção de predições corretas entre todas as amostras classificadas como aquela classe.

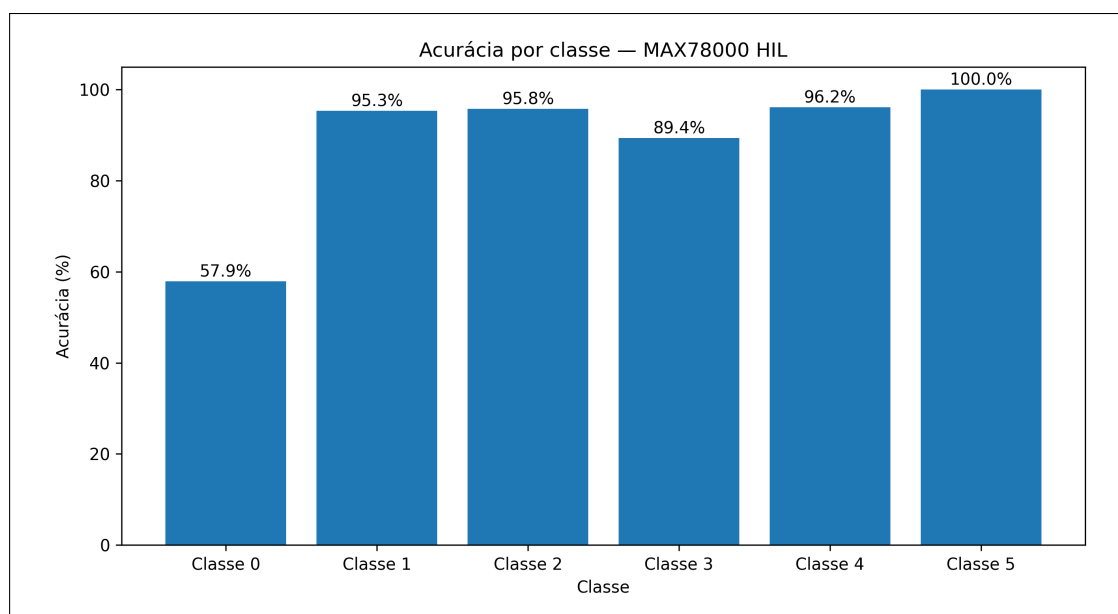
Tabela 6 – Métricas por classe obtidas na validação da MAX78000.

Classe	Descrição	Total	Acertos	Erros	Acurácia	Precisão
0	<i>barren land</i>	38	22	16	57,89%	95,65%
1	<i>trees</i>	235	224	11	95,32%	92,18%
2	<i>grassland</i>	190	182	8	95,79%	98,38%
3	<i>roads</i>	160	143	17	89,38%	90,51%
4	<i>buildings</i>	26	25	1	96,15%	62,50%
5	<i>water bodies</i>	351	351	0	100,00%	100,00%

Fonte: Autor (2026).

A Figura 26 apresenta a acurácia por classe de forma gráfica. Observa-se que a classe 5, correspondente a *water bodies*, obteve acerto total no conjunto avaliado, enquanto a classe 0, correspondente a *barren land*, apresentou o menor desempenho relativo. Essa classe teve 22 acertos em 38 amostras, sendo confundida principalmente com a classe 4, correspondente a *buildings*. Esse comportamento pode estar relacionado à semelhança visual entre terrenos baldios, solo exposto e superfícies construídas em imagens aéreas de baixa resolução.

Figura 26 – Acurácia por classe obtida na validação da MAX78000.



Fonte: Autor (2026).

Embora a classe 4, correspondente a *buildings*, tenha apresentado acurácia por classe elevada, sua precisão foi menor, atingindo 62,50%. Isso ocorre porque parte das amostras reais da classe 0, correspondente a *barren land*, foi classificada como classe 4, aumentando o número de falsos positivos nessa categoria. Portanto, a análise conjunta da acurácia por classe e da precisão é importante para compreender melhor o comportamento do modelo.

5.3.1 Latência, vazão e consumo de energia

Além da acurácia, foram avaliadas a latência, a vazão e a eficiência energética da MAX78000 em dois níveis distintos. O primeiro considera apenas o processamento embarcado, medida por meio do timer embarcado associado à execução do acelerador neural. O segundo considera o ciclo completo do teste *hardware-in-the-loop* (HIL), incluindo o envio da imagem por UART, a sincronização entre computador e placa, a execução da inferência, o retorno da resposta e a leitura serial pelo script Python.

Tabela 7 – Medições de latência, vazão e eficiência energética obtidas na MAX78000.

Medição	Latência média	Vazão	Potência	Energia/img	Eficiência
Processamento embarcado	0,611 ms/imagem	1636,650 img/s	0,060 W	0,03666 mJ/img	27,277 img/J
Sistema HIL completo	342,808 ms/imagem	2,917 img/s	0,060 W	20,568 mJ/img	48,62 img/J

Fonte: Autor (2026).

A diferença entre as medições evidencia que o tempo de processamento embarcado da MAX78000 é significativamente menor que o tempo total observado no ensaio HIL. A inferência pura da CNN considera apenas a execução interna da rede no acelerador neural, enquanto o processamento embarcado sem UART inclui também a cópia dos dados para a SRAM da CNN, o descarregamento da saída, a conversão dos valores retornados e a seleção da classe final por meio do *argmax*.

Por outro lado, quando a comunicação serial é considerada, o tempo aumenta de forma expressiva. O firmware com recepção UART inclui o envio da imagem para a placa, enquanto o sistema HIL completo inclui também etapas executadas no computador, como preparação dos dados, envio do comando de início, espera por confirmação, leitura da resposta e controle do experimento pelo script Python. Dessa forma, a métrica de processamento embarcado sem UART representa de maneira mais adequada a capacidade computacional da MAX78000, enquanto a latência do sistema HIL representa o tempo total do procedimento experimental utilizado para validação.

Para a análise energética, a MAX78000 foi alimentada com tensão de 5 V e apresentou corrente aproximada de 0,012 A durante o modo de inferência contínua. A potência consumida pela placa foi estimada pela relação:

$$P = V \times I \quad (5.5)$$

Substituindo os valores medidos:

$$P = 5 \times 0,012 = 0,060 \text{ W} \quad (5.6)$$

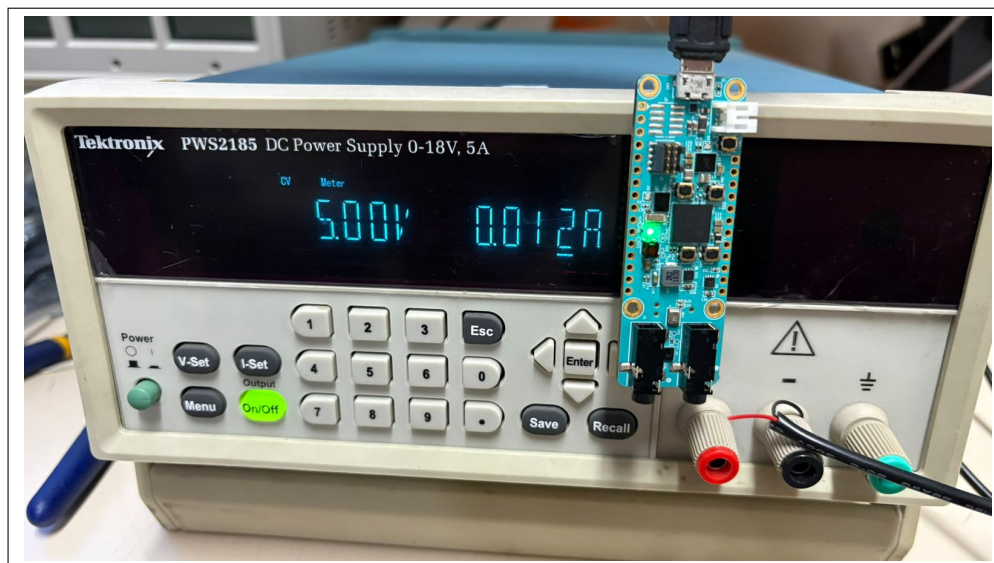
Assim, a potência estimada durante o experimento foi de aproximadamente 0,060 W, ou 60 mW. A energia por imagem foi calculada pela relação entre a potência estimada e a latência média de cada medição:

$$E_{\text{imagem}} = P \times t_{\text{imagem}} \quad (5.7)$$

A eficiência energética foi expressa em imagens por joule, indicando a quantidade de imagens processadas por unidade de energia consumida. Considerando o processamento embarcado sem UART, a MAX78000 apresentou energia estimada de 0,03666 mJ por imagem e eficiência de aproximadamente 27.277,50 imagens/J. Já no sistema HIL completo, a energia estimada foi de 20,568 mJ por imagem, com eficiência de 48,62 imagens/J. Essa diferença ocorre porque o HIL inclui o tempo de comunicação e controle externo do experimento, enquanto a medição do processamento embarcado sem UART considera as etapas executadas internamente na placa após a imagem já estar disponível em memória, incluindo a cópia dos dados para a SRAM da CNN, a inferência, o descarregamento da saída, a conversão dos valores retornados e a seleção da classe final por meio do *argmax*. Ressalta-se que os valores energéticos consideram a potência medida na placa MAX78000. No caso do sistema HIL, o consumo do computador hospedeiro não foi incluído no cálculo, sendo considerada apenas a energia associada ao tempo de operação da placa durante o procedimento experimental.

A Figura 27 apresenta a montagem experimental utilizada para a medição de consumo da MAX78000.

Figura 27 – Montagem experimental para medição de consumo da MAX78000.



Fonte: Autor (2026).

Os resultados mostram que a MAX78000 apresentou elevada eficiência energética, com acurácia de 94,70% em execução, latência média de 0,611 ms por imagem no processamento embarcado sem UART e potência estimada em aproximadamente 60 mW no modo de inferência contínua. Quando considerado o sistema HIL completo, a latência média foi de 342,808 ms por imagem, valor significativamente maior devido à comunicação serial e ao controle externo pelo computador hospedeiro. Assim, a MAX78000 demonstrou bom desempenho para aplicações TinyML de baixa potência, sendo importante diferenciar o tempo de processamento embarcado na placa da latência total do sistema experimental.

5.4 RESULTADOS NA PYNQ-Z2 COM FINN

A abordagem associada à PYNQ-Z2 foi avaliada a partir do fluxo FINN, utilizando CNNs quantizadas aplicadas ao dataset SAT-6. Diferentemente da Jetson Orin Nano e da MAX78000, que foram avaliadas principalmente por execução direta em hardware embarcado, esta etapa considerou inicialmente a geração e a análise de aceleradores em FPGA por meio do FINN.

Foram analisadas seis configurações geradas pelo fluxo `sat6_cnn`: `t1w2_500fps`, `t1w4_500fps`, `t1w8_500fps`, `t2w2_500fps`, `t2w4_500fps` e `t2w8_500fps`. A análise incluiu os resultados de desempenho do FINN e os resultados de área obtidos pelo fluxo Vivado/OOC. Além disso, como validação complementar, foi realizada a execução física na PYNQ-Z2 apenas da configuração `t2w8_500fps`, por ser a configuração de maior precisão numérica entre as versões avaliadas da topologia `t2`.

5.4.1 Resultados de desempenho do fluxo FINN

A Tabela 8 apresenta a vazão estável obtida no fluxo FINN para as configurações avaliadas. Observa-se que as configurações da topologia t_2 apresentaram maior vazão estimada em relação às configurações da topologia t_1 , mantendo valores próximos entre as diferentes larguras de palavra.

Tabela 8 – Resultados de desempenho obtidos no fluxo FINN.

Configuração	Vazão estável (imagens/s)
t_{1w2_500fps}	543,05
t_{1w4_500fps}	543,05
t_{1w8_500fps}	543,04
t_{2w2_500fps}	678,81
t_{2w4_500fps}	678,81
t_{2w8_500fps}	678,79

Fonte: Autor (2026).

Esses valores representam os resultados obtidos no fluxo de análise de desempenho do FINN. Portanto, eles devem ser interpretados como métricas do fluxo de geração e não como medição elétrica ou medição externa da placa. A execução física complementar apresentada posteriormente utiliza uma medição end-to-end por script Python na PYNQ-Z2, incluindo envio da entrada ao acelerador, execução e leitura da saída.

5.4.2 Resultados de área Vivado/OOC

A Tabela 9 apresenta os resultados de área obtidos pelo fluxo Vivado/OOC para as configurações avaliadas. Foram observados recursos como LUTs, FFs, BRAMs e DSPs, além do WNS e da frequência máxima estimada. Esses dados permitem avaliar o custo de cada arquitetura gerada em relação à capacidade do FPGA da PYNQ-Z2.

Tabela 9 – Resultados de área Vivado/OOC para as configurações geradas pelo FINN.

Configuração	LUTs	FFs	BRAM	DSP	WNS	Fmax (MHz)
t_{1w2_500fps}	4816 (9,05%)	5927 (5,57%)	7,5 (5,36%)	0 (0,00%)	0,162	101,65
t_{1w4_500fps}	6124 (11,51%)	10152 (9,54%)	7,5 (5,36%)	0 (0,00%)	0,475	104,99
t_{1w8_500fps}	13808 (25,95%)	19374 (18,21%)	10,0 (7,14%)	0 (0,00%)	0,078	100,79
t_{2w2_500fps}	6992 (13,14%)	4739 (4,45%)	9,5 (6,79%)	0 (0,00%)	0,149	101,51
t_{2w4_500fps}	7531 (14,16%)	5771 (5,42%)	9,5 (6,79%)	0 (0,00%)	0,295	103,04
t_{2w8_500fps}	19050 (35,81%)	9343 (8,78%)	10,0 (7,14%)	0 (0,00%)	0,121	101,22

Fonte: Autor (2026).

Os resultados mostram que o aumento da largura de palavra tende a elevar o uso de LUTs e FFs, principalmente nas configurações de 8 bits. A configuração t_{2w8_500fps} ,

utilizada posteriormente na validação física, apresentou o maior uso de LUTs entre as configurações avaliadas, com 19050 LUTs, correspondendo a 35,81% do dispositivo. Ainda assim, o resultado permaneceu dentro da capacidade disponível da PYNQ-Z2, mantendo WNS positivo e frequência máxima estimada próxima de 100 MHz.

5.4.3 Validação física complementar com t_{2w8_500fps}

Após a geração dos artefatos pelo FINN, foi realizada uma validação física complementar na PYNQ-Z2 utilizando a configuração t_{2w8_500fps} . Essa validação teve como objetivo verificar a execução real do acelerador gerado pelo FINN na placa, utilizando 1000 imagens reais do conjunto de teste do SAT-6.

A Tabela 10 apresenta os resultados obtidos nessa execução. A medição foi realizada de forma end-to-end por script Python na PYNQ-Z2, considerando o envio da entrada ao acelerador, a execução da inferência e a leitura da saída.

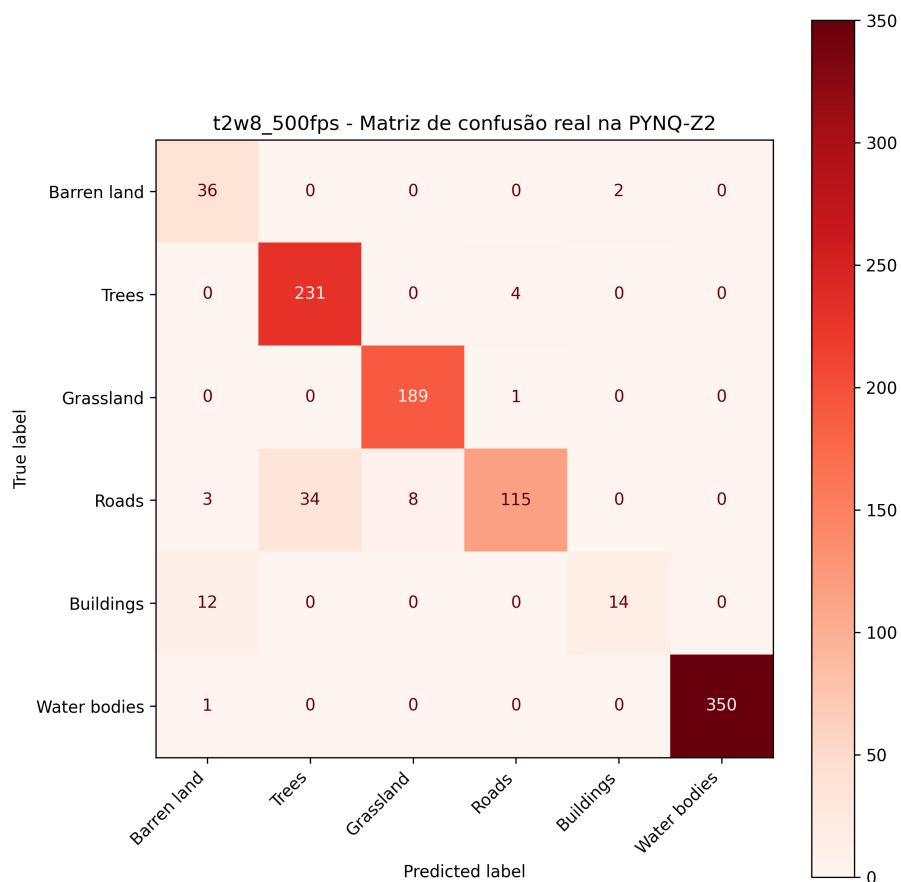
Tabela 10 – Resultado da execução física complementar na PYNQ-Z2

Métrica	Resultado
Configuração avaliada	t_{2w8_500fps}
Modelo	CNN quantizada
Entrada	$32 \times 32 \times 4$
Amostras avaliadas	1000
Acertos	935
Erros	65
Acurácia real na placa	93,50%
Tempo total	2,258213 s
Latência média	2,258213 ms/imagem
Vazão real	442,827969 imagens/s

Fonte: Autor (2026).

A matriz de confusão obtida na execução física é apresentada na Figura 28. O resultado mostra que o acelerador executou a CNN quantizada em hardware e produziu previsões para todas as seis classes do SAT-6.

Figura 28 – Matriz de confusão obtida na execução física da PYNQ-Z2 com t2w8_500fps.



Fonte: Autor (2026).

A Tabela 11 apresenta a acurácia por classe obtida na execução real. As classes *trees*, *grassland* e *water bodies* apresentaram desempenho elevado. As maiores dificuldades ocorreram nas classes *roads* e *buildings*, que concentraram a maior parte dos erros do modelo no recorte avaliado.

Tabela 11 – Acurácia por classe obtida na execução física da PYNQ-Z2

Classe	Descrição	Amostras	Acertos	Erros	Acurácia
0	<i>barren land</i>	38	36	2	94,74%
1	<i>trees</i>	235	231	4	98,30%
2	<i>grassland</i>	190	189	1	99,47%
3	<i>roads</i>	160	115	45	71,88%
4	<i>buildings</i>	26	14	12	53,85%
5	<i>water bodies</i>	351	350	1	99,72%

Fonte: Autor (2026).

A execução física complementar demonstrou que o acelerador gerado pelo FINN foi

capaz de realizar inferência real na PYNQ-Z2 com imagens SAT-6. A acurácia de 93,50% ficou próxima dos valores obtidos nas demais plataformas embarcadas, enquanto a vazão real medida por execução end-to-end foi de aproximadamente 442,83 imagens/s. A diferença entre a vazão estável apresentada pelo fluxo FINN e a vazão real medida na placa decorre do método de medição: a primeira é obtida pelos relatórios de desempenho do fluxo, enquanto a segunda inclui o caminho completo de execução via script na PYNQ-Z2.

5.5 COMPARAÇÃO ENTRE AS PLATAFORMAS

A Tabela 12 resume os principais resultados obtidos nas três abordagens avaliadas. Para interpretação adequada, deve-se considerar que as plataformas utilizam fluxos diferentes de implantação, modelos distintos, precisões numéricas diferentes e métodos de medição não totalmente equivalentes. Na Jetson Orin Nano, os valores apresentados correspondem à melhor configuração observada no benchmark, utilizando modo de potência de 15 W e batch 32. Na MAX78000, os resultados correspondem à validação em hardware real, com a latência média do sistema HIL e potência estimada a partir de medição elétrica em modo de inferência contínua. Na PYNQ-Z2, os valores de acurácia, latência e vazão correspondem à execução física complementar da CNN quantizada t_{2w8_500fps} , gerada pelo fluxo FINN.

Tabela 12 – Comparação geral dos resultados obtidos nas plataformas avaliadas.

Métrica	Jetson Orin Nano	MAX78000	PYNQ-Z2 / FINN
Modelo	CNN	CNN quantizada	CNN quantizada
Entrada	$28 \times 28 \times 4$	$28 \times 28 \times 4$	$32 \times 32 \times 4$
Execução	GPU embarcada	Acelerador CNN integrado	Acelerador FINN em FPGA
Configuração considerada	15 W, batch 32	Validação HIL / Processamento embarcado	t_{2w8_500fps}
Acurácia	99,34%	94,70%	93,50%
Amostras avaliadas	10000	1000	1000
Latência média	0,665477 ms/imagem	342,808 ms/imagem / 0,611 ms/imagem	2,258213 ms/imagem
Vazão	1502,68 imagens/s	2,917 imagens/s / 1636,650 imagens/s	442,827969 imagens/s
Potência média	6,346327 W	0,060 W	1,58 W
Energia por imagem	4,223 mJ/imagem	20,568 mJ/imagem / 0,03666 mJ/imagem	3,57 mJ/imagem
Eficiência energética	236,78 imagens/J	48,62 imagens/J / 27.277,50 imagens/J	280,27 imagens/J
Execução em hardware	Sim	Sim	Sim

Fonte: Autor (2026).

A energia por imagem foi estimada pela relação entre a potência média medida e a latência média por imagem. Para a Jetson Orin Nano, utilizou-se a potência média indicada pelo campo VDD_IN na melhor configuração de desempenho. Para a MAX78000, foi considerada a potência estimada a partir da medição externa de 5 V e 0,012 A, associada à latência média do sistema HIL completo. A eficiência energética foi calculada em imagens por joule, permitindo normalizar a comparação entre plataformas com características distintas. Para a PYNQ-Z2, essa métrica foi calculada considerando medição externa de potência durante a execução física.

A Jetson Orin Nano apresentou a maior acurácia e a menor latência entre as plataformas

avaliadas. Esse resultado é coerente com sua arquitetura, que combina CPU e GPU embarcada, oferecendo maior capacidade computacional e maior flexibilidade de software. Entretanto, essa vantagem ocorre com consumo de potência significativamente maior em comparação com plataformas voltadas a aplicações de baixo consumo.

A MAX78000 apresentou a melhor relação entre acurácia e consumo energético. Mesmo com recursos computacionais mais limitados, a plataforma atingiu 94,70% de acurácia em execução real e potência aproximada de 60 mW durante o modo de inferência contínua. A latência do sistema HIL, entretanto, inclui custos de comunicação e controle externo, não representando apenas o tempo interno de processamento da CNN no acelerador.

A PYNQ-Z2 demonstrou a viabilidade do fluxo FINN para geração e execução de um acelerador CNN em FPGA. Na execução física complementar com a configuração $t_{2w8} - 500\text{fps}$, a plataforma obteve 93,50% de acurácia, latência média de 2,258213 ms por imagem e vazão real de aproximadamente 442,83 imagens/s. Além disso, os relatórios do fluxo FINN e do Vivado/OOC permitiram analisar o uso de recursos da FPGA, mostrando o impacto das diferentes topologias e quantizações sobre LUTs, FFs, BRAMs, DSPs e frequência máxima estimada.

De forma geral, os resultados indicam que cada plataforma apresenta vantagens distintas. A Jetson Orin Nano oferece maior desempenho bruto e flexibilidade de software; a MAX78000 se destaca pelo baixo consumo energético e pela adequação a aplicações TinyML; e a PYNQ-Z2, por meio do FINN, apresenta uma alternativa baseada em hardware reconfigurável, permitindo gerar aceleradores especializados e analisar explicitamente o custo de hardware das arquiteturas produzidas.

5.6 DISCUSSÃO GERAL DOS RESULTADOS

Os resultados obtidos confirmam a viabilidade de executar inferência embarcada para classificação de imagens multiespectrais do SAT-6 nas três abordagens avaliadas. Entretanto, cada plataforma apresentou características próprias, refletindo compromissos distintos entre desempenho, consumo energético, complexidade de implantação, flexibilidade de desenvolvimento e nível de especialização do hardware.

A Jetson Orin Nano se destacou pelo desempenho computacional. A acurácia de 99,34%, associada à latência média de 0,665477 ms por imagem na melhor configuração de benchmark, indica que a plataforma é adequada para aplicações que exigem resposta rápida e elevada capacidade de processamento. A utilização de uma GPU embarcada e de bibliotecas de alto nível torna o fluxo mais flexível e próximo de ambientes convencionais de desenvolvimento em aprendizado profundo. Por outro lado, essa flexibilidade ocorre com maior consumo de

potência em comparação com plataformas voltadas a aplicações TinyML.

A MAX78000 se destacou pela eficiência energética. A acurácia de 94,70% em execução real mostra que, mesmo em um microcontrolador com recursos limitados, o acelerador CNN integrado foi capaz de executar a tarefa de classificação com desempenho competitivo. A potência aproximada de 60 mW em modo de inferência contínua reforça sua adequação a aplicações de ultra baixo consumo. A principal limitação está na maior rigidez do fluxo de implantação, nas restrições impostas pela arquitetura do acelerador e na maior dificuldade de adaptação do modelo em comparação com plataformas de software mais flexíveis.

A PYNQ-Z2 demonstrou a viabilidade do fluxo FINN para geração e execução de aceleradores CNN em FPGA. Na análise do fluxo, foram avaliadas diferentes configurações de topologia e quantização, permitindo observar o impacto sobre vazão estimada e uso de recursos de hardware. Na validação física complementar, a configuração $t2w8_500fps$ obteve acurácia de 93,50%, latência média de 2,258213 ms por imagem e vazão real de aproximadamente 442,83 imagens/s. Esses resultados mostram que o acelerador gerado pelo FINN foi funcional na PYNQ-Z2 e manteve desempenho próximo ao observado nas demais plataformas embarcadas.

De forma geral, não houve uma plataforma superior em todos os aspectos. A Jetson Orin Nano apresentou o melhor desempenho bruto e a menor latência. A MAX78000 apresentou a melhor relação entre acurácia e baixo consumo energético. A PYNQ-Z2, por sua vez, destacou-se pela possibilidade de geração e análise de aceleradores customizados em FPGA, oferecendo maior flexibilidade arquitetural, embora com maior complexidade de desenvolvimento e validação.

5.7 LIMITAÇÕES DOS RESULTADOS

Apesar dos resultados obtidos, algumas limitações devem ser consideradas. A primeira está relacionada ao fato de que as três plataformas não executaram exatamente a mesma arquitetura neural. A Jetson Orin Nano utilizou uma CNN executada em ambiente embarcado com GPU, a MAX78000 utilizou uma CNN quantizada adaptada ao acelerador integrado do microcontrolador, e a PYNQ-Z2 utilizou uma CNN quantizada gerada pelo fluxo FINN, com validação física complementar da configuração $t2w8_500fps$. Dessa forma, a comparação deve ser interpretada como uma análise entre abordagens de implantação embarcada para o mesmo problema de classificação, e não como uma comparação estritamente isolada de hardwares executando a mesma rede.

A segunda limitação está relacionada aos métodos de medição. Na Jetson Orin Nano, os resultados de latência, vazão e potência foram obtidos por benchmark no ambiente embarcado.

Na MAX78000, a acurácia foi obtida em hardware real, enquanto a latência média usada na comparação corresponde ao sistema HIL completo. Na PYNQ-Z2, a validação física complementar foi medida por execução end-to-end e o consumo energético externo não foi consolidado. Assim, as métricas de latência, vazão e energia não representam exatamente o mesmo nível de medição em todas as plataformas. Outra limitação está relacionada ao fluxo FINN. Embora tenham sido analisados relatórios de desempenho e área para diferentes configurações, a execução física na PYNQ-Z2 foi realizada apenas com a configuração t_{2w8_500fps} . Essa escolha foi adotada por se tratar de uma validação complementar e por representar a versão de maior precisão numérica entre as configurações avaliadas da topologia t_2 . Futuras investigações podem ampliar a execução física para as demais configurações e comparar diretamente o comportamento real de cada acelerador gerado.

Por fim, a MAX78000 teve potência estimada em modo de inferência contínua, mas a energia por imagem não foi consolidada de forma diretamente comparável às demais plataformas. Isso ocorre porque a medição de potência representa o comportamento da placa durante o modo contínuo, enquanto a latência do sistema HIL inclui comunicação e controle externo. Uma medição mais precisa da eficiência energética exigiria instrumentação específica para isolar o tempo e o consumo associados apenas à inferência no acelerador.

5.8 SÍNTESE DOS RESULTADOS

De forma geral, os experimentos demonstraram que a inferência embarcada aplicada ao dataset SAT-6 é viável nas três plataformas avaliadas. A Jetson Orin Nano apresentou o melhor desempenho geral em acurácia, latência e vazão, sendo a alternativa mais adequada para cenários em que desempenho computacional é o principal critério. A MAX78000 apresentou o melhor comportamento energético, com acurácia de 94,70% e potência aproximada de 60 mW durante inferência contínua. Esse resultado indica que microcontroladores com aceleradores neurais dedicados são alternativas relevantes para aplicações TinyML, especialmente quando o consumo energético é uma restrição central. A PYNQ-Z2 permitiu validar o uso do FINN para geração e análise de aceleradores CNN em FPGA. Os relatórios de desempenho e área possibilitaram observar o impacto das diferentes topologias e quantizações sobre a arquitetura gerada, enquanto a execução física complementar da configuração t_{2w8_500fps} demonstrou funcionamento real do acelerador na placa, com acurácia de 93,50% e vazão real de aproximadamente 442,83 imagens/s.

Esses resultados sustentam a proposta do trabalho ao evidenciar, de forma experimental, os compromissos envolvidos na escolha de plataformas para Edge AI e TinyML em aplicações de sensoriamento remoto embarcado. Plataformas com GPU embarcada favorecem desempenho e flexibilidade de software; microcontroladores com aceleradores neurais favorecem baixo consumo; e FPGAs oferecem flexibilidade arquitetural para aceleradores customizados, embora

exijam maior esforço de geração, integração e validação.

6 CONSIDERAÇÕES FINAIS

Este trabalho avaliou a aplicação de TinyML à classificação de imagens multiespectrais do dataset SAT-6 em três plataformas embarcadas: NVIDIA Jetson Orin Nano, MAX78000 e PYNQ-Z2 com FINN. O objetivo foi analisar os compromissos entre acurácia, latência, vazão, consumo energético, representação numérica e complexidade de implementação em arquiteturas distintas de hardware.

Os resultados demonstraram que a Jetson Orin Nano apresentou o melhor desempenho geral em acurácia e vazão, sendo mais adequada para aplicações que priorizam desempenho e flexibilidade de desenvolvimento. A MAX78000 destacou-se pelo baixo consumo de potência e pela execução da inferência em firmware C com acelerador CNN dedicado, mostrando-se apropriada para aplicações TinyML com restrições severas de energia. A PYNQ-Z2, por sua vez, evidenciou o potencial do fluxo FINN para geração de aceleradores customizados em FPGA, embora com maior complexidade de implementação.

A principal contribuição do trabalho foi organizar e comparar três fluxos distintos de implementação embarcada para uma mesma tarefa de sensoriamento remoto, incluindo preparação dos dados, normalização, quantização, execução em hardware real e análise das métricas obtidas. Além dos resultados quantitativos, o estudo evidenciou diferenças práticas entre os ecossistemas avaliados, especialmente quanto ao uso de Python, firmware em C, ferramentas de síntese e geração de aceleradores.

Como limitações, destaca-se que as plataformas não foram avaliadas sob protocolos totalmente equivalentes, devido às diferenças de hardware, ferramentas e formas de medição disponíveis. Em especial, a validação HIL da MAX78000 inclui custos de comunicação e controle externo, enquanto a PYNQ-Z2 depende do fluxo de síntese e geração do acelerador pelo FINN. Dessa forma, os resultados devem ser interpretados considerando o contexto experimental e o nível de medição adotado em cada plataforma.

6.1 TRABALHOS FUTUROS

Como continuidade deste trabalho, recomenda-se ampliar os experimentos com outros conjuntos de dados de sensoriamento remoto, incluindo bases com maior número de classes, diferentes resoluções espaciais e imagens obtidas em condições ambientais variadas. Essa ampliação permitiria avaliar a capacidade de generalização dos modelos TinyML em cenários mais próximos de aplicações reais.

Também se recomenda investigar diferentes arquiteturas de redes neurais compactas

e novas estratégias de otimização, como poda de pesos, treinamento ciente de quantização e diferentes níveis de quantização. Esses estudos poderiam aprofundar a análise do impacto da representação numérica na acurácia, na latência, no consumo energético e na compatibilidade com cada plataforma embarcada.

Por fim, trabalhos futuros podem adotar protocolos padronizados de medição de potência, energia por inferência, latência e vazão, permitindo uma comparação estatística mais rigorosa entre microcontroladores com acelerador neural, GPUs embarcadas e SoC-FPGAs. Além disso, a disponibilização dos códigos, modelos, scripts e resultados em repositório público ou institucional contribui para a reprodutibilidade dos experimentos e para a continuidade de pesquisas em TinyML aplicado ao sensoriamento remoto embarcado.

REFERÊNCIAS

ALSAADE, F. et al. Edge ai: A taxonomy, systematic review and future directions. **arXiv preprint arXiv:2407.04053**, 2024.

Alteryx. **Aprendizado supervisionado vs. não supervisionado: qual é a diferença?** 2024. <<https://www.alteryx.com/pt-br/glossary/supervised-vs-unsupervised-learning>>. Acessado em 24 de junho de 2025.

AMD Research. **Brevitas Export**. 2024. Disponível em: <https://finn.readthedocs.io/en/latest/brevitas_export.html>.

AMD Research. **FINN Documentation**. 2024. Disponível em: <<https://finn.readthedocs.io/>>. Acesso em: 01 de outubro de 2025.

AMD/Xilinx. **Zynq-7000 SoC Documentation**. 2024. Disponível em: <<https://www.xilinx.com/products/silicon-devices/soc/zynq-7000.html>>. Acesso em: 03 de julho de 2025.

Analog Devices. **Keywords Spotting Using the MAX78000**. 2020.

Analog Devices. Developing power-optimized applications on the max78000. **Application Note**, 2021.

Analog Devices. **MAX78000 Artificial Intelligence Microcontroller Datasheet**. 2024. Disponível em: <<https://www.analog.com/en/products/max78000.html>>. Acesso em: 07 de agosto de 2025.

ATLANTICO. **TinyML e Edge AI: inteligência artificial descentralizada na indústria**. 2024. <<https://www.atlantico.com.br/blog/industria/tinyml-e-edge-ai-inteligencia-artificial-descentralizada-na-industria/>>. Acessado em 17 de setembro de 2025.

AVELAR, E. A. Ajudando a “desmitificar” a inteligência artificial em educação: a abordagem do aprendizado de máquina. **Educação em Revista**, v. 41, p. e58030, 2025.

BASU, S.; GANGULY, S.; MUKHOPADHYAY, S.; DIBIANO, R.; KARKI, M.; NEMANI, R. Deepsat: A learning framework for satellite imagery. In: **ACM SIGSPATIAL**. [S.l.: s.n.], 2015.

BERTOLETI, P. **O que são Sistemas Embarcados? Uma visão detalhada**. 2025. <<https://pedrobertoleti.com.br/index.php/2025/05/02/o-que-sao-sistemas-embarcados-uma-visao-detalhada/>>. Acessado em 09 de setembro de 2025.

Birchwood University. **TinyML: The Future of AI at the Edge**. 2025. <<https://www.birchwoodu.org/tinyml-the-future-of-ai-at-the-edge/>>. Acessado em 12 de outubro de 2025.

BLALOCK, D.; ORTIZ, J. J. G.; FRANKLE, J.; GUTTAG, J. What is the state of neural network pruning? **Proceedings of Machine Learning and Systems**, v. 2, p. 129–146, 2020.

- CRATERE, A.; FARISSI, M. S.; CARBONE, A.; ASCIOLLA, M.; RIZZI, M.; DELL'OLIO, F.; NASCETTI, A.; SPILLER, D. **Efficient FPGA-accelerated Convolutional Neural Networks for Cloud Detection on CubeSats**. 2025. Disponível em: <<https://arxiv.org/abs/2504.03891>>.
- CUGNASCA, C. E. **Aprofundando os conceitos de Sistemas Embarcados**. 2021. <<https://integra.univesp.br/courses/2710/pages/texto-base-aprofundando-os-conceitos-de-sistemas-embarcados-parte-1-%7C-carlos-eduardo-cugnasca>>. Acessado em 09 de setembro de 2025.
- DABWAN, B. A.; JADHAV, M. E.; ISMIL, O. A.; ALTAHIR, E. A.; FARAH, E. A.; MOHAMMAD, A. A.; ALI, Y. A. Detecting gesture language for deaf and mute people using ultra-low-power tinymml model. In: **2024 International Conference on Data Science and Network Security (ICDSNS)**. [S.l.]: IEEE, 2024. p. 1–5.
- ELY, A. F.; BENEVENUTI, F.; KOROL, G.; BECK, A. C. S.; AZAMBUJA, J. R.; KASTENSMIDT, F. L. Image classification cnns using the finn engine for sram-based apsoc in satellite applications. In: **2024 37th SBC/SBMicro/IEEE Symposium on Integrated Circuits and Systems Design (SBCCI)**. [S.l.: s.n.], 2024. p. 1–5.
- GHOLAMI, A.; KIM, S.; DONG, Z.; YAO, Z.; MAHONEY, M. W.; KEUTZER, K. A survey of quantization methods for efficient neural network inference. **arXiv preprint arXiv:2103.13630**, 2021. Disponível em: <<https://arxiv.org/abs/2103.13630>>.
- GONG, T.; JANG, S. Y.; ACER, U. G.; KAWSAR, F.; MIN, C. Collaborative inference via dynamic composition of tiny ai accelerators on mcus. **arXiv preprint arXiv:2401.08637**, 2023.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. [S.l.]: MIT Press, 2016. <<http://www.deeplearningbook.org>>.
- Google Cloud. **Aprendizado supervisionado vs. não supervisionado**. 2024. <<https://cloud.google.com/discover/supervised-vs-unsupervised-learning?hl=pt-BR>>. Acessado em 18 de setembro de 2025.
- Google Cloud. **O que é machine learning?** 2024. <<https://cloud.google.com/learn/what-is-machine-learning?hl=pt-BR>>. Acessado em 18 de novembro de 2025.
- HAMANAKA, F.; ODAN, T.; KISE, K.; CHU, T. V. An exploration of state-of-the-art automation frameworks for fpga-based dnn acceleration. **IEEE Access**, v. 11, p. 5701–5713, 2023.
- HIMEUR, Y.; ABURAED, N.; ELHARROUSS, O.; VARLAMIS, I.; ATALLA, S.; MANSOOR, W.; AHMAD, H. A. Applications of knowledge distillation in remote sensing: A survey. **arXiv preprint arXiv:2409.12111**, 2024.
- HINTON, G.; VINYALS, O.; DEAN, J. Distilling the knowledge in a neural network. In: **NIPS Deep Learning and Representation Learning Workshop**. [s.n.], 2015. Disponível em: <<https://arxiv.org/abs/1503.02531>>.
- HUANG, Q. et al. Knowledge distillation facilitates the lightweight and efficient diagnosis of plant diseases across data scenarios. **Plant Phenomics**, v. 5, p. 0062, 2023.

HWU, W.-m. W.; KIRK, D. B. **Programming Massively Parallel Processors: A Hands-on Approach**. [S.l.]: Morgan Kaufmann, 2022.

Iberdrola. **O que é 'Machine Learning' (aprendizagem automática)?** 2024. <<https://www.iberdrola.com/quem-somos/nosso-modelo-inovacao/o-que-e-machine-learning>>. Acessado em 27 de julho de 2025.

IBM. **O que é aprendizado não supervisionado?** 2024. <<https://www.ibm.com/br-pt/think/topics/unsupervised-learning>>. Acessado em 19 de setembro de 2025.

IBM. **O que é aprendizado por reforço?** 2024. <<https://www.ibm.com/br-pt/think/topics/reinforcement-learning>>. Acessado em 22 de setembro de 2025.

JACOB, B.; KLIGYS, S.; CHEN, B.; ZHU, M.; TANG, M.; HOWARD, A.; ADAM, H.; KALENICHENKO, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: **Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition**. [s.n.], 2018. p. 2704–2713. Disponível em: <https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html>.

KHAN, A. et al. Tinymml: A systematic review and synthesis of existing research. 2024.

KUNG, H. T. Why systolic architectures? **Computer**, v. 15, n. 1, p. 37–46, 1982.

LAI, L.; SUDA, N.; CHANDRA, V. Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus. **arXiv preprint arXiv:1801.06601**, 2018.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, n. 7553, p. 436–444, 2015.

LÉONARD, C.; STOBBER, D.; SCHULZ, M. Fpga-enabled machine learning applications in earth observation: A systematic review. **arXiv preprint arXiv:2506.03938**, 2025.

MACEDO, V. Aplicação de inteligência artificial na detecção de ameaças em redes de computadores. **Revista Difatto**, mar 2025. Acesso em: 11 nov. 2025. Disponível em: <<https://revistadifatto.com.br/artigos/aplicacao-de-aplicacao-de-inteligencia-artificial-na-deteccao-de-ameacas-em-redes-de-computadores/>>.

MOOSMANN, J.; GIORDANO, M.; VOGT, C.; MAGNO, M. Tinyissimoyolo: A quantized, low-memory footprint, tinymml object detection network for low power microcontrollers. **arXiv preprint arXiv:2306.00001**, 2023.

MOOSMANN, J.; MÜLLER, H.; ZIMMERMAN, N.; RUTISHAUSER, G.; BENINI, L.; MAGNO, M. Flexible and fully quantized lightweight tinyissimoyolo for ultra-low-power edge systems. **IEEE Access**, v. 12, p. 75093–75107, 2024.

NICKOLLS, J.; DALLY, W. J. The gpu computing era. **IEEE Micro**, v. 30, n. 2, p. 56–69, 2010.

NIELSEN, M. A. **Neural Networks and Deep Learning**. 2015. <<http://neuralnetworksanddeeplearning.com/>>. Determination Press.

NVIDIA. **Jetson Nano Developer Kit User Guide**. 2019.

NVIDIA. **NVIDIA Jetson Documentation**. 2024. Disponível em: <<https://developer.nvidia.com/embedded/jetson-developer-kits>>. Acesso em: 12 de setembro de 2025.

Oracle. **O que é aprendizado por reforço?** 2024. <<https://www.oracle.com/br/artificial-intelligence/machine-learning/reinforcement-learning/>>. Acesso em 09 de setembro de 2025.

PATHAK, A.; BHAT, Z. P.; WANG, G.; JIANG, Z.; FAN, M.; WEI, R.; ZHA, D.; REYES, A. C.; CHEN, Y.-W.; ZHANG, C. Bed: An object detection system for edge devices. In: **2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)**. [S.l.: s.n.], 2022. p. 384–384.

PEDRO, O. A. F. **Aprendizado por Reforço: Ensinando Máquinas a Aprender com as Experiências da Vida**. 2024. <<https://medium.com/@otvioafpedro/aprendizado-por-refor%C3%A7o-ensinando-m%C3%A1quinas-a-aprender-com-as-experi%C3%Aancias-da-vida-c87e8e6fc7d5>>. Acesso em 02 de setembro de 2025.

SHI, W.; CAO, J.; ZHANG, Q.; LI, Y.; XU, L. Edge computing: Vision and challenges. **IEEE Internet of Things Journal**, v. 3, n. 5, p. 637–646, 2016.

SINGH, R. et al. Edge ai: A survey. **AI Open**, v. 4, p. 97–124, 2023.

SZE, V.; CHEN, Y.-H.; YANG, T.-J.; EMER, J. S. Efficient processing of deep neural networks: A tutorial and survey. **Proceedings of the IEEE**, v. 105, n. 12, p. 2295–2329, 2017.

TIWARI, L. **Generative AI for Visual Content Creation: Image, Video, and 3D: Generative Adversarial Network (GAN)**. 2024. Slides da disciplina COV877 – Special Module on Visual Computing. Acesso em: 5 jun. 2026. Disponível em: <<https://lokender.github.io/teaching/slides/COV877/Lecture-1-GAN.pdf>>.

ULKAR, M. G.; OKMAN, O. E. **Ultra-Low Power Keyword Spotting at the Edge**. 2021. Disponível em: <<https://arxiv.org/abs/2111.04988>>.

UMUROGLU, Y.; FRASER, N. J.; GAMBARDELLA, G.; BLOTT, M.; LEONG, P.; JAHRE, M.; VISSERS, K. Finn: A framework for fast, scalable binarized neural network inference. In: **Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays**. [S.l.: s.n.], 2017. p. 65–74.

VIEL, F. **HAViTHI: Acelerador em Hardware Baseado em Vision Transformer para Classificação de Imagens Hiperespectrais**. Tese (Tese de doutorado) — Universidade Federal de Santa Catarina, Florianópolis, 2023.

WANG, G.; BHAT, Z. P.; JIANG, Z.; CHEN, Y.-W.; ZHA, D.; REYES, A. C.; NIKTASH, A.; ULKAR, G.; OKMAN, E.; CAI, X.; HU, X. Bed: A real-time object detection system for edge devices. In: **Proceedings of the 31st ACM International Conference on Information & Knowledge Management**. [s.n.], 2022. Disponível em: <<https://doi.org/10.1145/3511808.3557168>>.

WARDEN, P.; SITUNAYAKE, D. **TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers**. Sebastopol: O'Reilly Media, 2019.

ZHANG, X. et al. Compressing recognition network of cotton disease with knowledge distillation. **Frontiers in Plant Science**, v. 15, p. 1433543, 2024.